

УЧЕБНИК ДЛЯ ТЕХ, КТО
НИКОГДА НЕ ПРОГРАММИРОВАЛ

Сайт с нуля

Как собрать интернет-магазин
и стать fullstack-разработчиком

60 глав · практика на живом коде

autodoc.tj

Содержание

- | | |
|----|--|
| 1 | Что такое сайт и что мы построим |
| 2 | Как работает интернет |
| 3 | Инструменты: редактор, терминал, браузер |
| 4 | Первый файл на экране |
| 5 | Теги и структура документа |
| 6 | Текст, ссылки, картинки, списки |
| 7 | Таблицы и формы |
| 8 | Практика: карточка товара и каталог |
| 9 | Подключаем CSS и учим селекторы |
| 10 | Цвет, шрифт, отступы, рамки |
| 11 | Flexbox и Grid: раскладка |
| 12 | Адаптивность: сайт на телефоне |
| 13 | Практика: каталог становится красивым |
| 14 | JavaScript: переменные, условия, циклы |
| 15 | Функции, массивы, объекты |
| 16 | DOM: меняем страницу из кода |
| 17 | События: реакция на действия |
| 18 | Практика: живой поиск и корзина |
| 19 | Что делает PHP и первый скрипт |
| 20 | Переменные, типы и вывод |

- 21** Условия и циклы

- 22** Массивы

- 23** Функции, include и require

- 24** Формы: GET и POST

- 25** Практика: каталог на PHP

- 26** Зачем нужна база данных

- 27** SELECT: достаём данные

- 28** INSERT, UPDATE, DELETE

- 29** JOIN: связываем таблицы

- 30** Индексы и почему всё тормозит

- 31** Практика: схема базы магазина

Книга пишется по частям. Здесь — главы, готовые на сегодня.

Что такое сайт и что мы построим

ЗАЧЕМ ЭТО

Зачем эта глава

Здравствуйте. Если вы открыли эту книгу, скорее всего, вы уже пробовали подступиться к программированию — и что-то пошло не так. Так бывает почти у всех, и дело обычно не в вас.

Смотрите, что происходит на типичных курсах. Первые две недели вам рассказывают про «переменные», «циклы» и «типы данных». Всё это правда нужно. Но вы сидите и думаете: *а где, собственно, сайт?* Знания вроде появляются, а куда их приложить — непонятно. Это как учиться водить, изучая устройство карбюратора: полезно, но до машины вы так и не дошли.

Поэтому первая глава — не про код вообще. Она про **карту местности**.

Мы разберёмся, из каких частей состоит любой сайт, кто эти части делает и как они между собой связаны. И когда в главе 20 появятся те самые переменные, вы уже будете точно знать, в каком месте магазина они живут и зачем нужны.

Никакого кода набирать пока не надо. Просто читайте — как будто вам показывают дом перед тем, как дать в руки инструменты.

РАЗБИРАЕМСЯ

Начнём с обычного магазина

Забудьте на минуту про компьютеры. Правда, совсем забудьте.

Представьте магазин автозапчастей на вашей улице. Тот самый, куда вы заходили за щётками стеклоочистителя. Что там есть?

Во-первых, торговый зал. Полки, товар, ценники, витрина у входа, касса. Всё разложено так, чтобы покупатель разобрался сам: колодки — сюда, фильтры — туда, акция — на видном месте. Здесь красиво и понятно. Покупатель дальше этой зоны не заходит.

Во-вторых, подсобка. Дверь с табличкой «Посторонним вход воспрещён». Там принимают товар, считают выручку, оформляют накладные, решают, кому дать скидку, а кому нет. Там же лежат деньги. Покупателю туда нельзя — и это не из вредности, а просто потому, что иначе цены на ценниках начнут переписывать все кому не лень.

В-третьих, складской журнал. Толстая тетрадь или программа на компьютере, где записано: какой товар, сколько штук, почём куплено, почём продаём, кто привёз, кому отгрузили. Скудная штука. Но потеряйте её — и магазин превратится в склад коробок, про которые никто ничего не знает.

Всё. Три зоны. Держите эту картинку в голове — сейчас будет главное.

► Сайт устроен ровно так же

Не «похоже». Не «примерно». А один в один:

В МАГАЗИНЕ НА УЛИЦЕ	НА САЙТЕ	КАК ЭТО ЗОВУТ ПРОГРАММИСТЫ
Торговый зал	То, что видно в браузере	Фронтенд (frontend — «передняя часть»)
Подсобка	Программа на сервере	Бэкенд (backend — «задняя часть»)
Складской журнал	База данных	БД (database)

Когда вы заходите на любой сайт — маркетплейс, банк, доставку еды — вы всегда стоите в торговом зале. Всё остальное происходит за дверью, куда вас не пускают.

А человек, который умеет работать **во всех трёх зонах**, называется **fullstack-разработчик**. Дословно — «полный набор». Именно им вы и станете к концу книги.

► Ещё немного про эти три зоны

Чтобы совсем закрепилось, вот та же тройка на других примерах.

Ресторан. Зал со столиками и меню — фронтенд. Кухня, где готовят и считают себестоимость, — бэкенд. Список продуктов на складе и книга заказов — база данных.

Поликлиника. Регистратура и коридор с креслами — фронтенд. Кабинет врача, где принимают решения, — бэкенд. Картотека с медкартами — база данных.

Такси в телефоне. Карта с машинками и кнопка «Заказать» — фронтенд. Программа, которая ищет ближайшего водителя и считает цену, — бэкенд. Хранилище поездок, водителей и оценок — база данных.

Заметили? Везде одно и то же: **красивая витрина, скрытая от глаз логика и место, где всё записано.** Любой сайт, любое приложение, любой сервис.

РАЗБИРАЕМСЯ

Пять инструментов, и всё

Хорошая новость: чтобы построить магазин, нужно не пятьдесят технологий. Нужно **пять**. Давайте познакомимся с каждой — с примерами, чтобы не осталось тумана.

► HTML — скелет

HTML отвечает на вопрос «что здесь есть?». Не «как выглядит», а именно «что есть».

Он говорит браузеру: тут заголовок, тут абзац текста, тут картинка, тут кнопка, тут список из трёх пунктов, тут таблица. И всё — на этом его работа заканчивается. Красотой он не занимается принципиально.

Примеры того, что делает HTML:

- на странице появился заголовок «Тормозные колодки»;
- под ним — абзац с описанием;
- слева — фотография товара;
- ниже — список характеристик: производитель, артикул, вес;
- в конце — кнопка «Купить».

Аналогия: скелет человека. Есть череп, рёбра, руки, ноги. На свидание в таком виде не пойдёшь, но сразу понятно: это человек, вот голова, вот конечности.

Ещё точнее — **чертёж дома**. На чертеже нет обоев и мебели, но видно: тут комната, тут окно, тут дверь.

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Это уже настоящая, рабочая страница. Уродливая до слёз — но рабочая. Браузер её откроет.

► CSS — внешность

CSS отвечает на вопрос «а как это выглядит?».

Тот же самый скелет, но теперь у него появились цвет, шрифт, размер, отступы, тени, скруглённые углы и понятное расположение на экране.

Примеры того, что делает CSS:

- кнопка стала красной, а буквы на ней белыми;
- заголовок — крупный и жирный, описание — помельче и посветлее;
- между карточками товаров появились ровные промежутки;
- товары выстроились в три колонки вместо кучи;
- на телефоне те же товары встали в одну колонку;
- при наведении мышкой карточка чуть приподнялась.

Аналогия: одежда, причёска и осанка. Скелет тот же, а впечатление совсем другое.

Ещё точнее — **ремонт в квартире**. Планировка не изменилась: те же комнаты, те же окна. Но после ремонта это другое жильё.

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

Одна строчка — и серая системная кнопка превратилась в красную кнопку с мягкими углами.

► JavaScript — рефлексы браузера

JavaScript отвечает на вопрос «что произойдёт, если человек что-то сделает?» — причём прямо здесь, в браузере, без перезагрузки страницы.

Примеры того, что делает JavaScript:

- нажали «Купить» — счётчик корзины мгновенно стал «3»;
- набрали в поиске «коло» — снизу выпали подсказки: «колодки», «кольцо», «коллектор»;
- кликнули на фото — оно открылось на весь экран;
- на телефоне нажали три полоски — выехало меню;
- начали заполнять форму и забыли телефон — поле подсветилось красным ещё до отправки;
- пролистали страницу вниз — шапка «прилипла» к верху экрана.

Аналогия: рефлекс. Дотронулись до горячего — рука отдёрнулась сама, мозг ещё даже не понял, что случилось.

Ещё точнее — **автоматические двери в магазине**. Никто не звонит директору, чтобы вам открыли. Датчик увидел человека — двери разъехались.

```
button.onclick = () => alert('Товар добавлен в корзину');
```

► PHP — мозг на сервере

А вот это самое важное место, и здесь нужно быть внимательным.

PHP работает **не в браузере**. Он живёт на **сервере** — на чужом компьютере, который стоит где-то в дата-центре и работает круглосуточно. Покупатель до него не дотягивается в принципе.

PHP принимает все серьёзные решения.

Примеры того, что делает PHP:

- проверяет, правильный ли пароль ввели при входе;
- решает, показать человеку кнопку «Админка» или не показывать;
- считает итоговую цену: цена поставщика + наценка + доставка;
- применяет скидку постоянному покупателю — или не применяет;
- записывает заказ и присваивает ему номер;
- отправляет письмо «ваш заказ принят»;
- не даёт продавцу Ахмаду редактировать товары продавца Сафара.

Аналогия: мозг. Или директор магазина — тот, кто решает, а не тот, кто улыбается в зале.

А теперь — правило, которое стоит запомнить прямо сейчас.

JavaScript выполняется **на компьютере покупателя**. Это значит, что покупатель может его открыть, почитать и переписать. Это не сложно — любой браузер позволяет.

PHP выполняется **на вашем сервере**. Покупатель его не видит и изменить не может.

Представьте, что цена товара считается на JavaScript. Тогда покупатель открывает инструменты браузера, меняет 250 на 1 — и оформляет заказ на сомони. Это как если бы в магазине разрешили покупателям самим переклеивать ценники.

Поэтому **цены, скидки, доступы и всё, что связано с деньгами, считает только PHP.**

Никогда — JavaScript. Мы к этому ещё вернёмся не раз, но услышать это лучше в первой главе.

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
}
```


► SQL — разговор с памятью

SQL — это язык, на котором разговаривают с базой данных. Спрашивают и записывают.

Примеры того, что делает SQL:

- найти все тормозные колодки дешевле 300 сомони, которые есть в наличии;
- достать данные одного товара по его номеру;
- добавить новый заказ;
- изменить статус заказа с «новый» на «отправлен»;
- посчитать, на какую сумму продал каждый продавец за месяц;
- найти покупателей, которые не заходили полгода.

Аналогия: разговор с кладовщиком. Вы не идёте на склад сами и не роетесь в коробках. Вы говорите: «дай мне все колодки Bosch, что есть в наличии» — и он приносит список.

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

► Коротко, чтобы уложилось

ИНСТРУМЕНТ	ОТВЕЧАЕТ НА ВОПРОС	ГДЕ РАБОТАЕТ	БЫТОВАЯ АНАЛОГИЯ
HTML	Что здесь есть?	В браузере	Скелет, чертёж дома
CSS	Как это выглядит?	В браузере	Одежда, ремонт
JavaScript	Что будет, если нажать?	В браузере	Рефлексы, автодвери
PHP	Что решаем и что показываем?	На сервере	Мозг, директор
SQL	Где это записано?	В базе данных	Кладовщик, картотека

Если сейчас каша в голове — это абсолютно нормально. Никто не запоминает пять новых понятий с первого раза. Мы к каждому вернёмся отдельной главой, и не по одному разу.

Выше промелькнуло пять кусочков кода, и они наверняка выглядели как шифр. Давайте расшифруем — **каждый** значок.

Запоминать ничего не нужно. Просто прочитайте один раз, чтобы код перестал пугать. Это как впервые посмотреть на ноты: сначала мурашки, потом оказывается, что там всего семь букв.

► HTML: почему всё в «уголках»

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
< и >	«уголки», угловые скобки	Внутри уголков — команда браузеру , а не текст для чтения. На экране их не будет
<h1>	«эйч-один»	Заголовок первого уровня , самый главный на странице. От английского <i>heading</i> — заголовок. Бывают <h1> ... <h6> : чем больше цифра, тем мельче заголовок
</h1>	«закрывающий эйч-один»	Косая черта / означает конец . «Заголовок закончился вот здесь»
<p>	«пи»	Абзац обычного текста. От <i>paragraph</i>
<button>	«баттон»	Кнопка , на которую можно нажать
Текст между тегами		То единственное, что реально увидит человек на экране

Пара «открывающий тег + содержимое + закрывающий тег» называется **элемент**.

Правило простое, как в математике: **что открыли — то надо закрыть**. Забыли закрыть — браузер начнёт фантазировать, и страница поедет.

А что такое `<div>` ?

Это слово вы будете встречать чаще всех остальных, поэтому знакомимся сразу.

`<div>` — это **пустая коробка**. Он ничего не означает и никак не выглядит. На экране его не видно вообще.

Зачем тогда нужен? Чтобы **сложить в него другие элементы** и потом обращаться ко всей группе сразу.

```
<div>
  <h1>Тормозные колодки</h1>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь заголовок, цена и кнопка лежат в одной коробке. И можно сказать: «вот эту коробку сделай белой, добавь рамку, скругли углы и отодвинь от соседних». Получится **карточка товара** — та самая, из которых собран любой каталог.

Название `div` — сокращение от английского *division*, «раздел», «отделение». Думайте о нём как о коробке из-под обуви: сама по себе не нужна, но сложить в неё вещи и подписать — очень удобно. Подробно разберём в главе 5.

► CSS: почему фигурные скобки

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>	селектор	Кого красим. Здесь — все кнопки на странице
<code>{ }</code>	фигурные скобки	«Начало и конец списка указаний». Всё внутри относится к кнопке
<code>background</code>	свойство	Что меняем — цвет фона
<code>:</code>	двоеточие	Разделитель: слева что меняем, справа на что
<code>#d92b2b</code>	значение	Цвет в виде кода: <code>#</code> и шесть символов. Здесь — красный
<code>;</code>	точка с запятой	Конец одного указания. Дальше пойдёт следующее
<code>color: white</code>		Цвет текста — белый. Не путайте с <code>background</code> : это фон
<code>border-radius: 8px</code>		Скруглить углы на 8 пикселей

Формула CSS всегда одна и та же:

кого `{` что меняем `:` на что `;` `}`

Выучите её один раз — и любой CSS в мире станет читаемым.

► JavaScript: точки и стрелки

```
button.onclick = () => alert('Товар добавлен в корзину');
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>		Кнопка, которую мы до этого нашли на странице
<code>.</code>	точка	«Возьми у этого вот это». Как «Иван ^{***} . ^{***} телефон» — телефон Ивана
<code>onclick</code>	«он-клик»	Свойство «что делать при нажатии». <i>on click</i> — «по щелчку»
<code>=</code>	равно, присваивание	Положи в левое то, что справа. Это не «равенство» из математики, а команда «запиши»
<code>() =></code>	«стрелочная функция»	«Вот действие , которое надо выполнить». В круглых скобках — место для входных данных, здесь они не нужны
<code>alert(...)</code>	«алерт»	Готовая команда браузера: показать всплывающее окошко
<code>'...'</code>	кавычки	Внутри кавычек — текст , а не команда. Компьютер не пытается его выполнить
<code>;</code>	точка с запятой	Конец строки-команды

► PHP: доллары и условия

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
}
```

Вот здесь появилось `if` — пожалуй, самое важное слово во всём программировании. Оно есть в каждом языке, и работает везде одинаково.

ЧТО ВИДИМ КАК ЧИТАЕТСЯ ЧТО ЭТО ЗНАЧИТ

<code>if</code>	«иф», по-английски «если»	Проверка. «Если условие верно — сделай то, что в фигурных скобках. Если нет — просто пропусти»
<code>()</code>	круглые скобки после <code>if</code>	Внутри — условие , которое проверяем
<code>\$</code>	доллар	В PHP так помечают переменную — коробочку с данными. <code>\$user</code> — «коробочка с пользователем»
<code>\$user['role']</code>		Достать из коробочки <code>\$user</code> поле <code>role</code> — роль. Квадратные скобки: «возьми вот эту часть»
<code>===</code>	«строго равно»	Сравнение: «а точно ли слева то же самое, что справа?» Три знака, не один
<code>'admin'</code>		Текст, с которым сравниваем
<code>{ }</code>	фигурные скобки	Тело условия — что делать, если проверка прошла
<code>echo</code>	«эхо»	Команда PHP: выведи это на страницу
<code>;</code>		Конец команды

Один `=` или три `===`? Самая частая ошибка новичка.

- `=` — это **записать**. `$x = 5` значит «положи пятёрку в коробочку x».
- `===` — это **спросить**. `$x === 5` значит «а в коробочке правда пятёрка?»

Первое — действие, второе — вопрос. Перепутаете — программа вместо проверки молча запишет значение, и ошибку вы будете искать долго.

А где же `else`?

`else` — «иначе» — это вторая половина `if`. Читается ровно как в жизни:

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
} else {
    echo 'Вам сюда нельзя';
}
```

Дословно: **если** роль администратор — покажи приветствие, **иначе** — откажи. Одно из двух выполнится обязательно, третьего не дано.

Вы пользуетесь этой конструкцией каждый день, просто не называете её так:

- **если** дождь — беру зонт, **иначе** — не беру;
- **если** денег хватает — покупаю, **иначе** — откладываю;
- **если** на светофоре зелёный — иду, **иначе** — стою.

Вот и всё программирование. Подробно — в главе 21.

► SQL: запрос как обычная фраза

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

SQL специально сделали похожим на английское предложение. Читайте по кусочкам:

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
SELECT	«селект» — выбрать	Какие колонки нам нужны
name, price		Название и цена. Запятая — «и ещё»
FROM	«фром» — из	Из какой таблицы берём
products		Таблица товаров
WHERE	«вэа» — где, при условии	Какие строки берём, а какие пропускаем
price < 300		Цена меньше 300
AND	«энд» — и	Оба условия должны выполняться одновременно
in_stock = 1		Есть в наличии (1 — да, 0 — нет)
;		Конец запроса

Целиком получается: «Выбери название и цену из таблицы товаров, где цена меньше 300 и товар в наличии». Практически человеческая фраза.

► Что общего у всех пяти языков

А теперь приятное. Присмотритесь к таблицам выше — значки-то повторяются:

ЗНАЧОК

ПОЧТИ В ЛЮБОМ ЯЗЫКЕ ОЗНАЧАЕТ

`{ }`

«блок»: вот начало группы, вот конец

`( )`

«сюда кладут данные»: условие или входные значения

`;`

«команда закончилась»

`' '` или `" "`

«внутри текст, выполнять его не надо»

`//` или `#`

комментарий: заметка для человека, компьютер её не читает

Разобравшись один раз, вы будете узнавать их везде — хоть в Python, хоть в Java, хоть в языке, который придумают через десять лет. Языков много, а привычки у них общие.

РАЗБИРАЕМСЯ

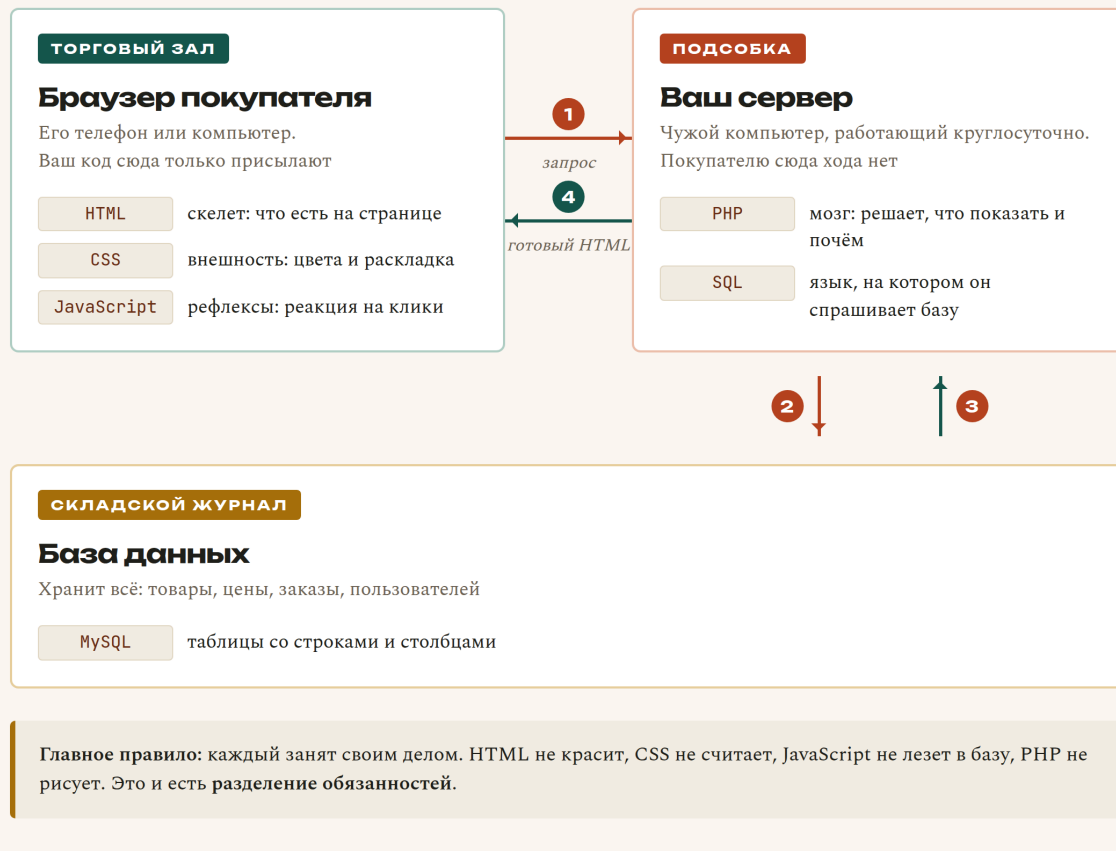
А теперь всё вместе

Давайте проследим, что происходит, когда покупатель открывает страницу товара. Прочитайте медленно — это самая важная схема в книге, всё остальное будет крутиться вокруг неё.

1. Покупатель нажимает ссылку «Тормозные колодки».
2. Браузер отправляет на сервер запрос: «дайте мне страницу товара номер 42».
3. На сервере просыпается PHP и думает: «сорок второй, значит. Надо узнать подробности».
4. PHP составляет запрос на SQL и отправляет его в базу данных: `SELECT * FROM products WHERE id = 42`.
5. База отвечает: «Колодки тормозные, Bosch, 250 сомони, на складе 7 штук».
6. PHP берёт эти данные и строит из них HTML: название вставляет в заголовок, цену — в ценник, картинку — в блок картинки.
7. Готовый HTML улетает обратно в браузер.
8. Браузер читает HTML, применяет CSS — и страница из чёрно-белого текста превращается в нормальный магазин.
9. Подключается JavaScript и начинает следить за кнопкой «Купить».

Что происходит, когда покупатель открывает страницу товара

Три зоны, пять технологий — и меньше секунды на всё



Всё это заняло меньше секунды. Пять технологий, и каждая сделала ровно свою часть работы.

Разделение обязанностей — вот главная идея всей веб-разработки. HTML не красит. CSS не считает. JavaScript не лезет в базу. PHP не рисует. Каждый занят своим делом и не мешает соседям.

Звучит как формальность, но именно поэтому сайты вообще можно чинить. Сломалась внешность — идёте в CSS и точно знаете, что база данных тут ни при чём.

РАЗБИРАЕМСЯ

Что именно мы построим

Не «примерчик на три кнопки». Настоящий интернет-магазин — такой, каким пользуются живые люди и в котором ходят реальные деньги.

Для покупателя:

- каталог запчастей с поиском по названию и по артикулу;
- фильтры: бренд, цена, наличие;
- карточка товара с фото, описанием и характеристиками;
- корзина: добавить, изменить количество, удалить;
- оформление заказа с адресом и телефоном;
- личный кабинет с историей заказов и их статусами.

Для продавца — потому что магазин будет не только ваш, а маркетплейс:

- регистрация продавца и его страница;
- выкладка своих товаров;
- свои заказы и работа с ними;
- баланс: сколько заработано и сколько уже выплачено.

Для администратора:

- модерация: проверять товары продавцов до публикации;
- управление всеми заказами и статусами;
- расчёт комиссии магазина;
- реестр выплат продавцам.

И взрослые вещи, которых обычно нет в учебниках:

- подключение внешнего API поставщика запчастей;
- пересчёт цены из рублей в сомони по живому курсу, плюс наценка и доставка;
- защита от взлома: SQL-инъекции, XSS, кража сессии;
- выкладка сайта на настоящий хостинг, под настоящим доменом, с замочком HTTPS.

Дойти до конца — это работа на несколько месяцев. Но каждая глава заканчивается чем-то, что можно открыть в браузере и показать. Ощущение «у меня получилось» будет приходить регулярно, а не один раз в самом конце.

РАЗБИРАЕМСЯ

Почему PHP, а не что-то модное

Отвечу честно, потому что вопрос вы всё равно услышите. В интернете любят повторять, что «PHP устарел».

PHP выбран не потому, что он самый модный. А потому, что для **первого** сайта он самый удобный:

- **Результат виден сразу.** Создали файл, открыли в браузере — работает. Не нужно ставить пятьдесят пакетов и настраивать сборщик, чтобы вывести на экран слово «привет».
- **Дешёвый хостинг.** PHP работает на хостинге за 3–5 долларов в месяц. Для первого настоящего проекта это важно: сайт можно показать людям, не разорившись.
- **Огромная база готовых ответов.** Какую бы проблему вы ни встретили, её уже кто-то решил и описал.
- **На нём работает больше половины сайтов в мире.** Один только WordPress — это примерно четверть всего интернета.

Но главное вот что. **Навык — не в языке.**

Когда вы научитесь думать «пришёл запрос → проверили права → сходили в базу → собрали ответ», вы пересядете на Python или Node.js за пару недель. Синтаксис разный, мышление одно и то же.

Языки приходят и уходят. Умение раскладывать задачу на части остаётся.

ГРАБЛИ

Грабли

Четыре мысли, которые чаще всего мешают новичкам. Если поймаете себя на любой из них — вспомните эту страницу.

«**Сначала выучу всё, потом начну делать**». Не сработает, проверено многими. Знания без практики выветриваются за неделю. Правильный порядок обратный: делаем маленькую вещь → упираемся в незнание → узнаём ровно столько, сколько нужно прямо сейчас → делаем дальше. Скучно, зато держится.

«**Надо сразу выбрать самый лучший язык**». Лучшего языка не существует, есть подходящий для задачи. Споры «PHP против Python» для новичка — это спор о том, каким молотком забивать первый в жизни гвоздь. Забейте хоть каким, потом разберётесь.

«**Настоящие программисты знают всё наизусть**». Нет. Разработчик с пятнадцатью годами опыта гуглит синтаксис по десять раз в день, и это нормально. Ценность не в памяти, а в умении разбить задачу на части и понять, что именно искать.

«**Я не математик, значит, не потяну**». Для веб-разработки нужна арифметика уровня седьмого класса: сложить цены, посчитать проценты, поделить на количество. Всё. Высшая математика понадобится, если вы пойдёте в машинное обучение — но это совсем другая история.

Пять упражнений. Ни в одном не нужен компьютер с редактором — только голова и браузер. Не пропускайте: именно на задачах прочитанное превращается в понятное.

Задача 1.1. Откройте любой знакомый сайт — маркетплейс, новости, банк. Выпишите пять вещей, которые вы **видите** (это работа фронтенда). Потом пять вещей, которые происходят **невидимо**: проверка пароля, расчёт доставки, сохранение заказа (это бэкенд).

Задача 1.2. Для каждой строки определите, кто отвечает — HTML, CSS, JavaScript, PHP или SQL:

1. Кнопка стала синей вместо серой.
2. На странице появился второй заголовок.
3. При наборе букв в поиске выпадают подсказки.
4. Проверка, что пароль при входе правильный.
5. Достать из базы 20 самых дешёвых товаров.
6. Меню на телефоне схлопнулось в три полоски.
7. Заказ записался, и ему присвоился номер.
8. Между карточками товаров появились отступы.

Задача 1.3. Объясните вслух — правда вслух, это работает — своими словами разницу между фронтендом и бэкендом. Уложитесь в два предложения и не используйте сами слова «фронтенд» и «бэкенд».

Задача 1.4. Почему скидку постоянного покупателя нельзя считать на JavaScript? Ответ — одна строчка.

Задача 1.5. Нажмите на любом сайте правую кнопку мыши и выберите «Посмотреть код страницы». Откроется HTML, который прислал сервер. Найдите в нём заголовок страницы (ищите `<h1>` или `<title>`). Если сейчас это выглядит как сплошная каша — так и должно быть. К главе 8 вы будете читать её свободно.

Задача 1.6. Придумайте три своих аналогии для тройки «витрина — подсобка — журнал». Не из книги, а свои: школа, аэропорт, парикмахерская — что угодно. Чем больше своих примеров, тем прочнее держится в голове.

Ответы — в Приложении Г.

В БОЮ В бою

А теперь заглянем в настоящий проект. В корне этого репозитория лежит код работающего магазина autodoc.tj. Открывать файлы пока не нужно — просто посмотрите на названия папок. Вы уже можете угадать, что где:

ПАПКА	ЧТО ВНУТРИ	КАКАЯ ЭТО ЗОНА
assets/css/	Файлы оформления	Фронтенд (CSS)
pages/	Страницы сайта	Фронтенд + бэкенд
includes/	Общая логика, которую используют все страницы	Бэкенд (PHP)
sql/	Структура базы данных	База данных
admin/	Админка	Бэкенд, «подсобка»
seller/	Кабинет продавца	Бэкенд, «подсобка»
api/	Точки, куда обращается JavaScript	Стык фронта и бэка

Узнаёте? Это тот самый магазин из начала главы. Торговый зал, подсобка и складской журнал — только разложенные по папкам.

Через шестьдесят глав вы откроете любой из этих файлов и будете понимать, что там написано, строчка за строчкой.

- Сайт — это **торговый зал** (фронтенд), **подсобка** (бэкенд) и **складской журнал** (база данных). Всегда, у любого сайта.
- **HTML** — скелет, **CSS** — внешность, **JavaScript** — рефлекс в браузере, **PHP** — мозг на сервере, **SQL** — разговор с памятью.
- JavaScript выполняется у покупателя и переписывается им за минуту. PHP выполняется у вас. Поэтому деньги и доступы считает **только PHP**.
- `if` — это «если», `else` — «иначе». `=` записывает, `===` спрашивает. `<div>` — пустая коробка для группировки.
- **Fullstack** — тот, кто работает во всех трёх зонах. Это вы через шестьдесят глав.
- Учиться нужно от задачи к теории, а не наоборот.

В следующей главе разберёмся, что вообще происходит между моментом, когда вы вводите адрес сайта, и моментом, когда на экране появляется картинка. Спойлер: там интересно.

Как работает интернет

ЗАЧЕМ ЭТО**Зачем эта глава**

Вы вводите в браузере адрес, жмёте Enter — и через полсекунды видите страницу. Кажется, что там нечего объяснять: нажал и появилось.

На самом деле за эти полсекунды происходит примерно десять отдельных событий, и каждое из них однажды у вас сломается. Сайт не открывается, картинки не грузятся, форма отпавилась «в никуда», на телефоне работает, а на компьютере нет.

Человек, который не понимает эту цепочку, в такие моменты просто разводит руками. Человек, который понимает, спокойно спрашивает себя: «так, а на каком шаге сломалось?» — и находит причину за пять минут.

Поэтому потратим главу на то, что происходит между вашим Enter и картинкой на экране. Кода снова не будет. Будет понимание, без которого дальше всё превратится в магию.

РАЗБИРАЕМСЯ**Интернет — это почта**

Самое вредное представление об интернете — «где-то там облако, и в нём всё лежит». Давайте заменим его на правильное.

Интернет — это почтовая служба. Огромная, быстрая, но обычная почта.

Есть отправитель. Есть получатель с адресом. Есть письмо с вопросом и письмо с ответом. Есть курьеры, которые везут письма по маршруту. Всё, никакой магии.

Когда вы открываете сайт, происходит ровно это:

1. Вы (браузер) пишете письмо: «здравствуйте, пришлите мне, пожалуйста, главную страницу».
2. Письмо едет по адресу.
3. Там его читает сервер и готовит ответ.
4. Ответ приезжает обратно.
5. Браузер разворачивает конверт и показывает содержимое.

Одно письмо туда, одно обратно. Это называется **запрос и ответ** (по-английски *request* и *response*). Запомните эту пару — она будет встречаться в книге сотни раз.

Что происходит между вашим Enter и картинкой на экране

Одно письмо туда, одно обратно — и десяток шагов между ними

- 1 Разбирает адрес** **БРАУЗЕР**
Делит `https://autodoc.tj/catalog` на протокол, домен и путь
- 2 Спрашивает номер дома** **DNS**
«Какой IP у autodoc.tj?» — «185.12.94.7». Справочник интернета
- 3 Стучится и договаривается о шифровании** **БРАУЗЕР**
Тот самый замочек в адресной строке появляется здесь
- 4 Отправляет запрос** **БРАУЗЕР**
`GET /catalog HTTP/1.1` — «покажи, пожалуйста, каталог»
- 5 Думает** **СЕРВЕР**
PHP идёт в базу через SQL, забирает товары и собирает HTML. 20–200 мс
- 6 Отвечает** **СЕРВЕР**
`200 OK` и готовый HTML. Или `404`, если такого адреса нет
- 7 Рисует страницу и просит остальное** **БРАУЗЕР**
За каждым стилем, скриптом и картинкой уходит отдельный запрос. Их бывает 30–80

Где обычно ломается: шаг 2 — домен куплен, но не настроен, сайт «не открывается». Шаг 5 — ошибка в вашем коде, приходит 500. Шаг 6 — опечатка в адресе, приходит 404.

РАЗБИРАЕМСЯ

Клиент и сервер

Два слова, которые нужно развести раз и навсегда.

Клиент — тот, кто просит. В нашем случае это **браузер**: Chrome, Safari, Firefox, Яндекс.Браузер — любой. Он живёт на вашем телефоне или компьютере.

Сервер — тот, у кого просят. Это компьютер, который включён круглосуточно и умеет отвечать на письма.

И вот тут первое открытие для многих: **сервер** — это обычный компьютер. Не волшебный ящик. У него есть процессор, память, диск и операционная система. Отличий всего три:

- он не выключается;
- у него нет монитора и клавиатуры — им управляют по сети;
- он стоит не дома, а в дата-центре, где хороший интернет и не пропадает электричество.

Ваш ноутбук тоже может стать сервером. Мы это сделаем уже в главе 4: запустим на нём маленький сервер и откроем собственный сайт. Никакой мистики.

Кстати, «клиент» и «сервер» — это роли, а не предметы. Один и тот же компьютер может быть сервером для одних и клиентом для других. Наш магазин будет сервером для покупателей и клиентом, когда пойдёт спрашивать цены у поставщика.

РАЗБИРАЕМСЯ

Что происходит, когда вы жмёте Enter

Теперь подробно. Вы набрали `autodoc.tj` и нажали Enter. Поехали.

► Шаг 1. Браузер разбирает адрес

Адрес сайта — это не одно слово, а несколько частей. Возьмём полный вид:

```
https://autodoc.tj/catalog/brakes?page=2
```

КУСОК	КАК НАЗЫВАЕТСЯ	ЧТО ЗНАЧИТ
<code>https</code>	протокол	На каком языке разговаривать. <code>https</code> — «по-человечески, но зашифрованно»
<code>://</code>	разделитель	Просто знак, что дальше идёт адрес
<code>autodoc.tj</code>	домен	Имя дома, куда едем
<code>/catalog/brakes</code>	путь	Какая именно комната в доме нужна
<code>?page=2</code>	параметры	Уточнение: «вторая страница»

Браузер это всё раскладывает и понимает, куда собирается.

► Шаг 2. Браузер узнаёт настоящий адрес — DNS

Вот тут интересное. Компьютеры не умеют находить друг друга по именам вроде `autodoc.tj`. Они работают с числами — IP-адресами, например `185.12.94.7`.

Имя нужно людям. Число нужно машинам.

Поэтому существует DNS — телефонный справочник интернета. Браузер спрашивает: «какой номер у `autodoc.tj`?» — и получает: «185.12.94.7».

Аналогия проще некуда. Вы же не помните номер телефона мамы — вы открываете контакты и жмёте «Мама». Контакты сами подставляют цифры. DNS — это контакты интернета.

⚠ Почему это важно знать: если сайт «не открывается», очень часто виноват именно DNS. Домен купили, но не настроили — и справочник просто не знает, куда вас отправить. В главе 58 мы это настроим руками.

► Шаг 3. Браузер стучится в дверь

Зная IP-адрес, браузер устанавливает **соединение** с сервером. Как позвонить и дожидаться, пока снимут трубку.

Если адрес начинался с `https`, то в этот момент собеседники ещё и договариваются о шифровании: «давай дальше разговаривать так, чтобы посторонние не поняли». Именно поэтому в браузере появляется замочек.

Чем `https` отличается от `http` на бытовом уровне:

- `http` — открытка. Её может прочитать любой почтальон по дороге.
- `https` — запечатанный конверт. Видно, что письмо едет, но не видно, что внутри.

Пароли и номера карт по открыткам не отправляют. Поэтому сегодня `https` обязателен.

► Шаг 4. Браузер отправляет запрос

Наконец само письмо. Выглядит оно примерно так:

```
GET /catalog/brakes HTTP/1.1
Host: autodoc.tj
User-Agent: Chrome
Accept-Language: ru
```

Не пугайтесь, читается легко:

СТРОКА	ЧТО ГОВОРИТ
GET	«Дай посмотреть». Это метод — тип просьбы
/catalog/brakes	Что именно дать
HTTP/1.1	На какой версии языка говорим
Host:	В какой дом стучимся
User-Agent:	Кто пришёл: браузер представляется
Accept-Language:	«Я предпочитаю русский»

Строки после первой называются **заголовками** — это уточнения к просьбе.

Методов запроса всего **несколько**, и два из них вам понадобятся постоянно:

МЕТОД	СМЫСЛ	ПРИМЕР ИЗ ЖИЗНИ МАГАЗИНА
GET	«Покажи»	Открыть каталог, посмотреть товар
POST	«Прими и запиши»	Оформить заказ, отправить форму
PUT	«Замени целиком»	Обновить карточку товара
DELETE	«Удали»	Удалить товар из корзины

Разницу GET и POST разберём подробно в главе 24, когда будем делать формы. Пока запомните так: **GET смотрит, POST меняет**.

► Шаг 5. Сервер думает

Письмо дошло. Сервер его прочитал и понял: просят страницу каталога тормозных колодок.

И вот здесь просыпается всё то, о чём мы говорили в первой главе. Запускается **PHP**. Он идёт в **базу данных** с запросом на **SQL**, получает список товаров, подставляет их в шаблон и собирает **HTML**.

Занимает это обычно от 20 до 200 миллисекунд. Если дольше — пользователи начинают уходить, и в главе 30 мы будем разбираться, почему тормозит.

► Шаг 6. Сервер отвечает

Ответ выглядит так:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html>
  <head><title>Тормозные колодки</title></head>
  <body>... </body>
</html>
```

Сверху — служебная часть, снизу, после пустой строки, — сам HTML.

200 OK — это **код ответа**. Сервер всегда сообщает, чем закончилось дело. Вот те коды, которые вы будете видеть каждый день:

КОД	ЧТО ОЗНАЧАЕТ	КОГДА ВЫ ЕГО ВСТРЕТИТЕ
200	Всё хорошо, вот страница	Обычная работа
301 / 302	Переехали, идите по другому адресу	Перенаправление, например с <code>http</code> на <code>https</code>
403	Вижу вас, но не пущу	Полез в админку без прав
404	Такой страницы нет	Опечатка в адресе, удалённый товар
500	Сервер сам сломался	Ошибка в вашем коде. Самый частый гость новичка
503	Сервер перегружен	Слишком много посетителей разом

Запомните различие, оно экономит часы: **404** — **вы ошиблись адресом**. **500** — **ошиблись вы, но в коде**. Первое лечится ссылкой, второе — чтением логов (глава 56).

► Шаг 7. Браузер рисует страницу

HTML приехал. Браузер читает его сверху вниз и строит из него страницу.

Но HTML — это только текст. По дороге ему встречаются ссылки на другие файлы: стили, картинки, скрипты. И за каждым файлом браузер **отправляет отдельный запрос**.

Вот это открытие обычно удивляет: одна страница — это не одно письмо, а **десятки**.

Запрос 1 → сама страница (HTML)
Запрос 2 → файл стилей (CSS)
Запрос 3 → скрипт (JavaScript)
Запрос 4 → логотип
Запрос 5 → фото товара №1
Запрос 6 → фото товара №2
... и так далее

Обычная страница магазина — это 30-80 запросов. Поэтому сайты и «весят»: не текст тяжёлый, а картинки, которых много.

► Шаг 8. Оживает JavaScript

Последним включается JavaScript и начинает следить за действиями: клики, ввод, прокрутка. Страница из картинки превращается в живую.

Собрали много новых слов. Разложим по полочкам — это ваш словарь на все оставшиеся главы.

СЛОВО	КАК ЧИТАЕТСЯ	ЧТО ЭТО ПРОСТЫМИ СЛОВАМИ
HTTP	«эйч-ти-ти-пи»	Язык, на котором браузер разговаривает с сервером. Правила переписки
HTTPS	«эйч-ти-ти-пи-эс»	То же самое, но зашифрованное. S — от <i>secure</i> , «безопасный»
Запрос (request)		Письмо от браузера к серверу: «дай мне вот это»
Ответ (response)		Письмо обратно: «вот, держи» или «не могу, вот почему»
Клиент		Тот, кто просит. Обычно браузер
Сервер		Тот, у кого просят. Компьютер, который всегда включён
IP-адрес	«ай-пи»	Номер компьютера в сети: 185.12.94.7
Домен		Человеческое имя сайта: autodoc.tj
DNS	«ди-эн-эс»	Справочник, который переводит имя в номер
Хостинг		Услуга: вам сдают в аренду место на сервере
Заголовки (headers)		Уточнения к письму: кто я, что понимаю, на каком языке
GET	«гет»	Метод «покажи»
POST	«пост»	Метод «прими и запиши»
Код ответа		Число-итог: 200 — хорошо, 404 — нет такого, 500 — сломалось
Порт		Номер двери в доме. У http дверь 80, у https — 443

Про порт чуть подробнее, потому что он вам скоро встретится. Если IP-адрес — это дом, то порт — квартира в этом доме. На одном компьютере может работать несколько программ, и каждая слушает свою дверь. Когда в главе 4 мы запустим сервер на своём компьютере, адрес будет выглядеть как `localhost:8000` — вот `8000` и есть номер двери.

А `localhost` — это «мой собственный компьютер». Всегда. На любой машине. Его IP — `127.0.0.1`, и он тоже всегда один и тот же.

НА ЭКРАНЕ

Посмотрите на это своими глазами

Всё описанное можно увидеть прямо сейчас, без единой строчки кода.

1. Откройте любой сайт в Chrome.
2. Нажмите F12 (на Mac — `Cmd+Option+I`). Откроется панель разработчика.
3. Перейдите на вкладку **Network** («Сеть»).
4. Обновите страницу клавишей F5.

Вы увидите список — это все те самые запросы. Каждая строка: имя файла, код ответа, размер, сколько миллисекунд занял.

Пощёлкайте по строкам. Справа откроются заголовки запроса и ответа — те же самые, что мы разбирали выше, только настоящие.

Это не учебная симуляция. Это реальная переписка вашего браузера с сервером. Профессиональные разработчики смотрят в эту вкладку каждый день.

ГРАБЛИ

Грабли

«Сайт лежит» — а на самом деле нет. Прежде чем паниковать, откройте его в другом браузере, с телефона по мобильному интернету и в режиме инкогнито. Очень часто «лежит» только у вас: кэш, DNS, провайдер, старая версия страницы.

Кэш — ваш главный обманщик. Браузер запоминает файлы, чтобы не качать их дважды. Полезно для скорости, вредно при разработке: вы поправили стили, а браузер показывает старые. Лечится жёстким обновлением: `Ctrl+F5` (на Mac `Cmd+Shift+R`). Запомните эту комбинацию — она сэкономит вам десятки часов недоумения.

Путать 404 и 500. 404 — «я не нашёл такой адрес». 500 — «я нашёл, но при выполнении всё упало». Это разные проблемы и разные места поиска.

Думать, что **https** защищает сайт от взлома. Не защищает. **https** шифрует только дорогу между браузером и сервером. Если в вашем коде дыра, шифрование канала не поможет никак. Безопасностью займёмся в главе 36.

Считать, что домен и хостинг — одно и то же. Домен — это имя. Хостинг — это место, где живёт сайт. Покупаются отдельно, часто у разных компаний. Как отдельно взять номер телефона и отдельно снять квартиру.

ЗАДАЧИ

Задачи

Задача 2.1. Разберите адрес на части — назовите протокол, домен, путь и параметры:

`https://autodoc.tj/search?q=kolodki&brand=bosch`

Задача 2.2. Какой код ответа вы получите в каждом случае?

1. Открыли главную страницу работающего сайта.
2. Ошиблись в адресе на одну букву.
3. Программист забыл точку с запятой, и PHP упал.
4. Зашли в админку, не будучи администратором.

Задача 2.3. Откройте панель разработчика (F12 → Network) на любимом сайте и обновите страницу. Сколько всего запросов ушло? Какой файл оказался самым тяжёлым? Запишите числа — в главе 57 мы вернёмся к ним, когда будем ускорять свой сайт.

Задача 2.4. Объясните своими словами, что такое DNS. Одно предложение, без слова «DNS».

Задача 2.5. Что произойдёт, если DNS сломается, а сайт при этом работает? Сможете ли вы попасть на сайт вообще?

Задача 2.6. Почему форму оформления заказа отправляют методом POST, а не GET? Подсказка: подумайте, что окажется в адресной строке браузера.

Ответы — в Приложении Г.

В коде `autodoc.tj` эта глава встречается буквально везде.

Загляните в любой файл в папке `api/` — например `api/vin_order_request.php`. Первые строки почти каждого такого файла проверяют метод запроса:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    http_response_code(405);  
    exit;  
}
```

Дословно: «если пришли не методом POST — верни код 405 („метод не разрешён“) и заканчивай». Это защита: страницу, которая создаёт заказ, нельзя открывать простой ссылкой.

Через двадцать глав вы будете писать такие проверки не задумываясь.

ИТОГ

Итог

- Интернет — это почта. **Запрос** туда, **ответ** обратно.
- **Клиент** просит (браузер), **сервер** отвечает. Сервер — обычный компьютер, который не выключают.
- DNS переводит имя `autodoc.tj` в номер `185.12.94.7`. Как контакты в телефоне.
- HTTP — язык переписки. **GET** смотрит, **POST** записывает.
- **Коды ответа**: 200 — хорошо, 404 — нет адреса, 500 — сломался код, 403 — не пушу.
- Одна страница — это десятки запросов: HTML, стили, скрипты, каждая картинка.
- F12 → **Network** показывает всю эту переписку вживую. Пользуйтесь.
- Ctrl+F5 обновляет страницу мимо кэша. Запомните прямо сейчас.

В следующей главе перестанем читать и наконец возьмём инструменты в руки.

Инструменты: редактор, терминал, браузер

ЗАЧЕМ ЭТО

Зачем эта глава

Хорошая новость: чтобы делать сайты, не нужен мощный компьютер и не нужны платные программы. Нужны три вещи, и все три бесплатные.

Плохая новость: одна из них — **терминал**, то самое чёрное окно с буквами. Именно на нём спотыкается больше всего новичков. Не потому, что он сложный, а потому, что выглядит недружелюбно.

В этой главе мы поставим всё необходимое и — главное — подружимся с терминалом. К концу главы вы будете уверенно перемещаться по папкам и запускать программы, не трогая мышку.

РАЗБИРАЕМСЯ

Три инструмента, и всё

ИНСТРУМЕНТ	ЧТО ДЕЛАЕТ	АНАЛОГИЯ ИЗ МАСТЕРСКОЙ
Редактор кода	В нём вы пишете код	Верстак, за которым работаете
Терминал	В нём вы запускаете программы	Пульт управления станками
Браузер	В нём вы смотрите результат	Зеркало: сразу видно, что получилось

Всё. Ни «мощного железа», ни платных подписок. Подойдёт любой компьютер, купленный за последние десять лет.

► Почему не Word и не «Блокнот»

Первый вопрос новичка: а нельзя писать код в Word? Нельзя, и вот почему.

Word — это **текстовый процессор**. Он не хранит просто буквы, он хранит оформление: шрифты, размеры, отступы. И когда вы сохраните файл, туда попадёт куча невидимого мусора. Браузер такое не поймёт.

Код — это **чистый текст**. Только символы, ничего больше.

«Блокнот» из Windows чистый текст умеет, но он совершенно ничем не помогает. Это как строить дом одним молотком: теоретически можно, практически — мучение.

► Что ставить: Visual Studio Code

Берите VS Code — бесплатный редактор от Microsoft. Он стал стандартом: на нём работают миллионы разработчиков, под него есть всё на свете.

Скачивается с code.visualstudio.com, ставится как обычная программа, работает на Windows, macOS и Linux.

► Что редактор делает за вас

Вот почему в нём в разы приятнее, чем в блокноте:

- **Подсвечивает синтаксис.** Теги одним цветом, текст другим, ошибки третьим. Глаз сам находит нужное место.
- **Закрывает скобки.** Написали `<div>` — редактор сам поставит `</div>`. Помните правило «что открыли, то закрой»? Он будет следить за этим вместо вас.
- **Подсказывает.** Начали писать `back` — предложит `background`, `background-color` и остальные варианты.
- **Показывает ошибки до запуска.** Забыли кавычку — подчеркнёт красным сразу.
- **Ищет по всему проекту.** Где у меня в тридцати файлах написано `price`? Одна команда.

► Три настройки, которые стоит сделать сразу

Откройте настройки (Ctrl+, или Cmd+,) и поищите по названию:

1. **Word Wrap** → включить (`on`). Длинные строки будут переноситься, а не уезжать за край экрана.
2. **Font Size** → 15 или 16. Глаза скажут спасибо, а вы просидите за кодом много часов.
3. **Auto Save** → `afterDelay`. Файл сохраняется сам. Половина «загадочных ошибок» у новичков — это несохранённый файл.

► Два расширения, которые пригодятся

В VS Code слева есть иконка с кубиками — это магазин расширений. Поставьте:

- **PHP Intelephense** — умные подсказки по PHP.
- **Russian Language Pack** — если хочется интерфейс на русском.

Остальное — потом, по мере надобности. Не увлекайтесь: новички часто ставят тридцать расширений вместо того, чтобы писать код.

РАЗБИРАЕМСЯ

Инструмент второй: терминал

Вот к нему и подошли. Давайте сразу снимем страх.

► Что это вообще такое

Терминал — это способ управлять компьютером словами, а не мышкой. Всё.

Вы уже умеете управлять мышкой: открыть папку, скопировать файл, запустить программу. В терминале вы делаете то же самое, но пишете команду.

Зачем, если мышкой привычнее? Четыре причины:

1. Многие программы просто не имеют окон. Тот же PHP-сервер запускается только командой.
2. Быстрее. Одна строка вместо десяти кликов.
3. Повторяемо. Команду можно записать, отдать коллеге, вставить в инструкцию.
4. На сервере другого способа нет. У сервера нет монитора — только терминал. Когда в главе 57 будем выкладывать сайт, работать придётся именно так.

► Где его найти

СИСТЕМА	КАК ОТКРЫТЬ
Windows	Пуск → напечатать PowerShell → Enter
macOS	Cmd+Пробел → напечатать Терминал → Enter
Linux	Ctrl+Alt+T
Прямо в VS Code	Меню Терминал → Новый терминал, или Ctrl+`

Последний способ — самый удобный. Терминал открывается прямо под кодом, и не нужно прыгать между окнами.

► Как читать эту непонятную строчку

Открыли терминал и увидели что-то вроде:

```
firdavs@MacBook ~/projects %
```

Расшифровываем:

ЧАСТЬ	ЧТО ЗНАЧИТ
firdavs	ваше имя пользователя
MacBook	имя компьютера
~/projects	где вы сейчас находитесь — самое важное
% или \$ или >	приглашение: «пиши команду»

Ключевая мысль: терминал всегда находится в какой-то папке. Как вы, открывая проводник, всегда стоите в какой-то папке. Команды выполняются относительно неё.

Значок ~ означает вашу домашнюю папку. На Windows это C:\Users\Firdavs, на Mac — /Users/firdavs.

► Семь команд, которых хватит надолго

Правда семь. Не сто, не тридцать — семь.

КОМАНДА	РАСШИФРОВКА	ЧТО ДЕЛАЕТ
<code>pwd</code>	<i>print working directory</i>	Где я сейчас? Показывает полный путь
<code>ls</code>	<i>list</i>	Что здесь лежит? Список файлов и папок
<code>cd имя</code>	<i>change directory</i>	Зайти в папку
<code>cd ..</code>		Подняться на уровень выше
<code>mkdir имя</code>	<i>make directory</i>	Создать папку
<code>touch имя</code>		Создать пустой файл
<code>clear</code>		Очистить экран, когда намусорили

На Windows в PowerShell работают все эти команды, кроме `touch`. Вместо него: `New-Item имя.txt`.

► Три приёма, которые экономят время

1. Клавиша Tab дописывает за вас. Начните печатать `cd proj` и нажмите Tab — терминал сам допишет `projects`. Это не только быстрее, но и защищает от опечаток.
2. Стрелка вверх достаёт прошлые команды. Нажмите ↑ — появится предыдущая команда. Ещё раз — позапрошлая.
3. Ctrl+C прерывает. Запустили что-то и хотите остановить — Ctrl+C. Работает почти везде. Понадобится, когда будем гасить наш сервер.

► Потренируйтесь прямо сейчас

Не читайте дальше — откройте терминал и наберите по очереди:

```
pwd
ls
mkdir moi-magazin
cd moi-magazin
pwd
ls
cd ..
```

Что произошло: посмотрели, где мы → посмотрели содержимое → создали папку → зашли в неё → убедились, что вошли → увидели, что она пустая → вышли обратно.

Всё. Вы только что поработали в терминале. Ничего страшного не случилось.

РАЗБИРАЕМСЯ

Инструмент третий: браузер

Браузер у вас уже есть. Но нужно узнать про него кое-что новое.

Возьмите **Chrome** или **Firefox** для разработки. Не потому, что они лучше как браузеры, а потому, что у них лучшие инструменты разработчика и вся документация написана под них.

► Панель разработчика — ваш второй дом

Нажмите F12 (на Mac **Cmd+Option+I**). Панель, которую вы уже открывали в прошлой главе. Вот вкладки, которые понадобятся:

ВКЛАДКА ДЛЯ ЧЕГО

Elements	Смотреть HTML страницы и менять его на лету . Можно поменять цвет кнопки и сразу увидеть результат
Console	Ошибки JavaScript. Первое место, куда смотреть, когда «ничего не работает»
Network	Все запросы: что загрузилось, что нет, сколько заняло
Sources	Исходные файлы страницы

Совет, который экономит нервы: приучитесь держать консоль открытой, когда пишете JavaScript. Половина проблем в ней прямым текстом объяснена — просто туда никто не смотрит.

Пока ставить не нужно, но чтобы вы знали, что впереди:

ЧТО	ЗАЧЕМ	В КАКОЙ ГЛАВЕ
PHP	Чтобы работал серверный код	Глава 19
MySQL	База данных	Глава 26
Git	Хранить историю изменений и не терять работу	Глава 53

Всё это ставится одним пакетом — **XAMPP** (Windows/Mac) или **OpenServer** (Windows). Один установщик кладёт сразу PHP, MySQL и веб-сервер. Подробную инструкцию дам в главе 19, когда дойдём до PHP, — чтобы не ставить впрок то, чем ещё не пользуемся.

Заведите порядок сразу, потом будете благодарны.

Создайте папку для всех своих проектов в домашней директории:

```
projects/  
  moi-magazin/      ← наш будущий магазин  
  eksperimenty/     ← пробы и опыты
```

Два правила, которые избавят от боли:

Никаких русских букв и пробелов в названиях файлов и папок. Не `Мой магазин/каталог товаров.php`, а `moi-magazin/catalog.php`. Веб-серверы, командная строка и системы контроля версий с русскими буквами и пробелами дружат плохо, и вы поймаете «мистическую» ошибку на ровном месте.

Не храните проект в облачных папках. Не кладите код в Dropbox, iCloud или «Рабочий стол», который синхронизируется. Облако начнёт синхронизировать тысячи мелких файлов и создавать конфликтные копии. Для истории изменений есть Git — им и займёмся в главе 53.

«Я боюсь что-то сломать в терминале». Команды из этой книги ничего не ломают. Опасны ровно две вещи: `rm -rf` (удаляет безвозвратно) и всё, что начинается с `sudo` (выполняется от имени администратора). Мы их не используем. Правило простое: **не вставляйте в терминал команду, которую не понимаете**, особенно из случайных статей в интернете.

Файл сохранён? Точно? Классика жанра: правите код, обновляете браузер, ничего не меняется. В VS Code несохранённый файл помечен **точкой** рядом с именем вкладки. Включите автосохранение и забудьте про эту проблему.

Расширение файла имеет значение. `index.html` — страница. `index.txt` — просто текст, браузер покажет исходник. На Windows расширения по умолчанию скрыты, и легко создать `index.html.txt`, не заметив этого. Включите показ расширений: Проводник → Вид → «Расширения имён файлов».

Кириллица в коде. В самом тексте страницы русский, конечно, можно и нужно. Нельзя — в именах файлов, папок и названиях переменных.

Задача 3.1. Установите VS Code и сделайте три настройки из главы: перенос строк, размер шрифта, автосохранение.

Задача 3.2. Откройте терминал и выполните цепочку. Запишите, что вывела каждая команда:

```
pwd
mkdir projects
cd projects
mkdir moi-magazin
cd moi-magazin
pwd
```

Задача 3.3. Найдите на своём компьютере домашнюю папку через `pwd`, а потом откройте её же в обычном проводнике. Убедитесь, что это одно и то же место. Важно почувствовать: терминал и проводник — два окна в один и тот же дом.

Задача 3.4. Откройте любой сайт, нажмите F12 → вкладка **Elements**. Найдите заголовок страницы, дважды кликните по тексту и замените его на своё имя. Нажмите Enter.

Что произошло на самом деле — вы взломали сайт? Ответ обязательно посмотрите в приложении, он важный.

Задача 3.5. Откройте F12 → **Console** на трёх разных сайтах. Найдите хотя бы один с красными ошибками. Их полно даже у крупных компаний — это нормально и полезно знать.

Задача 3.6. Создайте на компьютере папку `projects/moi-magazin` — там мы будем строить магазин всю оставшуюся книгу.

Ответы — в Приложении Г.

В БОЮ

Откройте в проводнике корень этого репозитория и посмотрите на имена файлов:

`seller_shop.php` , `product_edit.php` , `AutoEuroPriceProvider.php` .

Заметьте: **ни одного русского слова, ни одного пробела**. Только латиница, и слова разделены подчёркиванием. Это не эстетство — это то самое правило, которое избавляет от необъяснимых сбоев на боевом сервере.

ИТОГ

Итог

- Нужны три инструмента: **редактор** (VS Code), **терминал**, **браузер** (Chrome). Все бесплатные.
- Код — это **чистый текст**. Word и «Блокнот» не годятся.
- Терминал — управление компьютером словами. Он всегда стоит в какой-то папке.
- Хватит **семи команд**: `pwd`, `ls`, `cd`, `cd ..`, `mkdir`, `touch`, `clear`.
- **Tab** дописывает, **стрелка вверх** повторяет, **Ctrl+C** прерывает.
- **F12** открывает панель разработчика: Elements, Console, Network.
- В именах файлов — **только латиница, без пробелов**.
- Не держите проект в облачной папке.

В следующей главе делаем первую страницу и открываем её в браузере. Наконец-то руками.

Первый файл на экране

ЗАЧЕМ ЭТО**Зачем эта глава**

Три главы мы читали. Хватит.

Сейчас вы сделаете страницу, откроете её в браузере и увидите на экране то, что написали своими руками. Это займёт минут пятнадцать.

Момент, когда браузер впервые показывает **вашу** страницу, — важный. Не потому, что страница получится красивой (она получится страшной), а потому, что с этой секунды интернет перестаёт быть чем-то, что делают «где-то там другие люди».

Единственная просьба: **набирайте руками**. Не копируйте мышкой. Пальцы запоминают синтаксис быстрее, чем глаза.

РАЗБИРАЕМСЯ**Делаем страницу****► Шаг 1. Заводим папку**

Откройте терминал и выполните (или сделайте то же самое мышкой в проводнике):

```
cd ~  
mkdir -p projects/moi-magazin  
cd projects/moi-magazin
```

Мы в папке будущего магазина. Отсюда и начнём.

► Шаг 2. Открываем папку в редакторе

Запустите VS Code, выберите *Файл* → *Открыть папку* и укажите `projects/moi-magazin`.

Слева появится пустое дерево файлов. Это ваш проект.

Почему открывать именно папку, а не файл: редактор тогда видит весь проект целиком — может искать по всем файлам, подсказывать пути, показывать структуру. Привыкайте работать папками.

► Шаг 3. Создаём файл

В панели слева нажмите иконку «новый файл» и назовите его:

```
index.html
```

Имя важное, не придумывайте своё. `index.html` — это имя по умолчанию. Когда браузер приходит по адресу сайта, а конкретная страница не указана, сервер отдаёт именно `index.html`. Как «главная дверь» дома.

► Шаг 4. Пишем

Наберите это. Именно наберите:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
</html>
```

Сохраните: Ctrl+S (на Mac Cmd+S).

► Шаг 5. Открываем в браузере

Найдите файл в проводнике и дважды щёлкните по нему. Откроется браузер.

Вот что вы увидите:

Привет! Это моя первая страница.

Я сделал её сам, и она работает.

Работает. Вы только что сделали веб-страницу.

Посмотрите на адресную строку — там будет что-то вроде

`file:///Users/firdavs/projects/moi-magazin/index.html`. Обратите внимание на `file://` — это значит «файл с моего компьютера», а не из интернета. Пока так и нужно.

РАЗБОР ПО СЛОВАМ

Разбор по словам

Теперь разберём каждую строчку. Ни одна не написана «просто потому что».

► Строка 1: `<!DOCTYPE html>`

```
<!DOCTYPE html>
```

Читается «доктайп эйч-ти-эм-эл». Это **не тег**, а объявление: «дорогой браузер, дальше идёт современный HTML, показывай по современным правилам».

Зачем нужно: если его не написать, браузер включит «режим совместимости» — притворится браузером из 1999 года. Вёрстка поедет, и вы будете долго искать причину.

Пишется **всегда первой строкой**, ровно так, один раз на страницу. Больше о нём думать не нужно.

► Строки 2: `<html lang="ru">`

```
<html lang="ru">
```

ЧАСТЬ

ЧТО ЭТО

```
<html>
```

Корневой элемент. Внутри него живёт вся страница. Открывается в начале, закрывается в самом конце

```
lang="ru"
```

Атрибут. Сообщает: содержимое на русском

Вот и новое понятие — **атрибут**.

Атрибут — это уточнение к тегу. Пишется внутри уголков, после имени тега, в формате `имя="значение"`.

Аналогия: тег — это существительное, атрибут — прилагательное. `<html>` — «страница», `lang="ru"` — «русскоязычная».

Зачем `lang` на практике: браузер поймёт, на каком языке текст, и правильно предложит перевод. Программы для незрячих прочитают текст с русским произношением, а не с английским. Поисковики покажут ваш сайт русскоязычным пользователям.

► Строки 3–6: `<head>` — служебная часть

```
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
```

`<head>` («хед», голова) — это служебная информация о странице. Всё, что внутри `<head>`, на самой странице не видно.

Аналогия: это как поля на конверте — адрес, индекс, марка. Само письмо внутри, а это данные о письме.

СТРОКА	ЧТО ДЕЛАЕТ
<code><meta charset="UTF-8"></code>	Кодировка. Объясняет браузеру, как понимать буквы
<code><title></code>	Название страницы. Оно видно на вкладке браузера и в результатах поиска

Про `charset="UTF-8"` — запомните намертво.

Компьютер хранит не буквы, а числа. Кодировка — это таблица «какое число какой букве соответствует». UTF-8 — таблица, в которой есть всё: русский, английский, таджикский, китайский, эмодзи.

Если эту строчку не написать, браузер начнёт гадать. И русский текст превратится в `ЉŹŃЄĐ,Đ²ĐµŃ,` — знаменитые «кракозябры».

Правило: `<meta charset="UTF-8">` пишется всегда, в каждом файле, без исключений.

Заметьте ещё: у `<meta>` нет закрывающего тега. Такие теги называются **одиночными** — им нечего в себе содержать, они сами по себе законченная команда. Одиночных тегов немного: `<meta>`, `<img>`, `<br>`, `<input>`, `<hr>`.

► Строки 7–10: `<body>` — то, что видно

```
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
```

`<body>` («боди», тело) — всё видимое содержимое страницы. Текст, картинки, кнопки, меню — всё живёт здесь.

Правило простое: не видно на экране → это в `<head>`. Видно → это в `<body>`.

`<h1>` и `<p>` вы уже встречали в первой главе: заголовок и абзац.

► Общая схема любой HTML-страницы

<code><!DOCTYPE html></code>	← «дальше современный HTML»
<code><html></code>	← вся страница
<code><head></code>	← служебное, невидимое
кодировка	
название вкладки	
<code></head></code>	
<code><body></code>	← видимое содержимое
заголовки	
текст	
картинки	
<code></body></code>	
<code></html></code>	

Эта структура **одинакова у всех страниц в интернете**. Откройте код любого сайта — увидите её же. Через неделю вы будете набирать её не задумываясь.

► Про отступы

Заметили, что строки внутри `<head>` сдвинуты вправо? Это **отступы**, и они делаются специально.

Браузеру они не нужны — ему всё равно. Отступы нужны **человеку**: по ним сразу видно, что внутри чего лежит.

Правило: каждый вложенный уровень сдвигается вправо (обычно на 4 пробела или Tab).

Код без отступов — как текст без абзацев и знаков препинания: прочитать можно, но мучительно.

РАЗБИРАЕМСЯ

Делаем страницу похожей на магазин

Первая страница работает. Теперь наполним её чем-то осмысленным.

Замените содержимое `<body>` на это:

```
<body>
  <h1>Автозапчасти Firdavs</h1>
  <p>Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджики

  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <p>В наличии: 7 штук</p>
  <button>Купить</button>

  <h2>Масляный фильтр Mann</h2>
  <p>Артикул: W71280</p>
  <p>Цена: 45 сомони</p>
  <p>В наличии: 23 штуки</p>
  <button>Купить</button>
</body>
```

Не забудьте поменять и `<title>` — пусть будет «Автозапчасти — мой магазин».

Сохраните и обновите страницу в браузере клавишей F5.

Автозапчасти Firdavs

Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджикистану.

Тормозные колодки Bosch

Артикул: 0986424815

Цена: 250 сомони

В наличии: 7 штук

Купить

Масляный фильтр Mann

Артикул: W71280

Цена: 45 сомони

В наличии: 23 штуки

Купить

Вот честный результат — ровно то, что вы увидите у себя.

Давайте признаем прямо: **выглядит ужасно**. Чёрный текст на белом, всё в одну колонку, кнопки серые и системные. Ни один покупатель на таком сайте ничего не купит.

И это абсолютно правильный результат. Потому что мы написали только HTML — а HTML отвечает за содержание, не за красоту. Помните первую главу? Скелет.

Смотрите, что при этом **уже работает**:

- заголовки крупнее обычного текста, а `<h1>` крупнее `<h2>` ;
- абзацы отделены друг от друга воздухом;
- кнопки нажимаются — можете щёлкнуть, они «продавливаются»;
- на вкладке браузера видно название из `<title>` ;
- русский текст читается, кракозябр нет — спасибо `charset="UTF-8"` .

Всё это браузер сделал сам, по умолчанию, потому что вы **правильно назвали части**. Сказали «это заголовок» — он и оформлен как заголовок.

Красоту наведём в третьей части, начиная с главы 9. За один вечер эта же страница превратится в нормальный магазин. Но фундамент — вот он, уже стоит.

Открылся текст кода вместо страницы. Файл сохранился как `index.html.txt`. На Windows расширения по умолчанию скрыты. Лечение: Проводник → Вид → поставить галочку «Расширения имён файлов», потом переименовать.

Вместо русского — `ÐžÑ€Ð²ÐµÑ€ÑŒ`. Забыли `<meta charset="UTF-8">` или сохранили файл не в UTF-8. VS Code по умолчанию сохраняет правильно, так что почти всегда дело в забытой строчке.

Поменял файл, а в браузере всё по-старому. Сначала проверьте, что файл сохранён (в VS Code у несохранённого файла точка на вкладке). Потом нажмите **Ctrl+F5** — обновление мимо кэша.

Забыл закрыть тег. Написали `<h1>` и не написали `</h1>` — дальше вся страница станет огромным заголовком. Браузер не ругается, он просто делает как понял. VS Code обычно закрывает теги сам, но проверять полезно.

Кавычки «не те». Атрибуты пишутся в прямых кавычках: `lang="ru"`. Если скопировать код из Word или мессенджера, кавычки могут превратиться в «ёлочки» — и атрибут сломается. Ещё один довод набирать руками.

Задача 4.1. Сделайте страницу из главы у себя. Убедитесь, что она открывается и русский текст читается.

Задача 4.2. Поменяйте `<title>` на своё имя. Сохраните, обновите браузер. Где именно изменился текст? (Подсказка: смотрите не на страницу.)

Задача 4.3. Добавьте в каталог третий товар — на ваш выбор, с названием, артикулом, ценой и наличием. Скопируйте структуру существующих.

Задача 4.4. Специально сломайте страницу и посмотрите, что будет:

1. Уберите `</h1>` у первого заголовка. Обновите. Что стало со страницей?
2. Верните обратно.
3. Уберите строку `<meta charset="UTF-8">`. Обновите. Что стало с русским текстом?
4. Верните обратно.

Ломать специально — лучший способ понять, зачем нужна каждая строчка. Так вы потом узнаете симптом мгновенно.

Задача 4.5. Попробуйте `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` подряд с одинаковым текстом. Как меняется размер?

Задача 4.6. Откройте свою страницу и нажмите **F12** → **Elements**. Найдите там `<h1>`, который написали. Разверните дерево. Узнаёте свою структуру?

Задача 4.7. Откройте на любом крупном сайте код страницы (правая кнопка → «Посмотреть код страницы») и найдите в нём `<!DOCTYPE html>`, `<head>`, `<body>` и `<meta charset`. Убедитесь: у всех одно и то же.

Ответы — в Приложении Г.

В БОЮ

Откройте в этом репозитории файл `index.php` — главную страницу настоящего магазина.

Кода там много и он пока непонятен. Но найдите глазами `<head>`, `<meta charset`, `<title>` и `<body>`. Они там есть, ровно в том же порядке, что и в вашем файле.

Разница между вашей страницей и боевой — не в структуре. Структура одна. Разница в количестве наполнения. А его мы будем добавлять шестьдесят глав.

ИТОГ

- Страница называется `index.html` — это имя главной по умолчанию.
- `<!DOCTYPE html>` — первая строка всегда, включает современный режим.
- `<html lang="ru">` — вся страница, язык русский.
- `<head>` — служебное и невидимое: кодировка, название вкладки.
- `<body>` — всё, что видно человеку.
- `<meta charset="UTF-8">` обязателен, иначе русский превратится в кракозябры.
- Атрибут — уточнение к тегу: `имя="значение"`.
- Одиночные теги (`<meta>`, `<img>`, `<br>`) не закрываются.
- Отступы нужны человеку, а не браузеру. Делайте их всегда.
- Страница без CSS выглядит ужасно — и это нормально. HTML отвечает за смысл.

Со следующей главы начинается настоящий HTML: разберём теги по-серьёзному и научимся размечать что угодно.

Теги и структура документа

ЗАЧЕМ ЭТО

Зачем эта глава

В прошлой главе вы написали страницу и она заработала. Но написали вы её «по образцу» — как переписывают иероглифы, не зная языка.

Пора выучить язык. В этой главе разберёмся:

- что такое тег на самом деле и по каким правилам он живёт;
- почему одни элементы занимают всю строку, а другие — только своё место;
- зачем нужны `<div>` и `<span>`, если они ничего не делают;
- и главное — что такое **семантика** и почему из-за неё сайты попадают или не попадают в поиск.

После этой главы вы перестанете «списывать» и начнёте писать HTML осмысленно.

РАЗБИРАЕМСЯ

Что такое тег

Тег — это метка, которой вы называете кусок содержимого.

Вы не «рисуете заголовок». Вы говорите браузеру: вот это — заголовок. А как его показать, браузер решит сам.

```
<h1>Автозапчасти Firdavs</h1>
```

Здесь три части:

ЧАСТЬ	НАЗВАНИЕ	ЧТО ДЕЛАЕТ
<code><h1></code>	открывающий тег	«Внимание, начинается заголовок»
Автозапчасти Firdavs	содержимое	То, что видит человек
<code></h1></code>	закрывающий тег	«Заголовок закончился»

Всё вместе — элемент.

► Правило вложенности

Элементы можно класть друг в друга. Но закрывать нужно в обратном порядке — как матрёшки.

Правильно:

```
<p>Цена: <strong>250 сомони</strong></p>
```

Открыли `<p>`, внутри открыли `<strong>`, закрыли `</strong>`, потом закрыли `</p>`. Внутренняя матрёшка закрылась первой.

Неправильно:

```
<p>Цена: <strong>250 сомони</p></strong>
```

Тут `<p>` закрылся раньше, чем `<strong>`. Браузер не выдаст ошибку — он попытается догадаться. И часто догадается неправильно, а вы полчаса будете искать, почему поехала вёрстка.

Запомните так: что открыли последним — закрываете первым.

► Атрибуты — уточнения к тегу

Атрибут добавляет тегу подробностей. Пишется внутри открывающего тега:

```
<html lang="ru">

<a href="/catalog">Каталог</a>
```

Формула всегда одна:

имя="значение"

ПРАВИЛО

ПОЯСНЕНИЕ

Кавычки прямые

`lang="ru"`, а не `lang=«ru»`

Атрибутов может быть много

Разделяются пробелом

Порядок не важен

`` = ``

Некоторые атрибуты без значения

`<input required>` — просто «обязательное»

Два атрибута, которые есть почти у любого тега и понадобятся в главе 9:

- **class** — «этот элемент относится к такой-то группе». По классу CSS будет находить элементы и красить: `<div class="tovar">`.
- **id** — «у этого элемента уникальное имя». На странице **id** должен быть один: `<div id="korzina">`.

Разница простая: класс — как «студент группы 101» (таких много). **id** — как номер паспорта (такой один).

РАЗБИРАЕМСЯ

Блочные и строчные элементы

Вот важное различие, которое объясняет половину «непонятного поведения» вёрстки.

► Блочные — занимают всю строку

Блочный элемент всегда начинается с новой строки и растягивается на всю ширину, даже если содержимого в нём одно слово.

Примеры: `<h1>` ... `<h6>`, `<p>`, `<div>`, `<ul>`, `<table>`, `<form>`.

Аналогия: абзац в книге. Даже если в нём одно слово, он всё равно занимает отдельную строку и никого рядом не пускает.

► Строчные — занимают только своё место

Строчный элемент встраивается в строку и занимает ровно столько, сколько нужно его содержимому.

Примеры: `<a>`, `<strong>`, `<em>`, `<span>`, `<img>`.

Аналогия: выделенное слово в предложении. Оно внутри строки, не разрывает её.

► Сравните на примере

```
<p>Первый абзац</p>
<p>Второй абзац</p>
```

Встанут друг под другом, потому что `<p>` — блочный.

```
<strong>Первое</strong>
<strong>Второе</strong>
```

Встанут в одну строку, потому что `<strong>` — строчный.

Практический вывод. Когда вы недоумеваете «почему эти два блока не встают рядом» или «почему кнопка растянулась на весь экран» — почти всегда ответ здесь. В главе 11 мы научимся управлять этим через CSS.

РАЗБИРАЕМСЯ

и — коробки без смысла

Два тега, которые вы будете использовать чаще всех.

ТЕГ	ТИП	СМЫСЛ	ЗАЧЕМ
<code><div></code>	блочный	никакого	Сгруппировать блок содержимого
<code></code>	строчный	никакого	Выделить кусочек внутри строки

Оба ничего не означают и никак не выглядят. Это чистые контейнеры.

► Зачем нужна пустая коробка

Возьмём карточку товара из прошлой главы:

```
<h2>Тормозные колодки Bosch</h2>
<p>Артикул: 0986424815</p>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Четыре отдельных элемента. Браузер не знает, что они связаны — для него это просто заголовок, два абзаца и кнопка, идущие подряд.

Заворачиваем в коробку:

```
<div class="tovar">
  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь это одна вещь, у которой есть имя — `товар`. И можно сказать одной строчкой CSS: «все элементы с классом `товар` сделать белыми, с рамкой, скруглёнными углами и отступом». Все карточки оформятся разом.

► А `<span>` зачем

Когда нужно выделить кусочек **внутри** строки:

```
<p>Цена: <span class="cena">250</span> сомони</p>
```

Теперь можно покрасить только число, не трогая слова вокруг.

Правило выбора: нужно завернуть блок содержимого — `<div>`. Нужно выделить пару слов внутри текста — `<span>`.

РАЗБИРАЕМСЯ

Семантика: главная идея HTML

А вот теперь — самое важное в этой главе.

► Проблема

Технически можно построить весь сайт из `<div>`. Вообще весь:

```
<div class="zagolovok">Автозапчасти</div>
<div class="menu">Каталог Доставка Контакты</div>
<div class="tovar">Колодки Bosch</div>
<div class="podval">© 2026</div>
```

Работать будет. Выглядеть после CSS — прилично. Но это плохой код, и вот почему.

Для браузера, поисковика и программы для незрячих это четыре одинаковые безымянные коробки. Слово `zagolovok` в атрибуте `class` ничего им не говорит — они читают теги, а не ваши выдумки. С тем же успехом там могло быть написано `korobka1`.

► Решение: теги, у которых есть смысл

В HTML есть **семантические теги** — теги со значением. Вместо безымянной коробки вы говорите, что это за часть страницы:

ТЕГ	ЧТО ОЗНАЧАЕТ	ЧТО БЫЛО БЫ ВМЕСТО
<code><header></code>	Шапка: логотип, название	<code><div class="header"></code>
<code><nav></code>	Меню, навигация	<code><div class="menu"></code>
<code><main></code>	Основное содержимое страницы	<code><div class="content"></code>
<code><article></code>	Самостоятельный кусок: товар, статья	<code><div class="item"></code>
<code><section></code>	Раздел страницы	<code><div class="block"></code>
<code><aside></code>	Боковая колонка, дополнительное	<code><div class="sidebar"></code>
<code><footer></code>	Подвал: контакты, копирайт	<code><div class="footer"></code>

Пишутся они **точно так же**, как `<div>`, и ведут себя так же — блочные, без вида. Разница только в одном: у них **есть смысл**.

► Как выглядит правильная страница

```
<body>
  <header>
    <h1>Автозапчасти Firdavs</h1>
  </header>

  <nav>
    <a href="/catalog">Каталог</a>
    <a href="/delivery">Доставка</a>
    <a href="/contacts">Контакты</a>
  </nav>

  <main>
    <article>
      <h2>Тормозные колодки Bosch</h2>
      <p>Цена: 250 сомони</p>
      <button>Купить</button>
    </article>

    <article>
      <h2>Масляный фильтр Mann</h2>
      <p>Цена: 45 сомони</p>
      <button>Купить</button>
    </article>
  </main>

  <footer>
    <p>© 2026 Автозапчасти Firdavs. Худжанд.</p>
  </footer>
</body>
```

Автозапчасти Firdavs

[Каталог](#) [Доставка](#) [Контакты](#)

Тормозные колодки Bosch

Цена: 250 сомони

Купить

Масляный фильтр Mann

Цена: 45 сомони

Купить

© 2026 Автозапчасти Firdavs. Худжанд.

Выглядит так же скучно, как раньше, — семантика не про внешность. Но теперь структура страницы **читается** по одному только коду.

► Что вы за это получаете

Три очень практичных выигрыша:

1. **Поисковики понимают страницу.** Google видит `<main>` и знает: вот здесь главное, а `<nav>` и `<footer>` можно не учитывать. Правильная семантика — часть SEO, то есть попадания сайта в поиск.

2. **Незрячие люди могут пользоваться сайтом.** Программы-чтецы умеют перепрыгивать по разделам: «перейти к основному содержимому», «перейти к меню». Если у вас одни `<div>`, прыгать не по чему — человек вынужден слушать всё подряд, включая меню на каждой странице.

3. **Код читаемый.** Через полгода вы откроете свой файл. `</div></div></div></div>` в конце — ад. `</main></body>` — понятно.

Правило на всю жизнь: *есть подходящий по смыслу тег — берите его.* `<div>` — когда подходящего нет.

Новые слова этой главы, собранные вместе:

СЛОВО	ЧТО ОЗНАЧАЕТ
Тег	Метка в уголках: <code><p></code>
Элемент	Открывающий тег + содержимое + закрывающий
Атрибут	Уточнение внутри тега: <code>class="tovar"</code>
Вложенность	Элементы внутри элементов, как матрёшки
Блочный	Занимает всю строку: <code><div></code> , <code><p></code> , <code><h1></code>
Строчный	Занимает своё место в строке: <code></code> , <code><a></code> , <code></code>
Семантика	Смысл тега. <code><header></code> вместо <code><div class="header"></code>
<code>class</code>	Имя группы. Таких может быть много
<code>id</code>	Уникальное имя. Такой один на странице

Ставить `<h1>` несколько раз. `<h1>` — главный заголовок страницы, он один. Дальше по убыванию: `<h2>` для разделов, `<h3>` внутри них. Не прыгайте через уровень (`<h1>` → `<h3>`) и не выбирайте заголовок по размеру — размер настроим в CSS.

Выбирать тег по внешнему виду. «Мне нужен мелкий текст, возьму `<h6>` » — так нельзя. Тег выбирают по смыслу, внешний вид задаёт CSS. Это ошибка, которую совершают почти все новички.

Заворачивать всё подряд в лишние `<div>` . Если внутри коробки лежит ровно один элемент и коробка ничего не группирует — она не нужна. Лишние обёртки делают код нечитаемым.

Пробел внутри тега. `< p>` — не работает. Между `<` и именем тега пробела быть не должно.

Русские буквы в `class` и `id` . Технически возможно, практически — источник проблем. Только латиница: `class="tovar"` , а не `class="товар"` .

Задача 5.1. Перепишите свою страницу из главы 4, добавив семантические теги:

`<header>` с названием магазина, `<nav>` с тремя ссылками, `<main>` с товарами (каждый в `<article>`), `<footer>` с копирайтом.

Задача 5.2. Для каждого определите — блочный или строчный: `<p>`, `<span>`, `<div>`, `<a>`, `<h2>`, `<strong>`, `<article>`, `<em>`.

Задача 5.3. Найдите ошибку и объясните, в чём она:

```
<p>Цена: <strong>250 сомони</p></strong>
```

Задача 5.4. Замените `<div>` на подходящий семантический тег:

```
<div class="header"> ... </div>
<div class="menu"> ... </div>
<div class="content"> ... </div>
<div class="sidebar"> ... </div>
<div class="footer"> ... </div>
```

Задача 5.5. В каком случае берут `<div>`, а в каком `<span>`? Приведите свой пример на каждый.

Задача 5.6. Откройте крупный интернет-магазин, нажмите F12 → Elements и поищите теги `<header>`, `<nav>`, `<main>`, `<footer>`. Есть ли они? У некоторых крупных сайтов их нет — сплошные `<div>`. Как думаете, почему так вышло?

Задача 5.7. Объясните своими словами, почему `<div class="zagolovok">` хуже, чем `<h1>`. Два предложения.

Ответы — в Приложении Г.

Откройте `index.php` в корне репозитория и поищите в нём `<div class=`. Найдёте много.

Это правда жизни: боевой код никогда не бывает идеальным. Сайт autodoc.tj построен на готовом шаблоне Mazlay, а шаблоны обычно щедрЫ на `<div>`.

Важный вывод, который стоит усвоить сразу: **работающий код с недостатками лучше, чем идеальный код, которого нет**. Стремитесь к семантике в том, что пишете сами, но не переписывайте рабочий сайт ради красоты — у бизнеса есть задачи поважнее.

ИТОГ

Итог

- **Элемент** = открывающий тег + содержимое + закрывающий.
- Вложенность закрывается **в обратном порядке**, как матрёшки.
- **Атрибут** — уточнение: `имя="значение"`, кавычки прямые.
- `class` — имя группы (много), `id` — уникальное имя (один).
- **Блочные** занимают всю строку, **строчные** — только своё место.
- `<div>` — блочная коробка, `<span>` — строчная. Обе без смысла.
- **Семантика**: `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>` — то же самое, но со смыслом. Нужны для поиска, для незрячих и для читаемости.
- Тег выбирают **по смыслу**, а не по внешнему виду.

В следующей главе наполним каркас содержимым: текст, ссылки, картинки и списки.

Текст, ссылки, картинки, списки

ЗАЧЕМ ЭТО

Зачем эта глава

Каркас страницы у вас есть. Теперь научимся наполнять его тем, из чего состоит любой сайт: текстом, ссылками, картинками и списками.

Тегов здесь будет много, но пугаться не надо — это как выучить названия инструментов в мастерской. Каждый простой, каждый делает одну вещь. К концу главы вы сможете разметить любую страницу с текстом.

РАЗБИРАЕМСЯ

Текст

► Заголовки: от `<h1>` до `<h6>`

```
<h1>Автозапчасти Firdavs</h1>
<h2>Тормозная система</h2>
<h3>Колодки</h3>
```

Шесть уровней. Работают как оглавление книги: часть → глава → раздел.

Три правила, которые нарушают все:

1. `<h1>` на странице один. Это её название. Не два, не три.
2. Не прыгать через уровень. После `<h1>` идёт `<h2>`, а не `<h3>`.
3. Не выбирать по размеру. Нужен мелкий заголовок — берите правильный уровень и уменьшайте его в CSS.

Почему это важно: по заголовкам строится «оглавление» страницы. Им пользуются поисковики и программы для незрячих. Сломанная иерархия — сломанное оглавление.

► Абзацы и переносы

```
<p>Первый абзац текста.</p>
<p>Второй абзац текста.</p>
```

Абзац — `<p>`, блочный, сам отделяется отступами.

Важная особенность HTML: он не видит ваших переносов строк и лишних пробелов.

```
<p>Слово          другое
слово</p>
```

Браузер покажет: `Слово другое слово`. Все пробелы подряд он сожмёт в один, а перенос строки посчитает пробелом.

Это не ошибка, а правило: пустое место в коде нужно человеку, а не браузеру. Поэтому можно спокойно делать отступы и переносы для читаемости — на результат не влияет.

Нужен настоящий перенос строки внутри абзаца — есть одиночный тег `<br>`:

```
<p>Худжанд, ул. Ленина, 15<br>Телефон: +992 92 777 77 77</p>
```

⚠ Не используйте `<br><br>` для отступов между абзацами. Для этого есть `<p>` и отступы в CSS. `<br>` — только для случаев, когда перенос часть содержания: адрес, стихи, реквизиты.

► Выделение: `<strong>` и `<em>`

```
<p>Цена: <strong>250 сомони</strong></p>
<p>Товар <em>под заказ</em>, срок 5 дней.</p>
```

ТЕГ	КАК ВЫГЛЯДИТ	ЧТО ОЗНАЧАЕТ
<code></code>	жирный	Важно. Обратите внимание
<code></code>	курсив	Смысловое ударение

Есть ещё `<b>` (жирный) и `<i>` (курсив). Выглядят так же, но не несут смысла — просто «сделай жирным». Программа для незрячих выделит голосом `<strong>` и проигнорирует `<b>`.

Правило: важное — `<strong>`, ударение — `<em>`, а `<b>` и `<i>` не трогайте.

► Остальные полезные теги

```
<p>Артикул: <code>0986424815</code></p>
<p>Цена: <s>300</s> 250 сомони</p>
<p>Гарантия <small>(при сохранении чека)</small></p>
<blockquote>Отличные колодки, тормозит как новая.</blockquote>
<hr>
```

ТЕГ	ЧТО ДЕЛАЕТ	ГДЕ ПРИГОДИТСЯ
<code><code></code>	Моноширинный шрифт	Артикулы, коды, куски кода
<code><s></code>	Зачёркнутый	Старая цена при скидке
<code><small></code>	Мелкий текст	Примечания, сноски
<code><blockquote></code>	Цитата	Отзывы покупателей
<code><hr></code>	Линия-разделитель	Разделить разделы

РАЗБИРАЕМСЯ

Ссылки

Ссылка — то, ради чего интернет вообще существует. Тег `<a>`, от английского *anchor*, якорь.

```
<a href="/catalog">Каталог</a>
```

ЧАСТЬ	ЧТО ЭТО
<code><a></code>	Тег ссылки. Строчный — встраивается в текст
<code>href</code>	Куда ведём. От <i>hypertext reference</i>
Каталог	Что видит и нажимает человек

Без `href` ссылка не работает. Это её единственный обязательный атрибут.

► Виды адресов

Здесь новички путаются чаще всего, поэтому разберём подробно.

```
<a href="https://autodoc.tj/catalog">На другой сайт</a>
<a href="/catalog">От корня сайта</a>
<a href="catalog.html">Рядом с текущим файлом</a>
<a href="../index.html">На папку выше</a>
<a href="#dostavka">К месту на этой же странице</a>
```

ЗАПИСЬ	НАЗВАНИЕ	КУДА ВЕДЁТ
<code>https:// ...</code>	Абсолютный	Полный адрес. На чужие сайты только так
<code>/catalog</code>	От корня	От начала вашего сайта. Работает с любой страницы
<code>catalog.html</code>	Относительный	Файл в той же папке
<code>../index.html</code>	Относительный	<code>..</code> — подняться на папку выше
<code>#dostavka</code>	Якорь	К элементу с <code>id="dostavka"</code> на этой странице

Практический совет: внутри своего сайта используйте адреса **от корня** — `/catalog`. Относительные ломаются, когда вы переносите файл в другую папку, и это одна из самых раздражающих ошибок.

► Особые ссылки

```
<a href="tel:+992927777777">+992 92 777 77 77</a>
<a href="mailto:info@autodoc.tj">Написать нам</a>
```

На телефоне первая ссылка сразу звонит. Вторая открывает почтовую программу. Мелочь, а покупателю приятно.

► Открыть в новой вкладке

```
<a href="https://bosch.com" target="_blank" rel="noopener">Сайт Bosch</a>
```

`target="_blank"` — открыть в новой вкладке. `rel="noopener"` — обязательная добавка для безопасности: без неё чужая страница получает доступ к вашей.

Когда открывать в новой вкладке: ссылки на чужие сайты. Свои страницы — в той же вкладке, иначе завалите человеку браузер.

РАЗБИРАЕМСЯ

Картинки

```

```

`<img>` — одиночный тег, закрывать не нужно. Два обязательных атрибута:

АТТРИБУТ	ЧТО ЗНАЧИТ
<code>src</code>	Откуда брать. Путь к файлу (<i>source</i> — источник)
<code>alt</code>	Что здесь изображено словами

► Почему `alt` обязателен

Про него всегда забывают, а он делает три вещи:

1. Показывается, если картинка не загрузилась. Битый путь, медленный интернет — человек увидит хотя бы описание.
2. Его читают программы для незрячих. Без `alt` человек услышит «изображение» — и всё.
3. По нему картинки находят в поиске. Поиск по картинкам работает именно на `alt`.

Как писать `alt` :

```
 ✓  
 ✗  
 ⚠
```

Пустой `alt=""` — это осознанное «картинка декоративная, читать нечего». Для узоров и разделителей так правильно. Для товара — нет.

► Размеры

```

```

Указывать размеры полезно: браузер заранее резервирует место, и страница не «прыгает» во время загрузки. Знакомое раздражение, когда читаешь текст, а он уезжает вниз, — это как раз незадаанные размеры.

► Форматы

ФОРМАТ	ДЛЯ ЧЕГО
JPG	Фотографии товаров
PNG	Логотипы, картинки с прозрачностью
SVG	Иконки, схемы. Не мылятся при увеличении
WebP	Современный, весит меньше при том же качестве

⚠ Главная ошибка новичка с картинками: залить фото прямо с телефона. Такое фото весит 5 МБ. Двадцать товаров — 100 МБ на одной странице. С мобильного интернета это не откроется никогда. Фото для сайта сжимают до 100–300 КБ.

РАЗБИРАЕМСЯ

Списки

► Маркированный — когда порядок не важен

```
<ul>
  <li>Тормозные колодки</li>
  <li>Тормозные диски</li>
  <li>Тормозная жидкость</li>
</ul>
```

`<ul>` — *unordered list*, нумерованный список. `<li>` — *list item*, пункт.

► Нумерованный — когда порядок важен

```
<ol>
  <li>Выберите товар</li>
  <li>Добавьте в корзину</li>
  <li>Оформите заказ</li>
</ol>
```

`<ol>` — *ordered list*. Номера браузер расставляет сам — не пишите их руками, иначе при вставке нового пункта придётся перенумеровывать всё.

Правило: можно переставить пункты местами без потери смысла — `<ul>`. Нельзя (это шаги, рейтинг, инструкция) — `<ol>`.

► Вложенные списки

```
<ul>
  <li>Тормозная система
    <ul>
      <li>Колодки</li>
      <li>Диски</li>
    </ul>
  </li>
  <li>Двигатель</li>
</ul>
```

Внимание: вложенный список кладут **внутри** `<li>`, а не между ними. Так делают многоуровневые меню каталога.

► Список описаний

Редкий, но очень удобный для характеристик товара:

```
<dl>
  <dt>Производитель</dt>
  <dd>Bosch</dd>
  <dt>Артикул</dt>
  <dd>0986424815</dd>
  <dt>Гарантия</dt>
  <dd>12 месяцев</dd>
</dl>
```

`<dl>` — список, `<dt>` — термин, `<dd>` — описание.

Страница товара, использующая всё из этой главы:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Тормозные колодки Bosch — Автозапчасти Firdavs</title>
</head>
<body>
  <header>
    <h1>Автозапчасти Firdavs</h1>
  </header>

  <nav>
    <a href="/">Главная</a> ·
    <a href="/catalog">Каталог</a> ·
    <a href="/contacts">Контакты</a>
  </nav>

  <main>
    <article>
      <h2>Тормозные колодки Bosch</h2>

      <p>Артикул: <code>0986424815</code></p>
      <p>Цена: <s>300</s> <strong>250 сомони</strong></p>
      <p>Состояние: <em>в наличии, 7 комплектов</em></p>

      <h3>Характеристики</h3>
      <dl>
        <dt>Производитель</dt>
        <dd>Bosch, Германия</dd>
        <dt>Подходит для</dt>
        <dd>Toyota Camry 2011-2017</dd>
        <dt>Гарантия</dt>
        <dd>12 месяцев</dd>
      </dl>
```

```

<h3>Что входит в комплект</h3>
<ul>
  <li>Колодки передние – 4 шт.</li>
  <li>Смазка направляющих</li>
  <li>Датчик износа</li>
</ul>

<h3>Как заказать</h3>
<ol>
  <li>Позвоните или напишите нам</li>
  <li>Уточните марку и год автомобиля</li>
  <li>Заберите в Худжанде или закажите доставку</li>
</ol>

<button>Купить</button>

<hr>

<blockquote>Поставил на Камри – тормозит мягко, скрипа нет. Рекомендую!</blockquote>
<p><small>Отзыв покупателя, март 2026</small></p>
</article>
</main>

<footer>
  <p>Худжанд, ул. Ленина 15<br>
  Телефон: <a href="tel:+992927777777">+992 92 777 77 77</a></p>
  <p><small>© 2026 Автозапчасти Firdavs</small></p>
</footer>
</body>
</html>

```

Автозапчасти Firdavs

[Главная](#) · [Каталог](#) · [Контакты](#)

Тормозные колодки Bosch

Артикул: 0986424815

Цена: ~~300~~ **250 сомони**

Состояние: *в наличии, 7 комплектов*

Характеристики

Производитель

Bosch, Германия

Подходит для

Toyota Camry 2011–2017

Гарантия

12 месяцев

Что входит в комплект

- Колодки передние — 4 шт.
- Смазка направляющих
- Датчик износа

Как заказать

1. Позвоните или напишите нам
2. Уточните марку и год автомобиля
3. Заберите в Худжанде или закажите доставку

Купить

Поставил на Камри — тормозит мягко, скрипа нет. Рекомендую.

Отзыв покупателя, март 2026

Худжанд, ул. Ленина 15

Телефон: [+992 92 777 77 77](tel:+992927777777)

© 2026 Автозапчасти Firdavs

Опять некрасиво — и опять правильно. Зато посмотрите, сколько всего уже работает само по себе:

- заголовки выстроились по важности;
- ссылки синие и подчёркнутые, нажимаются;
- список с точками, нумерованный — с цифрами;
- зачёркнутая старая цена рядом с новой;
- цитата отодвинута от края;
- характеристики выстроились в два уровня;
- телефон на мобильном будет звонить одним касанием.

Всё это браузер сделал сам, потому что вы **правильно назвали каждую часть**. Добавим CSS — и та же страница станет магазином.

ГРАБЛИ

Грабли

Картинка не отображается. Три причины по частоте: неверный путь в `src` (проверьте регистр букв — `Kolodki.JPG` и `kolodki.jpg` для сервера разные файлы), файл не туда положили, опечатка в расширении.

`<br>` вместо абзацев. Признак кода, написанного наугад. Абзац — `<p>` .

Ссылка без `href` . `<a>Каталог</a>` — не ссылка, а просто текст.

Ссылки на весь сайт в новой вкладке. Раздражает. `target="_blank"` — только для чужих сайтов.

Фото по 5 МБ. Сожмите до 100–300 КБ, иначе сайт будет открываться минуту.

`alt="фото"` . Бессмысленно. Пишите, что на картинке, или ставьте пустой `alt=""` , если она декоративная.

ЗАДАЧИ

Задачи

Задача 6.1. Сделайте страницу товара из примера у себя. Возьмите свой товар, не колодки.

Задача 6.2. Найдите ошибки:

```
<a>Каталог</a>

<ul>
  <p>Первый пункт</p>
  <p>Второй пункт</p>
</ul>
<h1>Заголовок</h1>
<h1>Другой заголовок</h1>
```

Задача 6.3. Что покажет браузер? Ответьте, не запуская:

```
<p>Один      два
три</p>
```

Задача 6.4. Сделайте меню каталога вложенным списком, три раздела по два подраздела в каждом.

Задача 6.5. Разметьте свою визитку: имя `<h1>`, должность абзацем, телефон ссылкой `tel:`, почта ссылкой `mailto:`, три навыка списком.

Задача 6.6. Когда `<ul>`, а когда `<ol>` ?

1. Список ингредиентов плова.
2. Шаги приготовления плова.
3. Топ-5 самых продаваемых товаров.
4. Что входит в комплект поставки.

Задача 6.7. Скачайте любое фото, положите рядом с `index.html` и вставьте на страницу с осмысленным `alt`. Потом специально сломайте путь в `src` и обновите — увидите, как выглядит `alt` в деле.

Ответы — в Приложении Г.

Откройте `pages/` в этом репозитории и посмотрите, как в боевом коде выводятся характеристики товара и меню каталога. Найдёте те же `<ul>`, `<li>`, `<a>` и `<img>` — только значения в них подставляет РНР, а не человек руками.

Вот к чему мы идём: разметку пишете вы один раз, а данные в неё подставляются из базы. Одна страница товара обслуживает десять тысяч товаров.

ИТОГ

Итог

- Заголовки `<h1>` — `<h6>` — иерархия, а не размер. `<h1>` один на страницу.
- HTML сжимает пробелы и переносы. Форматируйте код как удобно.
- `<strong>` — важное, `<em>` — ударение. `<b>` и `<i>` не используйте.
- Ссылка `<a href="...">`. Внутри сайта — адреса от корня: `/catalog`.
- `tel:` и `mailto:` — звонок и письмо одним касанием.
- `` — одиночный тег. `alt` обязателен.
- Фото сжимать до 100–300 КБ. Не заливать прямо с телефона.
- `<ul>` — порядок не важен, `<ol>` — важен, `<dl>` — характеристики.

Дальше — таблицы и формы: научимся принимать данные от покупателя.

Таблицы и формы

ЗАЧЕМ ЭТО

Зачем эта глава

До сих пор мы только показывали информацию. Теперь научимся двум вещам посерьёзнее:

- **таблицы** — показывать данные, у которых есть строки и столбцы;
- **формы** — принимать данные от покупателя.

Форма — это первый раз, когда информация побежит в обратную сторону: не от сайта к человеку, а от человека к сайту. Регистрация, вход, поиск, заказ, отзыв — всё это формы.

Отправлять данные по-настоящему мы научимся в главе 24, когда появится РНР. Сейчас построим саму форму — и разберём каждое поле.

РАЗБИРАЕМСЯ

Таблицы

► Когда таблица уместна, а когда нет

Сразу главное правило, потому что его нарушают постоянно:

Таблица — для данных, у которых есть строки и столбцы. Прайс-лист, характеристики, расписание, отчёт о продажах.

Таблица — не для раскладки страницы. Двадцать лет назад сайты верстали таблицами, потому что другого способа не было. Сегодня для раскладки есть Flexbox и Grid (глава 11), а таблица в этой роли ломает мобильную версию и мешает незрячим.

► Из чего состоит таблица

```
<table>
  <thead>
    <tr>
      <th>Товар</th>
      <th>Артикул</th>
      <th>Цена</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Колодки Bosch</td>
      <td>0986424815</td>
      <td>250 сомони</td>
    </tr>
    <tr>
      <td>Фильтр Mann</td>
      <td>W71280</td>
      <td>45 сомони</td>
    </tr>
  </tbody>
</table>
```

ТЕГ	РАСШИФРОВКА	ЧТО ЭТО
<code><table></code>		Вся таблица
<code><thead></code>	<i>table head</i>	Шапка таблицы
<code><tbody></code>	<i>table body</i>	Тело: собственно данные
<code><tr></code>	<i>table row</i>	Строка
<code><th></code>	<i>table header</i>	Ячейка-заголовок. Жирная и по центру
<code><td></code>	<i>table data</i>	Обычная ячейка с данными

Логика такая: таблица состоит из **строк** (`<tr>`), а строка — из **ячеек** (`<th>` или `<td>`).

⚠ Частая путаница: люди пытаются описать таблицу по столбцам. Нельзя. HTML знает только строки. Столбцы получаются сами, когда в каждой строке одинаковое число ячеек.

► Объединение ячеек

```
<tr>
  <td colspan="2">Занимает два столбца</td>
  <td>Обычная</td>
</tr>
<tr>
  <td rowspan="2">Занимает две строки</td>
  <td>Ячейка</td>
</tr>
```

`colspan` — объединить по горизонтали, `rowspan` — по вертикали. Как в Excel.

► Подпись

```
<table>
  <caption>Прайс-лист на тормозную систему, март 2026</caption>
  ...
</table>
```

`<caption>` — заголовок таблицы. Пишется сразу после `<table>`. Полезен и людям, и незрячим.

РАЗБИРАЕМСЯ

Формы

Теперь главное в этой главе.

► Что такое форма

Форма — это конверт, в который складывают данные и отправляют на сервер.

```
<form action="/order.php" method="POST">
  <input type="text" name="phone">
  <button type="submit">Отправить</button>
</form>
```

ЧАСТЬ	ЧТО ДЕЛАЕТ
<code><form></code>	Сам конверт. Всё внутри отправится вместе
<code>action</code>	Куда отправить. Адрес обработчика на сервере
<code>method</code>	Как отправить. <code>GET</code> или <code>POST</code>
<code><input></code>	Поле для ввода
<code><button type="submit"></code>	Кнопка «отправить»

► GET или POST

Помните из главы 2? Теперь применим.

	GET	POST
Где едут данные	В адресной строке	Внутри письма, не видно
Видно в адресе	<code>?q=kolodki&brand=bosch</code>	Ничего не видно
Можно ли добавить в закладки	Да	Нет
Ограничение по размеру	Есть, около 2000 символов	Практически нет
Для чего	Поиск, фильтры	Заказы, вход, регистрация

Простое правило: данные, которые не жалко показать в адресной строке и которыми хочется поделиться ссылкой, — GET. Всё остальное — POST.

Пароль в адресной строке — это пароль в истории браузера, в логах сервера и в закладках. Поэтому вход — только POST.

► Атрибут `name` — самый важный

```
<input type="text" name="phone">
```

Без `name` поле не отправится вообще. Совсем. Человек заполнит, нажмёт кнопку, данные потеряются.

Почему: `name` — это **подпись на данных**. Сервер получит `phone=99292777777` и по слову `phone` поймёт, что это телефон. Без подписи он получит непонятную строку без имени и просто её выбросит.

Запомните: **нет** `name` — **нет данных**. Это ошибка номер один в формах.

РАЗБИРАЕМСЯ

Виды полей

`<input>` — один тег, но его поведение полностью меняется атрибутом `type`.

► Текстовые

```
<input type="text"      name="name"      placeholder="Ваше имя">
<input type="email"     name="email"     placeholder="pochta@example.com">
<input type="tel"       name="phone"     placeholder="+992 __ __ __">
<input type="password"  name="password">
<input type="number"    name="qty"       min="1" max="99" value="1">
<input type="search"    name="q"         placeholder="Поиск по каталогу">
```

TYPE

ЧТО ДАЁТ

<code>text</code>	Обычная строка
<code>email</code>	Браузер проверит, что есть <code>@</code> . На телефоне — клавиатура с <code>@</code>
<code>tel</code>	На телефоне откроется цифровая клавиатура
<code>password</code>	Символы прячутся за точками
<code>number</code>	Только числа, появляются стрелочки. <code>min</code> и <code>max</code> ограничивают
<code>search</code>	Поле поиска, у него появляется крестик очистки

Про мобильную клавиатуру — недооценённая мелочь. Поставили `type="tel"` — человек на телефоне сразу видит цифры вместо букв. Не поставили — он мучается, переключая раскладку. Такие детали и отличают удобный сайт от неудобного.

► Выбор

←!— Один из нескольких —→

```
<input type="radio" name="dostavka" value="samovyvoz" id="d1">
<label for="d1">Самовывоз</label>
```

```
<input type="radio" name="dostavka" value="kurier" id="d2">
<label for="d2">Курьером</label>
```

←!— Галочка —→

```
<input type="checkbox" name="soglasie" id="s1">
<label for="s1">Согласен на обработку данных</label>
```

Ключевая разница:

- **radio** — выбрать **одно** из нескольких. Чтобы они работали как группа, у них должен быть одинаковый `name`. Это самая частая ошибка: разные `name` — и можно выбрать все варианты сразу.
- **checkbox** — включить или выключить. Каждый сам по себе.

► Выпадающий список

```
<select name="brand">
  <option value="">Все бренды</option>
  <option value="bosch">Bosch</option>
  <option value="mann" selected>Mann</option>
  <option value="denso">Denso</option>
</select>
```

`value` — что уйдёт на сервер, текст между тегами — что видит человек. `selected` — вариант по умолчанию.

► Большое поле для текста

```
<textarea name="comment" rows="4" placeholder="Комментарий к заказу"></textarea>
```

⚠ У `<textarea>` есть закрывающий тег, в отличие от `<input>`. И значение по умолчанию пишется между тегами, а не в атрибут `value`.

► Прочее

```
<input type="date" name="dostavka_data">
<input type="file" name="foto">
<input type="hidden" name="tovar_id" value="42">
```

hidden — скрытое поле. Человек его не видит, но оно отправится вместе с формой. Так передают служебные данные: номер товара, защитный токен.

РАЗБИРАЕМСЯ

— обязательная вещь

```
<label for="phone">Телефон</label>
<input type="tel" id="phone" name="phone">
```

Связка простая: **for** у метки совпадает с **id** у поля.

Зачем это нужно:

1. Клик по надписи ставит курсор в поле. Особенно важно для маленьких галочек: попасть пальцем в квадратик 15×15 на телефоне трудно, а в надпись рядом — легко.
2. Программы для незрячих прочитают, что это за поле. Без метки человек услышит «поле ввода» и не узнает, что туда вводить.

Правило: каждое поле должно иметь **<label>**. Без исключений.

⚠ **placeholder** — не замена метки. Он исчезает, как только человек начинает печатать. Заполнили полформы, отвлеклись — и уже не помните, что в каком поле.

РАЗБИРАЕМСЯ

Проверка прямо в браузере

Браузер умеет проверять поля сам, без единой строчки кода:

```
<input type="email" name="email" required>
<input type="tel" name="phone" required pattern="[0-9+ ]{9,20}">
<input type="number" name="qty" min="1" max="99" required>
<input type="text" name="name" minlength="2" maxlength="50" required>
```

АТТРИБУТ	ЧТО ПРОВЕРЯЕТ
<code>required</code>	Поле обязательное, пустым не отправится
<code>min</code> / <code>max</code>	Границы для чисел
<code>minlength</code> / <code>maxlength</code>	Длина текста
<code>pattern</code>	Соответствие шаблону

Очень удобно. Но запомните намертво — мы вернёмся к этому в главе 36:

⚠ Проверка в браузере — это удобство, а не защита. Её отключают за десять секунд через F12. Все данные **обязательно** проверяются ещё раз на сервере в PHP. Всегда. Без исключений.

```
<h2>Оформление заказа</h2>

<form action="/order.php" method="POST">

    <input type="hidden" name="tovar_id" value="42">

    <p>
        <label for="name">Ваше имя</label><br>
        <input type="text" id="name" name="name" required minlength="2">
    </p>

    <p>
        <label for="phone">Телефон</label><br>
        <input type="tel" id="phone" name="phone" required placeholder="+992 _
    </p>

    <p>
        <label for="qty">Количество</label><br>
        <input type="number" id="qty" name="qty" value="1" min="1" max="20">
    </p>

    <p>
        <label for="brand">Предпочитаемый бренд</label><br>
        <select id="brand" name="brand">
            <option value="">Любой</option>
            <option value="bosch">Bosch</option>
            <option value="mann">Mann</option>
        </select>
    </p>

    <fieldset>
        <legend>Способ получения</legend>
        <input type="radio" id="d1" name="dostavka" value="samovyvoz" checked>
        <label for="d1">Самовывоз, Худжанд</label><br>
        <input type="radio" id="d2" name="dostavka" value="kurier">
        <label for="d2">Доставка курьером</label>
    </fieldset>

    <p>
```

```
<label for="comment">Комментарий</label><br>
<textarea id="comment" name="comment" rows="3"
placeholder="Марка, год, объём двигателя"></textarea>

</p>

<p>
<input type="checkbox" id="soglasie" name="soglasie" required>
<label for="soglasie">Согласен на обработку данных</label>
</p>

<button type="submit">Оформить заказ</button>
</form>
```

Новый тег: `<fieldset>` группирует связанные поля, а `<legend>` даёт группе название. Полезно для радиокнопок.

Прайс-лист

Прайс-лист на тормозную систему, март 2026

Товар	Артикул	Цена
Колодки Bosch	0986424815	250 сомони
Фильтр Mann	W71280	45 сомони
Диски Brembo	09.9468.11	780 сомони

Оформление заказа

Ваше имя

Телефон

Количество

Предпочитаемый бренд

Способ получения

- ☒ Самовывоз, Худжанд
☐ Доставка курьером

Комментарий

Марка, год, объём
двигателя

☐ Согласен на обработку данных

Оформить заказ

Нажмите «Оформить заказ», не заполнив поля, — браузер сам покажет подсказку «Заполните это поле». Это работает `required`, и мы не написали ни строчки кода.

Форма пока никуда не отправляет — обработчика `/order.php` не существует. Сделаем его в главе 24.

СЛОВО

ЧТО ОЗНАЧАЕТ

<code><table></code> / <code><tr></code> / <code><th></code> / <code><td></code>	Таблица / строка / ячейка-заголовок / ячейка данных
<code>colspan</code> / <code>rowspan</code>	Объединить ячейки по горизонтали / вертикали
<code><form></code>	Конверт для данных
<code>action</code>	Куда отправлять
<code>method</code>	Как отправлять: GET или POST
<code>name</code>	Подпись на данных. Без него данные не дойдут
<code>value</code>	Значение, которое уйдёт на сервер
<code><label for="..."></code>	Подпись к полю, связана с <code>id</code>
<code>placeholder</code>	Серая подсказка внутри поля. Исчезает при вводе
<code>required</code>	Обязательное поле
<code><fieldset></code> / <code><legend></code>	Группа полей / её название
<code>type="hidden"</code>	Невидимое поле для служебных данных

ГРАБЛИ

Грабли

Забывать `name`. Поле не отправится. Проверять в первую очередь.

Разные `name` у радиокнопок. Они перестают быть группой — можно выбрать всё сразу.

`placeholder` вместо `<label>`. Подсказка исчезает при вводе, и человек забывает, что заполняет.

Верить проверке в браузере. Отключается за секунды. Проверять на сервере.

Пароль методом GET. Он окажется в адресной строке, истории и логах. Только POST.

Таблица для раскладки страницы. Ломает мобильную версию. Для раскладки — CSS.

Кнопка без `type`. Внутри формы кнопка по умолчанию отправляет её. Если кнопка нужна для другого — пишите `type="button"`.

Задача 7.1. Сделайте таблицу-прайс на пять товаров: название, артикул, бренд, цена, наличие. С шапкой `<thead>` и подписью `<caption>`.

Задача 7.2. Найдите три ошибки:

```
<form>
  <input type="text" placeholder="Имя">
  <input type="radio" name="a" value="1"> Первый
  <input type="radio" name="b" value="2"> Второй
  <input type="password" name="pass">
</form>
```

Задача 7.3. Какой `type` подойдёт лучше всего?

1. Год выпуска автомобиля.
2. Адрес электронной почты.
3. Номер телефона.
4. Согласие с условиями.
5. Выбор города из двадцати.
6. Выбор одного из трёх способов оплаты.
7. Пароль при регистрации.

Задача 7.4. Сделайте форму поиска по каталогу: строка поиска, выпадающий список брендов, поля «цена от» и «цена до», кнопка. Какой метод здесь правильный — GET или POST? Объясните почему.

Задача 7.5. Сделайте форму обратной связи: имя, телефон, тема (выпадающий список), сообщение (textarea), галочка согласия. Все поля с `<label>` и обязательные.

Задача 7.6. Откройте форму входа на любом сайте, нажмите F12 → Elements и найдите поля. Какой у формы `method`? Какие `name` у полей?

Задача 7.7. Проверьте свою форму: нажмите отправить с пустыми полями. Появилась подсказка браузера? Теперь уберите `required` у одного поля и попробуйте снова.

Ответы — в Приложении Г.

В БОЮ В бою

Найдите в этом репозитории `auth/` — там формы входа и регистрации настоящего магазина. Посмотрите: у всех `method="POST"`, у всех полей есть `name`, есть скрытые поля со служебными данными.

Одно такое скрытое поле особенно интересное — **CSRF-токен**. Это защита от подделки запроса: злоумышленник не сможет отправить форму от вашего имени с чужого сайта. Разберём его в главе 36.

ИТОГ Итог

- Таблица — для данных со строками и столбцами, не для раскладки.
- `<tr>` — строка, `<th>` — заголовок, `<td>` — данные. Столбцов в HTML нет, они получаются сами.
- Форма — конверт. `action` — куда, `method` — как.
- GET для поиска и фильтров, POST для заказов, входа, регистрации.
- `name` обязателен. Нет `name` — данные не дойдут.
- Радиокнопки одной группы имеют одинаковый `name`.
- `<label for>` для каждого поля. `placeholder` его не заменяет.
- Проверка в браузере — удобство, защита только на сервере.

Следующая глава — практическая. Соберём полноценный каталог и закрепим весь HTML.

Практика: карточка товара и каталог

ЗАЧЕМ ЭТО

Зачем эта глава

Четыре главы мы разбирали теги поодиночке. Пора собрать из них настоящую страницу.

Сегодня вы сделаете **главную страницу магазина** целиком: шапку с меню, фильтры, каталог из шести товаров, подвал с контактами. Это будет тот самый файл, который мы будем улучшать всю оставшуюся книгу — сначала оформим в CSS, потом оживим на JavaScript, потом начнём подставлять товары из базы данных.

Новых тегов почти не будет. Будет **сборка** и несколько приёмов, о которых в учебниках обычно молчат.

РАЗБИРАЕМСЯ

Сначала — план

Профессиональный приём, который экономит часы: **прежде чем писать теги, набросайте структуру словами.**

Что должно быть на главной странице магазина?

Шапка

Название магазина

Меню: Каталог, Доставка, Контакты

Строка поиска

Основная часть

Заголовок «Каталог запчастей»

Фильтры: бренд, цена от/до

Список товаров, у каждого:

фото

название

артикул

цена

наличие

кнопка «Купить»

Подвал

Адрес и телефон

Копирайт

Теперь у каждой строчки этого плана есть подходящий тег. План превращается в код почти механически:

ПЛАН	ТЕГ
Шапка	<code><header></code>
Меню	<code><nav> + + + <a></code>
Строка поиска	<code><form> + <input type="search"></code>
Основная часть	<code><main></code>
Каждый товар	<code><article></code>
Фото	<code></code>
Цена	<code><p> + </code>
Подвал	<code><footer></code>

Совет: делайте так всегда, особенно на первых порах. Гораздо легче править план из десяти строк, чем распутывать сто строк готового кода.

Начнём с одного товара — потом просто повторим.

```
<article class="tovar">
  

  <h3>Тормозные колодки Bosch</h3>

  <p class="artikul">Артикул: <code>0986424815</code></p>
  <p class="cena"><strong>250</strong> сомони</p>
  <p class="nalichie">В наличии: 7 шт.</p>

  <form action="/cart.php" method="POST">
    <input type="hidden" name="tovar_id" value="1">
    <button type="submit">В корзину</button>
  </form>
</article>
```

Что здесь важного:

Заголовок товара — `<h3>`, а не `<h1>`. Потому что `<h1>` уже занят названием магазина, `<h2>` — заголовком «Каталог». Товар — третий уровень. Иерархия соблюдена.

Классы расставлены заранее. `tovar`, `cena`, `nalichie` пока ничего не делают — CSS-то нет. Но в главе 9 мы по ним найдём элементы и оформим. Привыкайте называть части сразу.

Кнопка внутри формы. Товар в корзину — это изменение данных, значит POST. Скрытое поле передаёт, какой именно товар положить.

► 1. Комментарии в коде

```
←!— Шапка сайта —→  
<header>  
...  
</header>  
←!— /Шапка —→
```

Всё между `←!—` и `—→` браузер игнорирует. Это заметки для человека.

Зачем: когда файл вырастет до трёхсот строк, комментарии станут указателями. Особенно полезно пометать **конец** большого блока — сразу видно, какой `</div>` что закрывает.

⚠ HTML-комментарии видны в исходном коде страницы любому. Никогда не пишите в них пароли, заметки вроде «тут костыль, не трогать» или служебные адреса.

► 2. ` ` — неразрывный пробел

```
<p>Цена: 250&nbsp;сомони</p>  
<p>Телефон: +992&nbsp;92&nbsp;777&nbsp;77&nbsp;77</p>
```

Обычный пробел позволяет браузеру перенести строку. Иногда это уродливо:

```
Цена: 250  
сомони
```

` ` — пробел, по которому **нельзя переносить**. Число и единица измерения останутся вместе.

Где ставить: между числом и единицей (`250 сомони` , `12 мес`), в номерах телефонов, между инициалами и фамилией.

Такие записи называются **спецсимволами** и всегда начинаются с `&`:

ЗАПИСЬ	ЧТО ДАЁТ	ЗАЧЕМ
<code>&nbsp;</code>	Неразрывный пробел	Не разрывать «250 сомони»
<code>&mdash;</code>	—	Тире по-русски
<code>&laquo;</code> <code>&raquo;</code>	« »	Кавычки-ёлочки
<code>&copy;</code>	©	Копирайт
<code>&lt;</code> <code>&gt;</code>	< >	Показать уголки как текст
<code>&</code>	&	Показать сам амперсанд

Последние три обязательны, если нужно **написать теги как текст**. Иначе браузер попытается их выполнить.

► 3. Один товар — потом копия

Не пишите шесть карточек подряд руками. Сделайте **одну и отладьте её**, а потом копируйте.

Почему это важно: если в единственной карточке ошибка, вы поправите её один раз. Если вы уже размножили карточку с ошибкой на шесть штук — править придётся шесть раз, и обязательно где-то забудете.

Это, кстати, прямая дорога к главному принципу программирования, который встретится в главе 23: **не повторяйся**. Скоро мы вообще перестанем копировать карточки — их будет создавать РНР в цикле.

Вот полный файл. Наберите его — это ваш магазин.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Автозапчасти Firdavs — запчасти в Худжанде</title>
  <meta name="description" content="Автозапчасти для японских и немецких
    автомобилей в Худжанде. Доставка по Таджикистану.">
</head>
<body>

<!-- Шапка -->
<header>
  <h1>Автозапчасти Firdavs</h1>
  <p>Запчасти для японских и немецких авто &mdash; Худжанд</p>

  <nav>
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/catalog">Каталог</a></li>
      <li><a href="/delivery">Доставка</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>

  <form action="/search" method="GET">
    <label for="q">Поиск по каталогу</label>
    <input type="search" id="q" name="q" placeholder="Название или артикул">
    <button type="submit">Найти</button>
  </form>
</header>
<!-- /Шапка -->

<main>
  <h2>Каталог запчастей</h2>

  <!-- Фильтры -->
```

```

<form action="/catalog" method="GET">
  <fieldset>
    <legend>Подобрать</legend>

    <label for="brand">Бренд</label>
    <select id="brand" name="brand">
      <option value="">Все</option>
      <option value="bosch">Bosch</option>
      <option value="mann">Mann</option>
      <option value="denso">Denso</option>
      <option value="brembo">Brembo</option>
    </select>

    <label for="min">Цена от</label>
    <input type="number" id="min" name="min" min="0" placeholder="0">

    <label for="max">до</label>
    <input type="number" id="max" name="max" min="0" placeholder="5000">

    <button type="submit">Показать</button>
  </fieldset>
</form>
<!-- /Фильтры -->

<!-- Товары -->
<article class="tovar">
  <h3>Тормозные колодки Bosch</h3>
  <p class="artikul">Артикул: <code>0986424815</code></p>
  <p class="cena"><strong>250</strong> &nbsp; сомони</p>
  <p class="nalichie">В наличии: 7 &nbsp; шт.</p>
  <form action="/cart.php" method="POST">
    <input type="hidden" name="tovar_id" value="1">
    <button type="submit">В корзину</button>
  </form>
</article>

<article class="tovar">
  <h3>Масляный фильтр Mann W71280</h3>
  <p class="artikul">Артикул: <code>W71280</code></p>
  <p class="cena"><strong>45</strong> &nbsp; сомони</p>
  <p class="nalichie">В наличии: 23 &nbsp; шт.</p>
  <form action="/cart.php" method="POST">

```

```

        <input type="hidden" name="tovar_id" value="2">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Свечи зажигания Denso</h3>
    <p class="artikul">Артикул: <code>IK20</code></p>
    <p class="cena"><strong>120</strong>&nbsp;сомони</p>
    <p class="nalichie nety">Под заказ, 5&nbsp;дней</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="3">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Тормозные диски Brembo</h3>
    <p class="artikul">Артикул: <code>09.9468.11</code></p>
    <p class="cena"><strong>780</strong>&nbsp;сомони</p>
    <p class="nalichie">В наличии: 2&nbsp;шт.</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="4">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Воздушный фильтр Mann C25114</h3>
    <p class="artikul">Артикул: <code>C25114</code></p>
    <p class="cena"><strong>65</strong>&nbsp;сомони</p>
    <p class="nalichie">В наличии: 14&nbsp;шт.</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="5">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Аккумулятор Bosch S4 60Ah</h3>
    <p class="artikul">Артикул: <code>0092S40050</code></p>
    <p class="cena"><strong>950</strong>&nbsp;сомони</p>

```

```

        <p class="nalichie">В наличии: 4 шт.</p>
        <form action="/cart.php" method="POST">
            <input type="hidden" name="tovar_id" value="6">
            <button type="submit">В корзину</button>
        </form>
    </article>
    <!-- /Товары -->
</main>

<!-- Подвал -->
<footer>
    <h2>Контакты</h2>
    <p>Худжанд, ул. Ленина 15<br>
        Телефон: <a href="tel:+992927777777">+992 92 777 77 77<br>
        Почта: <a href="mailto:info@autodoc.tj">info@autodoc.tj</a></p>

    <h3>Режим работы</h3>
    <ul>
        <li>Пн — Пт: 9:00 — 18:00</li>
        <li>Сб: 9:00 — 14:00</li>
        <li>Вс: выходной</li>
    </ul>

    <p><small>&copy; 2026 Автозапчасти Firdavs</small></p>
</footer>
<!-- /Подвал -->

</body>
</html>

```

РАЗБОР ПО СЛОВАМ

Разбор по словам

В коде появились две новые строки в `<head>`. Разберём — они обе важные.

► `<meta name="viewport" ...>`

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Это строка, без которой сайт не работает на телефоне.

Без неё мобильный браузер решает: «сайт, наверное, старый и рассчитан на компьютер». Он делает вид, что экран шириной 980 пикселей, и уменьшает всю страницу, чтобы влезла. Текст становится нечитаемым, и человек вынужден растягивать пальцами.

ЧАСТЬ

ЧТО ГОВОРИТ

`width=device-width`

«Ширина страницы = настоящая ширина экрана»

`initial-scale=1`

«Не уменьшать при открытии»

Пишется в каждой странице, всегда. Мы вернёмся к ней в главе 12, когда займёмся мобильной версией всерьёз.

► `<meta name="description" ...>`

```
<meta name="description" content="Автозапчасти для японских и немецких автомоби
```

Это текст, который человек увидит в результатах поиска Google — серое описание под синей ссылкой.

Если его не написать, поисковик выдернет случайный кусок текста со страницы, и выглядеть это будет так себе. Пишите 120–160 символов, по-человечески, с пользой для читателя.

Вместе с `<title>` это два поля, которые решают, кликнет человек по вашему сайту в поиске или пролистает дальше.

► Ещё раз про иерархию заголовков

Посмотрите на нашу страницу целиком:

<code><h1></code> Автозапчасти Firdavs	← название сайта, ОДИН
<code><h2></code> Каталог запчастей	← раздел
<code><h3></code> Тормозные колодки	← товар
<code><h3></code> Масляный фильтр	← товар
<code><h2></code> Контакты	← раздел
<code><h3></code> Режим работы	← подраздел

Ровное дерево без пропусков. Именно так поисковик строит «оглавление» вашей страницы.

НА ЭКРАНЕ

На экране

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

- [Главная](#)
- [Каталог](#)
- [Доставка](#)
- [Контакты](#)

Поиск по каталогу

Каталог запчастей

Подобрать

Бренд	Все	Цена от	0	до	5000	Показать
-------	----------------------------------	---------	--------------------------------	----	-----------------------------------	---

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

Масляный фильтр Mann W71280

Артикул: W71280

45 сомони

В наличии: 23 шт.

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

Воздушный фильтр Mann C25114

Артикул: C25114

65 сомони

В наличии: 14 шт.

Аккумулятор Bosch S4 60Ah

Артикул: 0092540050

950 сомони

В наличии: 4 шт.

Контакты

Вот он, ваш магазин. Пока — длинная чёрно-белая простыня.

Давайте честно: до нормального сайта отсюда далеко. Товары идут в столбик, формы разъехались, фильтры выглядят как случайный набор полей.

Но остановитесь на секунду и посмотрите, что уже есть:

- поиск работает как поле поиска, на телефоне вызовет нужную клавиатуру;
- фильтр по бренду раскрывается списком;
- поля цены принимают только числа;
- у каждого товара своя кнопка со своим номером;
- телефон нажимается и звонит;
- структура страницы правильная и семантическая;
- поисковик увидит и заголовок, и описание.

Скелет готов и он правильный. Дальше — одежда.

И вот что приятно: в следующей главе мы не будем трогать этот файл почти совсем. Мы создадим рядом файл со стилями, подключим его одной строчкой — и та же самая страница превратится в магазин. Разделение обязанностей из первой главы — вот оно, в действии.

ГРАБЛИ

Грабли

Забыть `viewport`. На телефоне сайт будет крошечным. Проверьте: откройте свою страницу на телефоне или включите F12 → значок телефона в углу панели.

Не закрыть тег в середине файла. Страница поедет начиная с этого места. Ищите так: сузьте область — прокомментируйте половину `<!-- -->`, посмотрите, осталась ли проблема, и так далее. Приём называется «деление пополам», им ищут ошибки во всём.

Забыть `alt` у картинок. В нашем примере картинок нет специально — чтобы страница работала у всех. Как только добавите свои фото, `alt` обязателен.

Скопировать карточку и забыть поменять `tovar_id`. Все шесть кнопок будут класть в корзину один и тот же товар. Классическая ошибка копирования.

Задача 8.1. Соберите эту страницу у себя целиком. Замените товары на свои — хоть на телефоны, хоть на книги, хоть на цветы. Магазин необязательно должен быть автомобильным.

Задача 8.2. Добавьте седьмой и восьмой товар. Не забудьте про `товар_id`.

Задача 8.3. Сделайте вторую страницу — `товар.html`, подробную карточку одного товара. Возьмите пример из главы 6 и добавьте: таблицу характеристик, форму заказа из главы 7, ссылку «← Назад в каталог».

Задача 8.4. Свяжите страницы: в каталоге сделайте название каждого товара ссылкой на `товар.html`. Проверьте, что переходы работают в обе стороны.

Задача 8.5. Добавьте в подвал вложенное меню каталога (`<ul>` внутри `<li>`): три раздела, в каждом по два подраздела.

Задача 8.6. Проверьте свою страницу валидатором. Откройте `validator.w3.org`, вкладка *Validate by Direct Input*, вставьте свой код и нажмите Check.

Валидатор — это проверка HTML на ошибки. Он найдёт незакрытые теги, неправильную вложенность, забытые атрибуты. Сколько ошибок у вас? Исправьте все.

Задача 8.7. Откройте страницу на телефоне (или через F12 → значок телефона). Всё читается? Теперь уберите строку `viewport`, обновите и посмотрите снова. Разницу видно?

Задача 8.8. Напишите план (словами, как в начале главы) для страницы «Доставка и оплата». Какие теги подойдут каждому пункту? Потом соберите её.

Ответы — в Приложении Г.

Вы сейчас сделали руками то, что настоящий магазин делает автоматически.

Откройте в репозитории `index.php` — там та же структура: шапка, каталог, подвал. Но карточки товаров не написаны шесть раз. Там один **шаблон карточки**, который PHP повторяет в цикле столько раз, сколько товаров пришло из базы.

Шесть товаров или шесть тысяч — кода одинаково.

Именно поэтому мы учим сначала HTML: **чтобы было что повторять**. В главе 25 вы возьмёте эту самую карточку и научите PHP размножать её самостоятельно.

ИТОГ

Итог

- Перед кодом — **план словами**. Потом каждой строке плана подбирается тег.
- Карточка товара: `<article>` + `<h3>` + данные + форма с `<input type="hidden">`.
- **Комментарии** `<!-- -->` — заметки для человека. Не пишите в них секретов.
- ` ` не даёт разорвать «250 сомони». `—`, `«`, `©` — тире, кавычки, копирайт.
- `<meta name="viewport">` обязателен, иначе сайт непригоден на телефоне.
- `<meta name="description">` — текст под ссылкой в результатах поиска.
- Иерархия заголовков без пропусков: `<h1>` → `<h2>` → `<h3>`.
- Отладьте одну карточку, потом копируйте. Скоро копировать перестанем совсем.
- Проверяйте себя валидатором `validator.w3.org`.

Часть II закончена. Вы умеете размечать страницы. Со следующей главы начинается CSS — и ваш магазин наконец станет красивым.

Подключаем CSS и учим селекторы

ЗАЧЕМ ЭТО

Зачем эта глава

Вы построили страницу магазина. Она правильная, семантическая — и абсолютно непривлекательная.

Начинаем это исправлять. CSS — это то, что превращает чёрно-белую простыню в сайт, на котором хочется что-то купить.

В этой главе разберёмся с двумя вещами:

1. Как подключить CSS к странице (три способа, правильный — один).
2. Как объяснить браузеру, что именно красить — это называется селекторы.

Второе важнее. Умение точно указать «вот этот элемент и никакой другой» — основа всей работы с CSS.

РАЗБИРАЕМСЯ

Что такое CSS

CSS — *Cascading Style Sheets*, «каскадные таблицы стилей».

Название страшное, суть простая: **список указаний, как показывать элементы страницы.**

Одно указание — одно **правило**:

```
button {  
  background: #b5411f;  
  color: white;  
  border-radius: 6px;  
}
```

Читается: «все кнопки на странице сделай с красно-рыжим фоном, белым текстом и скруглёнными на 6 пикселей углами».

Из чего состоит правило:

ЧАСТЬ	НАЗВАНИЕ	ЧТО ДЕЛАЕТ
<code>button</code>	Селектор	Кого оформляем
<code>{ }</code>		Границы списка указаний
<code>background</code>	Свойство	Что меняем
<code>#b5411f</code>	Значение	На что меняем
<code>;</code>		Конец одного указания

Пара «свойство: значение» называется **объявлением**. Их в правиле может быть сколько угодно.

РАЗБИРАЕМСЯ

Три способа подключить CSS

► Способ 1: отдельный файл — правильный

```
<head>
  <meta charset="UTF-8">
  <title>Автозапчасти Firdavs</title>
  <link rel="stylesheet" href="style.css">
</head>
```

Создаём рядом с `index.html` файл `style.css` и подключаем одной строкой.

АТТРИБУТ	ЧТО ЗНАЧИТ
<code><link></code>	Одиночный тег: «подключить внешний файл»
<code>rel="stylesheet"</code>	Что подключаем — таблицу стилей
<code>href="style.css"</code>	Где она лежит

Почему так правильно:

- один файл стилей на **все** страницы сайта — поменяли цвет кнопки в одном месте, изменилось везде;
- браузер скачает `style.css` один раз и запомнит — остальные страницы откроются быстрее;
- код разделён: разметка отдельно, оформление отдельно.

Так делают всегда. Остальные два способа — для особых случаев.

► Способ 2: в `<head>` страницы

```
<head>
  <style>
    button { background: #b5411f; }
  </style>
</head>
```

Стили внутри самой страницы. Годится для быстрой пробы или для крошечной одностраничной вещи. На настоящем сайте — нет: придётся дублировать на каждой странице.

► Способ 3: прямо в теге — так не надо

```
<button style="background: #b5411f;">Купить</button>
```

Называется **инлайн-стиль**. Работает, но плох почти всем:

- относится только к этому одному элементу;
- смешивает разметку и оформление;
- **перебивает** все остальные стили, из-за чего потом невозможно ничего изменить.

Встречается в реальном коде (обычно там, где стиль задаёт программа), но своими руками так писать не стоит.

Вот тут — самое полезное в главе.

► По имени тега

```
p { color: #333; }  
h1 { font-size: 32px; }  
button { background: #b5411f; }
```

Найдёт **все** такие теги на странице. Просто, но грубо: покрасит вообще все кнопки, включая те, которые вы красить не собирались.

► По классу — главный рабочий инструмент

```
<p class="cena">250 сомони</p>
```

```
.cena { color: #b5411f; font-weight: bold; }
```

Точка перед именем означает «это класс».

Класс можно:

- повесить на **много** элементов — все оформятся одинаково;
- повесить **несколько** на один элемент через пробел:

```
<p class="nalichie nety">Под заказ</p>
```

```
.nalichie { color: green; }  
.nety     { color: gray; }
```

Элемент получит оба правила. Очень удобно: базовый вид плюс модификация.

90% вашего CSS будет по классам. Привыкайте.

► По id — редко

```
<div id="korzina">...</div>
```

```
#korzina { position: fixed; right: 20px; }
```

Решётка означает «это id». Напоминаю: `id` уникален, один на странице.

Используйте редко. У `id` слишком большая «сила» (см. ниже про приоритеты), и он мешает переиспользованию.

► Вложенность — «внутри чего-то»

```
.tovar h3 { font-size: 18px; }
```

Читается: «заголовки `<h3>`, которые лежат **внутри** элемента с классом `tovar`». Другие `<h3>` на странице не тронутся.

Пробел здесь — важный знак. Он означает «внутри».

```
nav a      { color: #333; }      /* ссылки внутри меню */
footer a    { color: gray; }      /* ссылки в подвале */
.tovar .cena { font-size: 20px; } /* цена внутри карточки товара */
```

► Несколько селекторов сразу

```
h1, h2, h3 {
    font-family: Georgia, serif;
}
```

Запятая означает «и ещё». Одно правило применится ко всем трём.

► По состоянию — псевдоклассы

```
a:hover      { color: #b5411f; }      /* когда навели мышь */
button:hover { background: #8f3418; }
input:focus  { border-color: #b5411f; } /* когда поле активно */
li:first-child { font-weight: bold; }   /* первый элемент списка */
li:last-child { border-bottom: none; }  /* последний */
```

Двоеточие означает «в состоянии».

`:hover` — самый частый. Это то самое «кнопка потемнела, когда навёл мышь». Мелочь, а сайт сразу кажется живым.

► Все элементы

```
* { margin: 0; padding: 0; }
```

Звёздочка — «вообще все». Применяют обычно один раз в начале файла, чтобы сбросить разную браузерных настроек по умолчанию.

► Сводка селекторов

ЗАПИСЬ	КОГО ВЫБЕРЕТ
<code>p</code>	Все абзацы
<code>.cena</code>	Все с <code>class="cena"</code>
<code>#korzina</code>	Элемент с <code>id="korzina"</code>
<code>.tovar h3</code>	<code><h3></code> внутри <code>.tovar</code>
<code>h1, h2</code>	И заголовки первого, и второго уровня
<code>a:hover</code>	Ссылка под курсором
<code>*</code>	Все элементы

Этих семи хватит на 95% задач. Есть ещё десятки, но они понадобятся нескоро.

Каскад: кто победит, если правил несколько

«Каскадные» в названии CSS — вот про это.

Ситуация: на один элемент действуют два правила, и они противоречат друг другу. Кто выиграет?

► Правило 1: кто ниже, тот и прав

```
p { color: red; }
p { color: blue; }
```

Текст будет **синим**. Побеждает то правило, которое написано **позже**.

Отсюда практический вывод: подключайте свои стили **после** чужих (например, после готового шаблона), иначе они не перебьют.

► Правило 2: кто точнее, тот и прав

Но порядок работает только при равной точности. Более точный селектор побеждает независимо от порядка:

```
.товар .цена { color: red; }    /* точнее */
.цена         { color: blue; }  /* написан ниже, но проигрывает */
```

Цена будет **красной**.

Шкала силы, от слабого к сильному:

СИЛА	ЧТО ЭТО	ПРИМЕР
1	Тег	p
10	Класс, псевдокласс	.цена, :hover
100	id	#korzina
1000	Инлайн-стиль	style="..."
∞	!important	color: red !important;

Считается сложением. `.tovar .cena` = 10 + 10 = 20, побеждает `.cena` = 10.

► Про `!important`

```
.cena { color: red !important; }
```

Перебивает всё. И именно поэтому — **не пользуйтесь им**.

`!important` кажется решением («не работает — допишу important»), но это долг, который придётся отдавать. Через месяц вам понадобится перекрыть этот стиль, и вы напишете ещё один `!important`. Потом ещё. Через полгода в файле их сорок, и никто не понимает, что откуда берётся.

Правильный способ — написать **более точный селектор**.

Создайте рядом с `index.html` файл `style.css` :

```
/* Сброс: у браузеров разные отступы по умолчанию */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* Основа страницы */
body {
    font-family: Georgia, 'Times New Roman', serif;
    font-size: 16px;
    line-height: 1.6;
    color: #2b2622;
    background: #faf7f2;
    padding: 24px;
}

/* Шапка */
header {
    border-bottom: 2px solid #b5411f;
    padding-bottom: 16px;
    margin-bottom: 24px;
}

h1 {
    font-size: 30px;
    color: #b5411f;
}

/* Меню */
nav ul {
    list-style: none;
    display: flex;
    gap: 20px;
    margin: 12px 0;
}
```

```
nav a {
    color: #2b2622;
    text-decoration: none;
    font-weight: bold;
}

nav a:hover {
    color: #b5411f;
}

/* Карточка товара */
.tovar {
    background: white;
    border: 1px solid #e3d8c6;
    border-radius: 8px;
    padding: 16px;
    margin-bottom: 16px;
}

.tovar h3 {
    font-size: 18px;
    margin-bottom: 8px;
}

.artikul {
    color: #7c7166;
    font-size: 14px;
}

.cena {
    font-size: 22px;
    color: #b5411f;
    margin: 8px 0;
}

.nalichie {
    color: #14584f;
    font-size: 14px;
}

/* Нет в наличии — приглушаем */
.nalichie.nety {
```

```

        color: #9a9088;
    }

    /* Кнопки */
    button {
        background: #b5411f;
        color: white;
        border: none;
        padding: 10px 20px;
        border-radius: 6px;
        font-size: 15px;
        cursor: pointer;
        margin-top: 10px;
    }

    button:hover {
        background: #8f3418;
    }

    /* Подвал */
    footer {
        margin-top: 32px;
        padding-top: 16px;
        border-top: 1px solid #e3d8c6;
        color: #7c7166;
        font-size: 14px;
    }

```

И подключите его в `<head>` :

```
<link rel="stylesheet" href="style.css">
```

Было — та самая простыня из прошлой главы. Стало:

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

[Главная](#) [Каталог](#) [Доставка](#) [Контакты](#)

Поиск по каталогу

Найти

Каталог запчастей

Подобрать

Бренд

Цена от до

Показать

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

В корзину

Масляный фильтр Mann W71280

Артикул: W71280

45 сомони

В наличии: 23 шт.

В корзину

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

В корзину

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

В корзину

Мы не тронули HTML. Ни одного тега. Добавили одну строку `<link>` и файл стилей — и страница превратилась в магазин.

Вот оно, разделение обязанностей из первой главы. HTML говорит **что**, CSS говорит **как**.

Обратите внимание на детали, которые уже работают:

- меню выстроилось в строку (это `display: flex`, разберём в главе 11);
- карточки стали белыми с рамкой и тенью-отступами;
- цена крупная и рыжая, артикул мелкий и серый;
- «Под заказ» приглушено серым, потому что у него два класса: `nalichie nety`;
- наведите мышь на кнопку — она потемнеет, на ссылку в меню — станет рыжей.

ЗАПИСЬ	КАК ЧИТАЕТСЯ	ЧТО ЗНАЧИТ
<code>.cena</code>	«точка цена»	Класс <code>cena</code>
<code>#korzina</code>	«решётка корзина»	Элемент с <code>id="korzina"</code>
<code>.tovar h3</code>	«h3 внутри tovar»	Пробел = «внутри»
<code>h1, h2</code>	«h1 и h2»	Запятая = «и ещё»
<code>a:hover</code>	«а при наведении»	Двоеточие = состояние
<code>*</code>	«все»	Любой элемент
<code>/* ... */</code>	комментарий	Заметка для человека
<code>cursor: pointer</code>		Курсор станет «рукой» — видно, что кликабельно
<code>text-decoration: none</code>		Убрать подчёркивание у ссылки
<code>list-style: none</code>		Убрать точки у списка
<code>box-sizing: border-box</code>		Считать ширину вместе с рамкой и отступами. Разберём в главе 10

Про комментарии в CSS. В HTML они писались `<!-- -->`, в CSS — `/* */`. Разные языки, разные знаки. В PHP и JavaScript будет `//`. Не путайтесь: это тот случай, когда каждый язык изобрёл своё.

Стили не применились вообще. Четыре причины по частоте:

1. Опечатка в пути: файл называется `style.css`, а в `<link>` написано `styles.css`.
2. `<link>` стоит не в `<head>`.
3. Забыли `rel="stylesheet"`.
4. Браузер показывает старую версию — жмите `Ctrl+F5`.

Забыть точку перед классом. `cena { }` ищет тег `<cena>`, которого не бывает. Нужно `.cena { }`. Одна из самых частых ошибок новичка, и браузер про неё молчит.

Забыть точку с запятой. Одно пропущенное `;` ломает следующее объявление, а не своё. Ищешь ошибку в одном месте, а она строкой выше.

Путать `-` и пробел в именах классов. `class="tovar cena"` — это два класса. `class="tovar-cena"` — один класс с дефисом в имени. Разница огромная.

Лечить `!important`. Симптом лечится, болезнь остаётся. Пишите точнее селектор.

Задача 9.1. Подключите `style.css` к своей странице из главы 8 и перенесите туда стили из примера. Убедитесь, что всё применилось.

Задача 9.2. Что выберет каждый селектор? Ответьте словами:

1. `.tovar`
2. `.tovar h3`
3. `nav a:hover`
4. `h1, h2, h3`
5. `#poisk`
6. `footer .telefon`
7. `button:hover`

Задача 9.3. Какой будет цвет текста? Объясните почему:

```
p      { color: red; }  
.text  { color: blue; }  
p.text { color: green; }
```

```
<p class="text">Какого я цвета?</p>
```

Задача 9.4. Поменяйте цветовую схему магазина на свою. Найдите три цвета (основной, фон, акцент) и замените. Сайты для подбора: [coolors.co](#).

Задача 9.5. Добавьте эффекты при наведении:

- карточка товара слегка меняет цвет рамки;
- ссылки в подвале подчёркиваются;
- поле поиска подсвечивается при фокусе (`input:focus`).

Задача 9.6. Сделайте товары «нет в наличии» визуально отличными: приглушите цену, а кнопку сделайте серой. Подсказка: два класса на одном элементе.

Задача 9.7. Откройте любой сайт, F12 → Elements. Выберите элемент — справа увидите его стили. Найдите там правило с зачёркнутым текстом: это правило, которое **проиграло** в каскаде. Найдите, кто его перебил.

Задача 9.8. Прямо в панели F12 поменяйте цвет фона любого сайта. Обновите страницу. Изменения пропали — почему? (Ответ важный, посмотрите обязательно.)

Ответы — в Приложении Г.

В БОЮ

В этом репозитории все наши стили лежат в **одном** файле — `assets/css/custom.css`.
Файлы темы `assets/mazlay-*` не редактируются никогда.

Почему так: тема куплена и может обновиться. Если править её файлы, при обновлении все правки затрутсЯ. Поэтому свои стили пишут **отдельно и подключают позже** — и они перебивают тему по правилу «кто ниже, тот и прав».

Вы только что выучили это правило. Вот его практическое применение на боевом проекте.

Ещё деталь: файл подключается так — `custom.css?v=<время изменения файла>`. Приписка `?v=...` меняется при каждой правке и заставляет браузер скачать файл заново. Это лекарство от того самого «поправил стили, а в браузере старое».

Итог

- CSS-правило: селектор `{` свойство `:` значение `;` `}`.
- Подключать — отдельным файлом через `<link rel="stylesheet" href="style.css">`.
- Инлайн-стили `style="..."` — не надо.
- Селекторы: `тег`, `.класс`, `#id`, `.a .b` (внутри), `a, b` (и ещё), `:hover` (состояние).
- Классы — главный инструмент. Их 90% вашего CSS.
- Каскад: при равной силе побеждает тот, кто ниже; более точный побеждает всегда.
- Сила: тег 1 → класс 10 → id 100 → инлайн 1000.
- **!important** не использовать. Пишите точнее селектор.
- Комментарии в CSS — `/* так */`.

Дальше — свойства: цвет, шрифт, отступы, рамки. И главная головоломка новичка, блочная модель.

Цвет, шрифт, отступы, рамки

ЗАЧЕМ ЭТО

Зачем эта глава

Селекторы вы знаете — умеете указать, кого красить. Теперь разберёмся, **чем** красить.

Свойств в CSS несколько сотен. Учить все не нужно и вредно. Мы возьмём три десятка, которыми пользуются каждый день, и разберём **блочную модель** — ту самую вещь, из-за которой у новичков «непонятно почему всё разъезжается».

Блочная модель — самая важная тема в CSS. Если понять её один раз, половина будущих проблем исчезнет.

РАЗБИРАЕМСЯ

Цвет

► Четыре способа записать цвет

```
color: red; /* по имени */
color: #b5411f; /* шестнадцатеричный */
color: rgb(181, 65, 31); /* по составу */
color: rgba(181, 65, 31, 0.5); /* с прозрачностью */
```

СПОСОБ

КОГДА ПРИМЕНЯЮТ

Имя (red, white)

Быстрая проба. В работе почти не используют — оттенков мало

#b5411f

Основной способ. Так пишут 90% времени

rgb()

Когда цвет считает программа

rgba()

Когда нужна прозрачность

► Как читать #b5411f

Решётка и шесть символов. Это три пары:

```
#b5  41  1f
  ↑   ↑   ↑
красный зелёный синий
```

Каждая пара — количество своего цвета от `00` (нет совсем) до `ff` (максимум).

ЗАПИСЬ	ЧТО ПОЛУЧИТСЯ
<code>#000000</code>	Чёрный (ничего нет)
<code>#ffffff</code>	Белый (всё на максимуме)
<code>#ff0000</code>	Чистый красный
<code>#b5411f</code>	Много красного, немного зелёного, чуть синего → рыжий

Если пары одинаковые, можно писать сокращённо: `#fff` = `#ffffff`, `#000` = `#000000`.

Не нужно вычислять цвета в уме. Их берут из палитры. Полезные сайты: `coolors.co` (генератор палитр), `colorhunt.co` (готовые сочетания).

► Прозрачность

```
background: rgba(181, 65, 31, 0.1); /* 10% непрозрачности */
```

Четвёртое число — от `0` (полностью прозрачный) до `1` (полностью плотный).

Очень полезно для лёгких подложек: тот же цвет, что и акцент, но еле заметный.

► Два разных свойства: `color` и `background`

```
.cena {
  color: white;           /* цвет ТЕКСТА */
  background: #b5411f;    /* цвет ФОНА */
}
```

Путают постоянно. `color` — только про буквы.

► Основные свойства

```
body {  
  font-family: Georgia, 'Times New Roman', serif;  
  font-size: 16px;  
  line-height: 1.6;  
  color: #2b2622;  
}
```

СВОЙСТВО	ЧТО ДЕЛАЕТ
font-family	Какой шрифт
font-size	Размер
font-weight	Насыщенность: <code>normal</code> , <code>bold</code> или число 100–900
font-style	<code>italic</code> для курсива
line-height	Межстрочное расстояние
text-align	Выравнивание: <code>left</code> , <code>center</code> , <code>right</code>
text-decoration	<code>none</code> убирает подчёркивание, <code>underline</code> добавляет
text-transform	<code>uppercase</code> — ВСЁ ЗАГЛАВНЫМИ, <code>lowercase</code> — строчными
letter-spacing	Расстояние между буквами

► Почему в `font-family` несколько шрифтов

```
font-family: Georgia, 'Times New Roman', serif;
```

Это список запасных вариантов. Браузер пробует по очереди: нет Georgia — берёт Times New Roman, нет и его — берёт любой шрифт с засечками.

Последним всегда ставят семейство: `serif` (с засечками), `sans-serif` (без засечек), `monospace` (моноширинный). Так вы гарантируете хоть какой-то разумный результат на любом устройстве.

Названия из двух слов берут в кавычки: `'Times New Roman'`.

► `line-height` — недооценённое свойство

```
line-height: 1.6;
```

Число без единиц означает «в 1,6 раза больше размера шрифта».

Это самое влиятельное свойство для читаемости текста. Строки, стоящие впритык, читать тяжело — глаз теряет строку.

ЗНАЧЕНИЕ	ГДЕ ПРИМЕНЯЮТ
1.2	Крупные заголовки
1.5-1.7	Обычный текст. Золотая середина
2	Очень воздушно, для коротких блоков

Поставьте `line-height: 1.6` в `body` — и сайт сразу станет приятнее. Одна строчка.

► Единицы размера

```
font-size: 16px;      /* пиксели — фиксированно */
font-size: 1.2rem;    /* относительно размера страницы */
font-size: 1.2em;     /* относительно размера родителя */
width: 50%;           /* от ширины родителя */
```


ЕДИНИЦА	СМЫСЛ	КОГДА
px	Пиксель, жёстко	Рамки, мелкие отступы
rem	Кратно базовому размеру страницы (обычно 16px)	Шрифты, отступы. Лучший выбор
em	Кратно размеру родителя	Осторожно: вложенность умножается
%	Доля от родителя	Ширина блоков

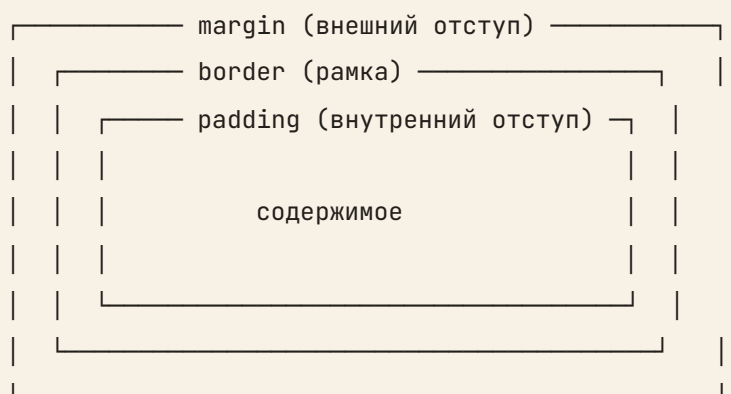
Совет: начинайте с `px` — они понятнее. Когда освоитесь, переходите на `rem`: человек, увеличивший шрифт в настройках браузера, увидит увеличенный сайт, а не поехавший.

РАЗБИРАЕМСЯ

Блочная модель — главная тема главы

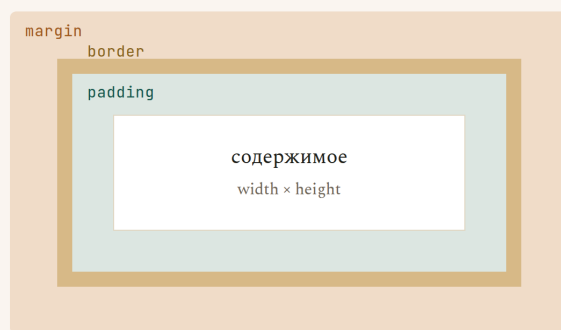
Вот та вещь, из-за которой всё «непонятно разъезжается».

Каждый элемент на странице — это прямоугольная коробка. У коробки четыре слоя, изнутри наружу:



Блочная модель: четыре слоя каждой коробки

Изнутри наружу — и почему width обманывает



- content — содержимое**
Текст, картинка, вложенные блоки. Сам товар
- padding — отступ внутри**
Между содержимым и рамкой. Фон сюда заходит. Пузырчатая плёнка в коробке
- border — рамка**
Стенки коробки. Толщина, тип, цвет
- margin — отступ снаружи**
До соседних блоков. Прозрачный, фон сюда не заходит. Расстояние между коробками на складе

Ловушка: по умолчанию `width: 300px` — это ширина только содержимого, а padding и border добавляются сверху. Итог 344px вместо 300. Лечится одной строчкой: `* { box-sizing: border-box; }` — пишете её первой в каждом проекте.

СЛОЙ	ЧТО ЭТО	АНАЛОГИЯ
content	Само содержимое	Товар
padding	Отступ внутри , между содержимым и рамкой	Пузырчатая плёнка в коробке
border	Рамка	Стенки коробки
margin	Отступ снаружи , до соседей	Расстояние между коробками на складе

► padding и margin — не путайте

Это самая частая путаница у новичков. Запомните так:

- padding** отодвигает содержимое от края своей же коробки. Фон коробки на него распространяется.
- margin** отодвигает коробку от соседей. Он прозрачный — фон туда не заходит.

Практический пример: у кнопки **padding** делает её больше и «толще» (кликать удобнее), а **margin** отодвигает её от текста выше.

► Как их записывать

```
padding: 16px;                /* со всех сторон */
padding: 16px 24px;           /* сверху-снизу | слева-справа */
padding: 8px 16px 12px 16px;  /* сверху | справа | снизу | слева */

padding-top: 16px;            /* только сверху */
padding-left: 24px;           /* только слева */
```

Четыре значения читаются по часовой стрелке, начиная сверху: верх, право, низ, лево.

То же самое работает для `margin`.

► `box-sizing` — строчка, которая всё чинит

А вот главная ловушка блочной модели.

```
.karta {
  width: 300px;
  padding: 20px;
  border: 2px solid #ccc;
}
```

Вопрос: какой ширины получится карточка?

Ответ по умолчанию — **344 пикселя**. Потому что браузер считает так:

```
300 (содержимое) + 20 + 20 (padding) + 2 + 2 (border) = 344
```

То есть `width: 300px` задаёт ширину **содержимого**, а рамка и отступы добавляются сверху. Абсолютно не то, чего ожидает человек.

Отсюда все «почему блоки не влезают в строку» и «почему две колонки по 50% съехали».

Лечится одной строчкой в начале файла:

```
* {
  box-sizing: border-box;
}
```

Теперь `width: 300px` означает 300 пикселей вместе с рамкой и отступами. Как человек и думал.

Пишите это в каждом проекте. Это первое, что добавляют в свой CSS все разработчики.

► Схлопывание внешних отступов

Ещё одна странность, о которой полезно знать заранее.

```
.a { margin-bottom: 20px; }  
.b { margin-top: 30px; }
```

Между блоками будет 30 пикселей, а не 50. Вертикальные `margin` соседей не складываются — берётся больший.

Это не ошибка, а правило CSS. Знать полезно, чтобы не удивляться. Обходится использованием `padding` у родителя или `gap` во флексе (глава 11).

РАЗБИРАЕМСЯ

Рамки, скругления, тени

```
.tovar {  
  border: 1px solid #e3d8c6;  
  border-radius: 8px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);  
}
```

► `border` — рамка

```
border: 1px solid #e3d8c6;  
      ↑   ↑   ↑  
      толщина тип цвет
```

Типы: `solid` (сплошная), `dashed` (пунктир), `dotted` (точки), `none` (нет).

Можно только с одной стороны:

```
border-bottom: 2px solid #b5411f; /* подчёркивание заголовка */
border-left: 4px solid #b5411f; /* цветной корешок у блока */
```

► **border-radius** — скругление

```
border-radius: 8px; /* все углы */
border-radius: 50%; /* круг — если элемент квадратный */
```

Скруглённые углы визуально «смягчают» интерфейс. 6–10 пикселей — универсальное значение для карточек и кнопок.

► **box-shadow** — тень

```
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
           ↑ ↑ ↑      ↑
           сдвиг размытие цвет с прозрачностью
           X Y
```

Главный совет по теням: делайте их еле заметными. **0.08** вместо **0.5**. Тень должна создавать ощущение объёма, а не бросаться в глаза. Резкие чёрные тени — самый узнаваемый признак любительской вёрстки.

```
button {  
    background: #b5411f;  
    color: white;  
    border: none;  
    padding: 10px 20px;  
    border-radius: 6px;  
    font-size: 15px;  
    font-family: inherit;  
    cursor: pointer;  
    transition: background 0.2s;  
}  
  
button:hover {  
    background: #8f3418;  
}  
  
button:active {  
    transform: translateY(1px);  
}
```

Три детали, которые отличают хорошую кнопку:

cursor: pointer — курсор превращается в руку. Без этого человек не понимает, что на элемент можно нажать.

font-family: inherit — кнопки по умолчанию используют системный шрифт, а не ваш. Эта строчка заставляет их взять шрифт страницы.

transition: background 0.2s — цвет меняется плавно за 0,2 секунды, а не рывком. Одно из самых выгодных свойств в CSS: минимум усилий, максимум ощущения качества.

:active — состояние «прямо сейчас нажимают». **translateY(1px)** сдвигает кнопку на пиксель вниз — она как будто продавливается.

СВОЙСТВО	ЧТО ДЕЛАЕТ
<code>color</code>	Цвет текста
<code>background</code>	Цвет фона
<code>font-family</code>	Шрифт, списком запасных вариантов
<code>font-size</code>	Размер шрифта
<code>font-weight</code>	Насыщенность: <code>bold</code> или 100–900
<code>line-height</code>	Межстрочное расстояние. <code>1.6</code> для текста
<code>text-align</code>	Выравнивание
<code>text-decoration: none</code>	Убрать подчёркивание
<code>padding</code>	Отступ внутри коробки
<code>margin</code>	Отступ снаружи , до соседей
<code>border</code>	Рамка: толщина, тип, цвет
<code>border-radius</code>	Скругление углов
<code>box-shadow</code>	Тень
<code>box-sizing: border-box</code>	Считать ширину вместе с рамкой и отступами
<code>cursor: pointer</code>	Курсор-рука
<code>transition</code>	Плавное изменение
<code>inherit</code>	«Взять у родителя»

ГРАБЛИ

Грабли

Забывать `box-sizing: border-box`. Блоки перестают влезать, колонки съезжают. Первая строчка в любом CSS-файле.

Путать `padding` и `margin`. Внутри — `padding`, снаружи — `margin`. Проверить легко: покрасьте элемент в яркий цвет. Фон зальёт `padding`, но не `margin`.

Тени как из фотошопа 2005 года. `rgba(0,0,0,0.08)` — хорошо. `box-shadow: 5px 5px 0 black` — плохо.

Слишком много шрифтов. Два шрифта на сайт: один для заголовков, один для текста. Три — уже перебор.

Мелкий `line-height` в тексте. Значение по умолчанию (`normal` , около 1.2) слишком плотное для чтения. Ставьте 1.5–1.7.

Задавать `width` кнопкам и полям. Лучше `padding` — тогда элемент подстроится под содержимое сам.

ЗАДАЧИ

Задачи

Задача 10.1. Посчитайте в уме итоговую ширину, без `box-sizing: border-box`:

```
.blok { width: 200px; padding: 15px; border: 3px solid black; }
```

А с `border-box` ?

Задача 10.2. Что делает каждая строчка? Опишите словами:

```
padding: 10px 20px;
margin: 0 auto;
border-bottom: 2px solid #b5411f;
border-radius: 50%;
line-height: 1.6;
text-transform: uppercase;
```

(Второе — фокус, который часто встречается. Разберитесь, что он делает.)

Задача 10.3. Оформите карточку товара: белый фон, рамка `1px` светло-серая, скругление `8px` , внутренний отступ `20px` , внешний отступ снизу `16px` , еле заметная тень.

Задача 10.4. Сделайте кнопку «Купить» приятной: свой цвет, белый текст, без рамки, скругление, `cursor: pointer`, плавное потемнение при наведении, продавливание при нажатии.

Задача 10.5. Наберите три цвета для магазина и примените: основной (заголовки), фоновый (страница), акцентный (цены и кнопки). Возьмите на `coolors.co`.

Задача 10.6. Эксперимент с блочной моделью. Сделайте `<div>` с текстом, задайте ему яркий фон и по очереди попробуйте:

1. `padding: 30px` — что изменилось?
2. Уберите, поставьте `margin: 30px` — что изменилось?
3. В чём разница по фону?

Задача 10.7. Откройте F12 → Elements, выберите любой элемент. Внизу справа увидите разноцветную схему блочной модели: синий — содержимое, зелёный — `padding`, жёлтый — `border`, оранжевый — `margin`. Наведите на каждый слой.

Задача 10.8. Найдите на своём сайте три места, где напрашивается `transition`, и добавьте.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `assets/css/custom.css` в этом репозитории. Найдите там правила для карточек товаров.

Увидите то же самое: `border`, `border-radius`, `padding`, `box-shadow`, `transition`. Никакой магии — те же двадцать свойств, что вы только что выучили.

Обратите внимание: цвета в боевом файле вынесены в переменные наверху файла. Это следующий уровень организации — поменял одно значение, изменилось везде. Тоже несложно, дойдём.

ИТОГ

- Цвет: `#b5411f` — основной способ, `rgba(...)` — когда нужна прозрачность.
- `color` — текст, `background` — фон.
- `line-height: 1.6` — самое выгодное свойство для читаемости.
- В `font-family` — список запасных вариантов, последним семейство.
- Блочная модель: `content` → `padding` → `border` → `margin`.
- `padding` внутри, `margin` снаружи.
- `* { box-sizing: border-box; }` — первая строчка любого CSS.
- Вертикальные `margin` соседей схлопываются — берётся больший.
- Тени еле заметные: `rgba(0,0,0,0.08)`.
- `cursor: pointer` и `transition` — две мелочи, которые делают сайт живым.

Дальше — раскладка. Научимся ставить блоки рядом, в сетку и по центру.

Flexbox и Grid: раскладка

ЗАЧЕМ ЭТО**Зачем эта глава**

Ваш каталог сейчас идёт в один столбик. Шесть карточек одна под другой, на широком экране справа пусто.

Причина известна из главы 5: `<article>` — блочный элемент, а блочные занимают всю строку.

В этой главе научимся управлять раскладкой. Двумя инструментами:

- **Flexbox** — выстроить элементы в ряд (или в столбец) и распределить между ними место;
- **Grid** — разложить элементы по сетке из строк и столбцов.

Это самая практичная тема во всём CSS. После неё вы сможете сверстать любую страницу.

РАЗБИРАЕМСЯ**Flexbox: элементы в ряд****► Одна строчка меняет всё**

```
nav ul {  
  display: flex;  
}
```

Всё. Пункты меню, которые стояли столбиком, встали в строку.

Как это работает. Вы говорите родителю: «ты теперь флекс-контейнер». И он начинает выстраивать своих детей в ряд, независимо от того, блочные они или нет.

Два понятия, которые нужно развести:

- **Контейнер** — родитель, которому написали `display: flex;`
- **Элементы** — его прямые дети. Они выстраиваются.

Важно: `display: flex` действует только на прямых детей. Внуки не затрагиваются.

► Направление

```
.spisok {  
  display: flex;  
  flex-direction: row;    /* в строку — по умолчанию */  
}
```

ЗНАЧЕНИЕ

КАК РАССТАВИТ

<code>row</code>	В строку, слева направо (по умолчанию)
<code>column</code>	В столбец, сверху вниз
<code>row-reverse</code>	В строку, справа налево
<code>column-reverse</code>	В столбец, снизу вверх

► Промежутки — `gap`

```
nav ul {  
  display: flex;  
  gap: 20px;  
}
```

`gap` ставит расстояние между элементами. Без внешних краёв, без схлопывания, без возни с `margin`.

Пользуйтесь `gap` вместо `margin` для промежутков во флексе. Это проще и предсказуемее.

► Распределение по горизонтали: `justify-content`

```
header {  
  display: flex;  
  justify-content: space-between;  
}
```

ЗНАЧЕНИЕ**ЧТО ДЕЛАЕТ**

<code>flex-start</code>	Всё влево (по умолчанию)
<code>center</code>	Всё по центру
<code>flex-end</code>	Всё вправо
<code>space-between</code>	Первый влево, последний вправо, остальные равномерно
<code>space-around</code>	Равные промежутки вокруг каждого
<code>space-evenly</code>	Совсем равные промежутки везде

`space-between` — самое востребованное. Это классическая шапка сайта: логотип слева, меню справа, между ними пусто.

► **Выравнивание по вертикали:** `align-items`

```
.товар-строка {  
  display: flex;  
  align-items: center;  
}
```

ЗНАЧЕНИЕ**ЧТО ДЕЛАЕТ**

<code>stretch</code>	Растянуть на всю высоту (по умолчанию)
<code>center</code>	По центру вертикали
<code>flex-start</code>	По верхнему краю
<code>flex-end</code>	По нижнему краю
<code>baseline</code>	По базовой линии текста

`align-items: center` — решение вечной задачи «как выровнять по вертикали». До Flexbox это была настоящая головная боль с костылями. Теперь — одна строчка.

► **Как запомнить, где какое**

Частая путаница: `justify-content` или `align-items`?

Запомните привязку к направлению:

- `justify-content` работает вдоль основного направления (при `row` — по горизонтали);
- `align-items` — поперёк (при `row` — по вертикали).

Если поставить `flex-direction: column`, они поменяются ролями. Логично, но поначалу непривычно.

► Перенос на новую строку

```
.katalog {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
```

По умолчанию флекс сжимает элементы, лишь бы уместить в одну строку. `flex-wrap: wrap` разрешает переносить их на следующую.

Для каталога товаров это обязательно. Без него шесть карточек сожмутся в ниточки.

► Как элементы делят место: `flex`

```
.tovar {
  flex: 1;           /* все поровну */
}
```

```
.osnovnoe { flex: 2; } /* вдвое шире */
.bokovaya { flex: 1; }
```

`flex: 1` означает «занимай равную долю свободного места».

Практический приём для карточек:

```
.tovar {
  flex: 1 1 280px;
}
```

Читается: «расти можно, сжиматься можно, базовая ширина 280px». Карточки сами заполнят ряд, а лишние перенесутся вниз. Ровно то, что нужно каталогу.

► Центрирование чего угодно

```
.centr {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 300px;  
}
```

Три строчки — и содержимое ровно по центру и по горизонтали, и по вертикали. Задача, которая двадцать лет была мемом среди верстальщиков.

РАЗБИРАЕМСЯ

Grid: раскладка по сетке

Flexbox думает одной линией — ряд или столбец. Grid думает двумя измерениями сразу — строки и столбцы.

► Простая сетка

```
.katalog {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}
```

Три колонки одинаковой ширины, промежуток 20 пикселей.

fr — новая единица, «доля свободного места». **1fr 1fr 1fr** = три равные доли.

```
grid-template-columns: 2fr 1fr;           /* левая вдвое шире */  
grid-template-columns: 300px 1fr;        /* левая фиксированная, правая тянется */  
grid-template-columns: repeat(4, 1fr); /* то же, что 1fr 1fr 1fr 1fr */
```

► Самая полезная строчка Grid

```
.katalog {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));  
  gap: 20px;  
}
```

Выглядит сложно, читается просто:

КУСОК

ЧТО ГОВОРИТ

```
repeat(auto-fill, ...)
```

«Сделай столько колонок, сколько влезет»

```
minmax(260px, 1fr)
```

«Каждая колонка не уже 260px, но может растянуться»

Что это даёт: на широком мониторе получится пять колонок, на ноутбуке три, на планшете две, на телефоне одна. Автоматически, без единого медиазапроса.

Это лучшее решение для каталога товаров, и его стоит просто запомнить наизусть.

► Раскладка всей страницы

Grid умеет то, чего не умеет Flexbox, — расставлять блоки по именованным областям:

```
body {  
  display: grid;  
  grid-template-areas:  
    "shapka shapka"  
    "bok     osnova"  
    "podval podval";  
  grid-template-columns: 240px 1fr;  
  gap: 20px;  
}  
  
header { grid-area: shapka; }  
aside  { grid-area: bok; }  
main   { grid-area: osnova; }  
footer { grid-area: podval; }
```

Раскладка нарисована прямо в коде — видно глазами, как выглядит страница.

► Flexbox или Grid?

Простое правило:

ЗАДАЧА	ИНСТРУМЕНТ
Элементы в ряд: меню, кнопки, шапка	Flexbox
Выровнять по центру	Flexbox
Сетка карточек: каталог, галерея	Grid
Раскладка всей страницы	Grid
Содержимое внутри карточки	Flexbox

И главное: **они не конкуренты, а напарники**. Обычная страница — Grid снаружи, Flexbox внутри каждой ячейки.

Добавьте в `style.css` :

```
/* Каталог — автоматическая сетка */
.katalog {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 20px;
  margin-top: 20px;
}

/* Карточка — колонка, кнопка прижата вниз */
.tovar {
  display: flex;
  flex-direction: column;
  background: white;
  border: 1px solid #e3d8c6;
  border-radius: 8px;
  padding: 18px;
  transition: box-shadow 0.2s, border-color 0.2s;
}

.tovar:hover {
  border-color: #b5411f;
  box-shadow: 0 4px 16px rgba(181, 65, 31, 0.12);
}

/* Пустое место «распирает» карточку, прижимая кнопку к низу */
.tovar form {
  margin-top: auto;
  padding-top: 12px;
}

/* Шапка: название слева, меню справа */
header {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
```

```

}

/* Фильтры в строку */
.filtrы fieldset {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    gap: 12px;
    border: 1px solid #e3d8c6;
    border-radius: 8px;
    padding: 16px;
}

```

В HTML нужно завернуть товары в контейнер:

```

<div class="katalog">
    <article class="tovar">... </article>
    <article class="tovar">... </article>
    ...
</div>

```

► Приём с `margin-top: auto`

Обратите внимание на одну хитрость:

```

.tovar { display: flex; flex-direction: column; }
.tovar form { margin-top: auto; }

```

`margin-top: auto` во флексе означает «забери всё свободное место сверху». Кнопка прижимается к низу карточки.

Зачем: у товаров названия разной длины, карточки получаются разной высоты, и кнопки оказываются на разных уровнях — выглядит неряшливо. С этим приёмом кнопки всегда на одной линии.

Мелочь, которая отличает аккуратную вёрстку от небрежной.

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

[Главная](#)[Каталог](#)[Доставка](#)[Контакты](#)Поиск по каталогу

Каталог запчастей

Подобрать

Бренд

Цена от

до

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

Масляный фильтр Mann W71280

Артикул: W71280

45 сомони

В наличии: 23 шт.

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

Воздушный фильтр Mann C25114

Артикул: C25114

65 сомони

В наличии: 14 шт.

Аккумулятор Bosch S4 60Ah

Артикул: 0092540050

950 сомони

В наличии: 4 шт.

Тот же HTML, те же товары — но теперь это выглядит как магазин, а не как список.

Попробуйте изменить ширину окна браузера: колонки перестроятся сами. Мы не написали для этого ни одного условия — работает `auto-fill` с `minmax`.

СВОЙСТВО	ГДЕ ПИШЕТСЯ	ЧТО ДЕЛАЕТ
<code>display: flex</code>	у родителя	Выстроить детей в ряд
<code>flex-direction</code>	у родителя	<code>row</code> (строка) или <code>column</code> (столбец)
<code>gap</code>	у родителя	Промежуток между элементами
<code>justify-content</code>	у родителя	Распределение вдоль направления
<code>align-items</code>	у родителя	Выравнивание поперёк
<code>flex-wrap: wrap</code>	у родителя	Разрешить перенос на новую строку
<code>flex: 1</code>	у ребёнка	Занять равную долю места
<code>margin-top: auto</code>	у ребёнка	Прижать к низу
<code>display: grid</code>	у родителя	Включить сетку
<code>grid-template-columns</code>	у родителя	Сколько колонок и какой ширины
<code>1fr</code>		Доля свободного места
<code>repeat(auto-fill, minmax(260px, 1fr))</code>		Столько колонок, сколько влезет
<code>grid-template-areas</code>	у родителя	Раскладка страницы «картинкой»

ГРАБЛИ

Грабли

Писать `display: flex` не тому элементу. Флекс задаётся родителю, а выстраиваются дети. Хотите поставить карточки в ряд — `display: flex` пишется контейнеру, а не карточке.

Забывать `flex-wrap: wrap`. Восемь карточек сожмутся в узкие полоски вместо переноса на

вторую строку.

Путать `justify-content` и `align-items`. Запомните: `justify` — вдоль, `align` — поперёк.

Задавать карточкам жёсткую `width`. Ломает и Flexbox, и Grid. Пусть шириной управляет раскладка: `flex: 1 1 280px` или `minmax()`.

Строить сетку карточек на Flexbox. Можно, но в последнем ряду карточки растянутся и станут шире остальных. Grid такого не делает — для сеток берите Grid.

Городить `float`. Если увидите этот способ в старой статье — не берите. `float` придуман для обтекания картинки текстом, раскладку им делали от безысходности до появления Flexbox.

ЗАДАЧИ

Задачи

Задача 11.1. Переведите каталог на Grid по примеру из главы. Проверьте, как перестраивается при изменении ширины окна.

Задача 11.2. Что произойдёт в каждом случае?

```
.a { display: flex; justify-content: center; }  
.b { display: flex; flex-direction: column; align-items: center; }  
.c { display: flex; justify-content: space-between; align-items: center; }  
.d { display: grid; grid-template-columns: 1fr 2fr; }
```

Задача 11.3. Сделайте шапку: название слева, меню справа, по центру по вертикали. Одним `display: flex`.

Задача 11.4. Выводите блок ровно по центру экрана — и по горизонтали, и по вертикали. Три строчки.

Задача 11.5. Сделайте карточку товара горизонтальной: фото слева, описание справа, кнопка прижата к правому краю.

Задача 11.6. Сколько будет колонок при ширине окна 1200px, если написано `repeat(auto-fill, minmax(280px, 1fr))` и `gap: 20px`? Посчитайте, потом проверьте в браузере.

Задача 11.7. Разложите страницу через `grid-template-areas`: шапка на всю ширину, слева фильтры 250px, справа каталог, внизу подвал на всю ширину.

Задача 11.8. Откройте flexboxfroggy.com — игра, где нужно довести лягушку до листика с помощью свойств Flexbox. 24 уровня, полчаса времени, и Flexbox укладывается в голову намертво. Есть русская версия.

Ответы — в Приложении Г.

В БОЮ В бою

Магазин autodoc.tj построен на Bootstrap — это готовый набор классов для раскладки. Там пишут не `display: flex`, а `class="row"` и `class="col-md-4"`.

Под капотом у Bootstrap — тот же Flexbox. Классы просто дают ему готовые имена.

Вывод, который стоит запомнить: сначала изучают, как работает механизм, потом — готовые надстройки над ним. Кто выучил только классы Bootstrap, беспомощен, как только нужно что-то нестандартное. Кто понимает Flexbox, разберётся в любом фреймворке за вечер.

ИТОГ

- **Flexbox** — одна линия: ряд или столбец. **Grid** — сетка из строк и столбцов.
- `display: flex` пишется **родителю**, выстраиваются **дети**.
- `gap` — промежутки. Лучше, чем `margin`.
- `justify-content` — **вдоль** направления, `align-items` — **поперёк**.
- `flex-wrap: wrap` обязателен, если элементов много.
- Центрирование: `display: flex; justify-content: center; align-items: center;`
- **Каталог товаров:** `grid-template-columns: repeat(auto-fill, minmax(260px, 1fr))`.
Запомните наизусть — работает без медиазапросов.
- `margin-top: auto` прижимает кнопку к низу карточки.
- Обычная страница: **Grid снаружи, Flexbox внутри**.

Дальше — адаптивность: сделаем так, чтобы сайт был удобен на телефоне.

Адаптивность: сайт на телефоне

ЗАЧЕМ ЭТО

Зачем эта глава

Один факт, который меняет приоритеты: больше половины покупателей зайдут в ваш магазин с телефона. В Таджикистане доля мобильных ещё выше — многие вообще не пользуются компьютером.

Это значит, что сайт, удобный только на большом экране, — это сайт, неудобный для большинства.

В этой главе научимся делать так, чтобы одна и та же страница выглядела правильно и на мониторе, и на телефоне. Инструментов понадобится немного, и один вы уже видели.

РАЗБИРАЕМСЯ

Что такое адаптивность

Адаптивный сайт — тот, который подстраивается под ширину экрана.

Не «мобильная версия на отдельном адресе» — так делали пятнадцать лет назад, и это означало поддерживать два сайта. Сегодня подход другой: один HTML, один CSS, разное поведение в зависимости от ширины.

Три вещи, которые для этого нужны:

1. `<meta name="viewport">` — вы уже написали её в главе 8;
2. Гибкие размеры вместо жёстких;
3. Медиазапросы — правила, которые включаются при определённой ширине.

Половина проблем решается до всяких медиазапросов — просто отказом от жёстких чисел.

► Что заменить

```
/* Плохо — не поместится на телефоне */  
.karta { width: 400px; }  
  
/* Хорошо — подстроится */  
.karta { max-width: 400px; width: 100%; }
```

`max-width` означает «не шире, чем», но уже — можно. На широком экране карточка будет 400px, на узком — по ширине экрана.

Правило: пишите `max-width` вместо `width` почти всегда.

► Контейнер по центру

```
.container {  
  max-width: 1200px;  
  margin: 0 auto;  
  padding: 0 16px;  
}
```

Три строчки, которые есть на любом сайте:

- `max-width` — на огромном мониторе текст не растянется на всю ширину (читать строку в 300 символов невозможно);
- `margin: 0 auto` — **центрирование блока**. `auto` слева и справа делит свободное место поровну;
- `padding: 0 16px` — на телефоне текст не прилипнет к краям экрана.

► Гибкие картинки

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Две строчки, которые нужно написать в каждом проекте. Без них большое фото вылезет за края экрана и появится горизонтальная прокрутка — одна из самых раздражающих вещей на телефоне.

`height: auto` сохраняет пропорции при сжатии.

РАЗБИРАЕМСЯ

Медиазапросы

Теперь главный инструмент.

Медиазапрос — это блок правил, которые включаются только при определённом условии. Обычно условие — ширина экрана.

```
/* Обычные правила — работают всегда */  
.katalog {  
  grid-template-columns: repeat(3, 1fr);  
}  
  
/* А это включится только на экранах уже 768px */  
@media (max-width: 768px) {  
  .katalog {  
    grid-template-columns: 1fr;  
  }  
}
```

Читается: «если ширина экрана не больше 768 пикселей — сделать одну колонку».

► Как это устроено

```
@media (max-width: 768px) { ... }  
@media (min-width: 1024px) { ... }  
@media (min-width: 768px) and (max-width: 1023px) { ... }
```

УСЛОВИЕ

КОГДА СРАБОТАЕТ

max-width: 768px	Экран уже или равен 768px
min-width: 1024px	Экран шире или равен 1024px
and	Оба условия сразу

⚠ Частая путаница: `max-width` в медиазапросе означает «до этой ширины», то есть более узкие экраны. Легко перепутать с `max-width` у элемента, где смысл другой.

► Типовые точки перелома

```
/* Телефон:      до 600px */  
/* Планшет:      600-900px */  
/* Ноутбук:      900-1200px */  
/* Большой экран: от 1200px */
```

Эти числа не священны. Правильный подход — сужать окно браузера и смотреть, где вёрстка начинает выглядеть плохо. Вот там и ставить точку перелома.

Гнаться за конкретными моделями телефонов бессмысленно: их сотни, и каждый год выходят новые.

Есть два способа писать адаптивный CSS.

► Способ 1: от большого к малому

```
.katalog { grid-template-columns: repeat(4, 1fr); }

@media (max-width: 900px) { .katalog { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 600px) { .katalog { grid-template-columns: 1fr; } }
```

Сначала пишем для компьютера, потом «чиним» для телефона.

► Способ 2: mobile-first — от малого к большому

```
.katalog { grid-template-columns: 1fr; }

@media (min-width: 600px) { .katalog { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 900px) { .katalog { grid-template-columns: repeat(4, 1fr); } }
```

Сначала пишем для телефона, потом добавляем колонки на широких экранах.

► Почему mobile-first лучше

Практическая причина. Телефон получает только базовые правила — простые и лёгкие. Компьютер получает базовые плюс дополнительные. Телефон, у которого обычно слабее и связь, и процессор, делает меньше работы.

Причина посерьёзнее — про мышление. Начиная с узкого экрана, вы вынуждены сразу отвечать на вопрос: *что здесь действительно важно?* На 360 пикселях не поместится всё — придётся выбирать главное.

А потом на большом экране добавить второстепенное — легко.

Если начать с компьютера, вы напихаете всего, а потом будете мучительно решать, что выбросить. Обычно кончается тем, что не выбрасывают ничего, и на телефоне получается каша.

Добавьте в конец `style.css` :

```
/* ===== Основа: пишем для телефона ===== */

.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 16px;
}

img {
    max-width: 100%;
    height: auto;
}

body {
    font-size: 16px;
    padding: 16px;
}

h1 { font-size: 24px; }
h2 { font-size: 20px; }

/* Шапка на телефоне — в столбик */
header {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    gap: 12px;
}

/* Меню переносится, если не влезает */
nav ul {
    display: flex;
    flex-wrap: wrap;
    gap: 16px;
    list-style: none;
}
```

```

/* Каталог: одна колонка */
.katalog {
    display: grid;
    grid-template-columns: 1fr;
    gap: 16px;
}

/* Фильтры в столбик */
.filtr try fieldset {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

.filtr try input,
.filtr try select {
    width: 100%;
    padding: 10px;
    font-size: 16px;
}

/* Кнопки на телефоне — во всю ширину, пальцем не промахнёшься */
button {
    width: 100%;
    padding: 14px;
    font-size: 16px;
}

/* ===== Планшет: от 600px ===== */
@media (min-width: 600px) {
    body { padding: 24px; }
    h1 { font-size: 28px; }

    .katalog { grid-template-columns: repeat(2, 1fr); }

    .filtr try fieldset {
        flex-direction: row;
        flex-wrap: wrap;
        align-items: center;
    }

    .filtr try input,

```

```

        .filtry select { width: auto; }

        button { width: auto; }
    }

    /* ===== Компьютер: от 900px ===== */
    @media (min-width: 900px) {
        h1 { font-size: 32px; }

        header {
            flex-direction: row;
            justify-content: space-between;
            align-items: center;
        }

        .katalog {
            grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
        }
    }
}

```

Заметьте порядок: сначала — как на телефоне, потом добавки.

РАЗБИРАЕМСЯ

Мелочи, которые решают на телефоне

Несколько вещей, о которых обычно не пишут в учебниках, но которые сразу видно живому человеку.

► Размер шрифта в полях — минимум 16px

```

input, select, textarea {
    font-size: 16px;
}

```

Почему именно 16. Safari на iPhone автоматически увеличивает страницу, когда человек нажимает на поле со шрифтом меньше 16px. Выглядит как сбой: человек тапнул в поле — сайт «прыгнул».

Ровно 16px и больше — не прыгает.

► Кнопки не меньше 44 пикселей

```
button, .knopka {  
    min-height: 44px;  
    padding: 12px 20px;  
}
```

44 пикселя — размер, который Apple вывела как минимальный для уверенного попадания пальцем. Меньше — человек промахивается и раздражается.

► Ссылки в меню — с отступами

Маленькие ссылки рядом друг с другом на телефоне — беда. Отступ 12–16px между ними спасает от случайных нажатий.

► Горизонтальная прокрутка — всегда ошибка

Если на телефоне страницу можно двигать вбок — что-то вылезло за края. Обычно виновники: широкая картинка без `max-width: 100%`, таблица, блок с жёсткой `width` или длинное слово без переносов.

Ищется так: в F12 включите режим телефона и постепенно сужайте — увидите, на чём начинается прокрутка.

► Таблицы на телефоне

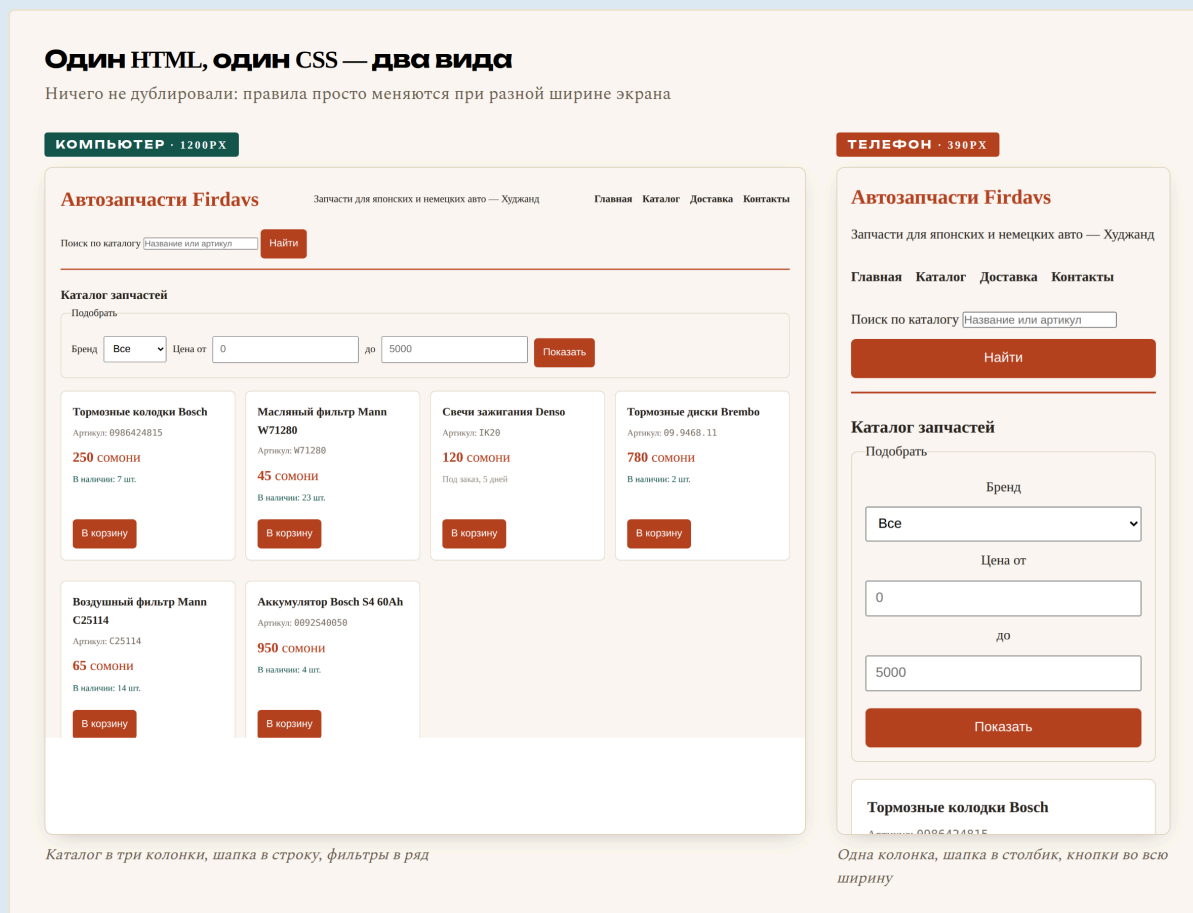
Таблица шириной в шесть колонок на экране 360px не помещается никак. Простейшее решение — разрешить ей прокручиваться внутри своего блока:

```
.tablica-obertka {  
    overflow-x: auto;  
}
```

```
<div class="tablica-obertka">  
    <table>...</table>  
</div>
```

Теперь прокручивается только таблица, а страница остаётся на месте.

Тот же самый файл, та же страница. Слева — на компьютере, справа — на телефоне шириной 390 пикселей:



Тормозные колодки Bosch
Артикул: 0986424815

Мы не делали вторую версию сайта. Один HTML, один CSS — просто правила меняются при разной ширине.

Обратите внимание, что изменилось на узком экране:

- каталог стал одноколоночным;
- шапка встала в столбик;
- фильтры выстроились вертикально и растянулись на всю ширину;
- кнопки стали во всю ширину — пальцем не промахнуться;
- поля крупные, страница не прыгает при нажатии.

Способ 1: F12 → значок телефона. В панели разработчика есть кнопка переключения на мобильный вид (слева вверху панели, значок телефона с планшетом). Можно выбрать модель или задать ширину вручную.

Способ 2: просто сузить окно браузера. Грубее, но быстро и показывает, где вёрстка ломается.

Способ 3: открыть на настоящем телефоне. Самый честный. Пока сайт лежит у вас на компьютере, откройте его так:

```
php -S 0.0.0.0:8000
```

Узнайте свой IP в локальной сети (команда `ipconfig` на Windows, `ifconfig` или `ip a` на Mac и Linux) и откройте на телефоне `http://192.168.1.5:8000`, подставив свой адрес. Компьютер и телефон должны быть в одной Wi-Fi-сети.

Обязательно проверяйте на настоящем телефоне. Эмулятор в браузере не покажет, как реагирует ваш палец, как ведёт себя клавиатура и насколько мелкий на самом деле текст.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>@media (max-width: 768px)</code>	Правила для экранов уже 768px
<code>@media (min-width: 900px)</code>	Правила для экранов шире 900px
<code>max-width</code> у элемента	«Не шире, чем». Уже — можно
<code>margin: 0 auto</code>	Центрировать блок по горизонтали
<code>width: 100%</code>	Во всю ширину родителя
<code>overflow-x: auto</code>	Разрешить горизонтальную прокрутку внутри блока
<code>min-height: 44px</code>	Минимальная высота — палец попадёт
mobile-first	Сначала стили для телефона, потом добавки для широких экранов
точка перелома	Ширина, на которой меняется раскладка

ГРАБЛИ

Грабли

Забывать `<meta name="viewport">`. Всё остальное бессмысленно: телефон уменьшит страницу целиком, и медиазапросы даже не сработают. Проверьте первым делом.

Жёсткая `width` у блоков. Главная причина горизонтальной прокрутки. Замените на `max-width`.

Шрифт в полях меньше 16px. Страница будет прыгать на iPhone.

Слишком много точек перелома. Три-четыре достаточно. Десять — и вы сами перестанете понимать, что где включается.

Прятать содержимое на телефоне через `display: none`. Соблазнительно («не влезает — уберу»), но человек с телефона хочет того же, что и с компьютера. Не прячьте, а перестраивайте.

Проверять только в эмуляторе. Откройте на настоящем телефоне хотя бы раз — обязательно найдёте что-то, чего эмулятор не показал.

Задача 12.1. Сделайте свой каталог адаптивным по примеру из главы. Проверьте на трёх ширинах: 360px, 768px, 1200px.

Задача 12.2. На какой ширине работает каждый запрос?

```
@media (max-width: 600px) { }  
@media (min-width: 900px) { }  
@media (min-width: 600px) and (max-width: 899px) { }
```

Задача 12.3. Перепишите в стиле mobile-first:

```
.blok { display: flex; gap: 20px; }  
@media (max-width: 700px) {  
  .blok { display: block; }  
}
```

Задача 12.4. Сузьте окно браузера на своём сайте до 320px — самый узкий современный телефон. Появилась горизонтальная прокрутка? Найдите виновника и почините.

Задача 12.5. Сделайте таблицу прайса из главы 7 прокручиваемой на телефоне.

Задача 12.6. Откройте свой сайт на настоящем телефоне. Запишите три вещи, которые оказались неудобными. Почините их.

Задача 12.7. Проверьте свой сайт в pagespeed.web.dev — это бесплатный инструмент Google. Он покажет оценку для мобильных и список проблем. Какая оценка? Что он предлагает исправить?

Задача 12.8. Найдите на любом сайте место, где на телефоне неудобно. Сформулируйте, почему именно, и как бы вы это исправили. Умение замечать неудобство — половина работы дизайнера.

Ответы — в Приложении Г.

В БОЮ

В бою

Загляните в `assets/css/custom.css` и поищите `@media`. Найдёте несколько блоков.

Самое интересное там — правила для меню каталога: на компьютере это боковая колонка, на телефоне оно превращается в выпадающий список. Не спрятано, а **перестроено** — ровно то, о чём мы говорили.

И маленький штрих из жизни: карточки товаров на боевом сайте специально сделаны с крупными кнопками. Не для красоты — потому что покупатель обычно стоит у машины, на улице, одной рукой держит телефон, и промахиваться ему нечем.

ИТОГ

Итог

- Больше половины покупателей придут с телефона. Это не «потом», это сразу.
- Три опоры адаптивности: **viewport**, **гибкие размеры**, **медиазапросы**.
- `max-width` вместо `width`. `img { max-width: 100%; height: auto; }` в каждый проект.
- `margin: 0 auto` центрирует блок.
- `@media (max-width: ...)` — для узких, `(min-width: ...)` — для широких.
- **Mobile-first**: сначала телефон, потом добавки. Заставляет выбрать главное.
- Шрифт в полях — **минимум 16px**, иначе iPhone будет прыгать.
- Кнопки — **минимум 44px** высотой.
- Горизонтальная прокрутка на телефоне — всегда ошибка.
- Не прячьте содержимое на телефоне. Перестраивайте.
- Проверяйте на **настоящем** телефоне.

Следующая глава — практическая. Доведём магазин до вида, который не стыдно показать.

Практика: каталог становится красивым

ЗАЧЕМ ЭТО

Зачем эта глава

Каталог у вас уже работает и перестраивается под телефон. Но между «работает» и «выглядит хорошо» есть расстояние, и сегодня мы его пройдем.

Разберём то, о чём обычно не говорят в учебниках по CSS, а зря — потому что именно это отличает сайт, сделанный дизайнером, от сайта, сделанного программистом:

- **переменные** — как перекрасить весь сайт, поменяв три строчки;
- **шрифты** — как подключить нормальный шрифт вместо системного;
- **шесть правил вёрстки**, которые сразу видно;
- **состояния** — что показывать, когда товаров нет.

К концу главы у вас будет магазин, который не стыдно показать.

РАЗБИРАЕМСЯ

Переменные CSS

► Проблема

Откройте свой `style.css` и посчитайте, сколько раз там встречается `#b5411f`. Наверняка раз восемь: заголовок, цена, кнопка, наведение на кнопку, рамка, линия в шапке.

Теперь представьте, что заказчик просит сменить фирменный цвет. Восемь замен, и одну вы точно пропустите.

► Решение

```
:root {  
  --cvet-osnovnoy: #b5411f;  
  --cvet-temnyi: #8f3418;  
  --cvet-text: #2b2622;  
  --cvet-tihii: #7c7166;  
  --cvet-fon: #faf7f2;  
  --cvet-karta: #ffffff;  
  --cvet-ramka: #e3d8c6;  
  --cvet-uspeh: #14584f;  
  
  --radius: 8px;  
  --otstup: 16px;  
  --ten: 0 2px 8px rgba(34, 30, 26, 0.06);  
}
```

Теперь используем:

```
h1 { color: var(--cvet-osnovnoy); }  
button { background: var(--cvet-osnovnoy); border-radius: var(--radius); }  
.tovar { border: 1px solid var(--cvet-ramka); box-shadow: var(--ten); }
```

ЧАСТЬ

ЧТО ЗНАЧИТ

:root	«Корень документа» — переменные видны везде
--cvet-osnovnoy	Имя переменной. Обязательно два дефиса в начале
var(--cvet-osnovnoy)	Подставить значение

Что вы получаете. Меняете одну строчку — перекрашивается весь сайт. Можно за минуту показать заказчику три варианта цветовой схемы.

Как называть переменные. По смыслу, а не по цвету:

```
--cvet-osnovnoy: #b5411f;  ✓ понятно, зачем  
--ryzhiy: #b5411f;        ✗ а если станет синим?
```

Это общее правило для программирования: имена дают **по роли**, а не по текущему значению.

Системные шрифты (Georgia, Arial) есть у всех, но выглядят обыденно. Свой шрифт сразу поднимает уровень.

► Как подключить

```
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Manrope:wght@400;600;800"
    rel="stylesheet">
</head>
```

```
body {
  font-family: 'Manrope', system-ui, sans-serif;
}
```

Шрифты берутся с fonts.google.com — бесплатно, в том числе для коммерческих сайтов.

⚠ **Проверяйте кириллицу.** Далеко не все шрифты её поддерживают. На Google Fonts есть фильтр по языкам — выберите Cyrillic, иначе русский текст подставится другим шрифтом и будет выглядеть чужеродно.

► Хорошие пары для магазина

ЗАГОЛОВКИ	ТЕКСТ	НАСТРОЕНИЕ
Manrope	Manrope	Современно, чисто
Unbounded	Golos Text	Ярко, характерно
Playfair Display	Source Sans 3	Дорого, солидно
Oswald	PT Sans	Плотно, по-деловому

Правило: два шрифта максимум. Три — уже разнобой. Многие хорошие сайты обходятся одним, играя насыщенностью и размером.

► Сколько начертаний брать

```
family=Manrope:wght@400;600;800
```

Каждое начертание — отдельный файл, который скачает браузер. Три штуки достаточно: обычное (400), полужирное (600), очень жирное (800). Не подключайте все девять — сайт будет открываться дольше.

РАЗБИРАЕМСЯ

Шесть правил, которые сразу видно

Не дизайнерская теория, а конкретные приёмы. Примените — и разница будет заметна.

► 1. Отступы кратны одному числу

Возьмите базу — обычно 8 пикселей — и используйте только кратные: 8, 16, 24, 32, 40, 48.

```
:root {  
  --s1: 8px;  
  --s2: 16px;  
  --s3: 24px;  
  --s4: 32px;  
}
```

Почему работает: случайные 13px, 17px, 22px создают ощущение неаккуратности, даже если человек не может объяснить, что его смущает. Кратная сетка выглядит собранно.

► 2. Меньше цветов

Три-четыре цвета, не больше:

- основной (акцент, кнопки, цены);
- тёмный (текст);
- светлый (фон);
- один служебный (успех/ошибка).

Радуга из десяти цветов — верный признак любительской работы.

► 3. Меньше размеров шрифта

Четыре-пять размеров на весь сайт:

```
--text-mal: 14px; /* мелочи: артикул, примечания */
--text-osn: 16px; /* основной текст */
--text-bol: 20px; /* цена, подзаголовки */
--text-zag: 28px; /* заголовки разделов */
--text-glav: 36px; /* заголовок страницы */
```

► 4. Воздух важнее украшений

Самая частая ошибка новичка — всё лепится друг к другу.

Увеличьте отступы вдвое от того, что кажется достаточным. Почти всегда станет лучше.

Пустое место — не потерянное место, а инструмент: оно показывает, что с чем связано.

► 5. Выравнивание по одной линии

Всё, что можно выровнять, должно быть выровнено. Заголовки, текст, кнопки в карточках — по одной вертикали.

Помните приём `margin-top: auto` из главы 11? Он ровно для этого: кнопки во всех карточках оказываются на одной линии.

► 6. Контраст текста

Светло-серый текст на белом фоне выглядит «дизайнерски», но читается плохо, особенно на телефоне при солнце.

Минимум — `#666` на белом. Лучше темнее. Проверить можно на webaim.org/resources/contrastchecker.

РАЗБИРАЕМСЯ

Состояния, о которых забывают

Ваш каталог показывает шесть товаров. А что будет, если товаров ноль?

Три состояния, которые новички не делают, а потом получают «сайт сломался»:

► Пусто

```
<div class="pusto">
  <p class="pusto-znak">🔍</p>
  <h3>Ничего не нашлось</h3>
  <p>Попробуйте изменить фильтры или поискать по артикулу.</p>
  <a href="/catalog" class="knopka">Показать все товары</a>
</div>
```

```
.pusto {
  text-align: center;
  padding: 48px 24px;
  color: var(--cvet-tihii);
}

.pusto-znak { font-size: 40px; margin-bottom: 8px; }
```

Важно: пустой экран должен подсказывать, что делать дальше. Не просто «ничего не найдено», а кнопка выхода из тупика.

► Загрузка

Когда данные ещё едут, показывают заглушки-«скелеты» вместо пустоты:

```
.skelet {
  background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%);
  background-size: 200% 100%;
  animation: pereliv 1.4s infinite;
  border-radius: var(--radius);
  height: 180px;
}

@keyframes pereliv {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}
```

Это серый прямоугольник с бегущим бликом. Человек видит, что процесс идёт, и не думает, что сайт завис.

► Ошибка

```
<div class="oshibka">  
  <p>Не удалось загрузить товары. Проверьте связь и попробуйте снова.</p>  
  <button>Повторить</button>  
</div>
```

Правило: сообщение об ошибке должно говорить, что делать, а не только что случилось.

Плохо: «Ошибка 500». Хорошо: «Не удалось загрузить. Попробуйте обновить страницу».

```
/* ===== Переменные ===== */
:root {
  --cvet-osnovnoy: #b5411f;
  --cvet-temnyi: #8f3418;
  --cvet-text: #2b2622;
  --cvet-tihii: #7c7166;
  --cvet-fon: #faf7f2;
  --cvet-karta: #ffffff;
  --cvet-ramka: #e8dfd0;
  --cvet-uspeh: #14584f;

  --s1: 8px; --s2: 16px; --s3: 24px; --s4: 32px;
  --radius: 10px;
  --ten: 0 2px 8px rgba(34, 30, 26, 0.05);
  --ten-navel: 0 8px 24px rgba(181, 65, 31, 0.14);
}

/* ===== Сброс ===== */
* { margin: 0; padding: 0; box-sizing: border-box; }

img { max-width: 100%; height: auto; display: block; }

body {
  font-family: 'Manrope', system-ui, -apple-system, sans-serif;
  font-size: 16px;
  line-height: 1.6;
  color: var(--cvet-text);
  background: var(--cvet-fon);
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: var(--s3) var(--s2);
}

/* ===== Шапка ===== */
header {
  display: flex;
```

```

    flex-direction: column;
    gap: var(--s2);
    padding-bottom: var(--s3);
    border-bottom: 2px solid var(--cvet-osnovnoy);
    margin-bottom: var(--s4);
}

.logo {
    font-size: 26px;
    font-weight: 800;
    color: var(--cvet-osnovnoy);
    letter-spacing: -0.02em;
}

.slogan { color: var(--cvet-tihii); font-size: 14px; }

nav ul { display: flex; flex-wrap: wrap; gap: var(--s3); list-style: none; }

nav a {
    color: var(--cvet-text);
    text-decoration: none;
    font-weight: 600;
    padding: var(--s1) 0;
    border-bottom: 2px solid transparent;
    transition: color .2s, border-color .2s;
}

nav a:hover {
    color: var(--cvet-osnovnoy);
    border-bottom-color: var(--cvet-osnovnoy);
}

/* ===== Поиск ===== */
.poisk { display: flex; gap: var(--s1); }

.poisk input {
    flex: 1;
    padding: 12px var(--s2);
    font-size: 16px;
    font-family: inherit;
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
}

```

```

        background: var(--cvet-karta);
        transition: border-color .2s, box-shadow .2s;
    }

    .poisk input:focus {
        outline: none;
        border-color: var(--cvet-osnovnoy);
        box-shadow: 0 0 0 3px rgba(181, 65, 31, 0.12);
    }

    /* ===== Кнопки ===== */
    button, .knopka {
        display: inline-flex;
        align-items: center;
        justify-content: center;
        min-height: 44px;
        padding: 12px var(--s3);
        font-size: 15px;
        font-family: inherit;
        font-weight: 600;
        color: #fff;
        background: var(--cvet-osnovnoy);
        border: none;
        border-radius: var(--radius);
        cursor: pointer;
        text-decoration: none;
        transition: background .2s, transform .1s;
    }

    button:hover, .knopka:hover { background: var(--cvet-temnyi); }
    button:active { transform: translateY(1px); }

    /* ===== Фильтры ===== */
    .filtry {
        background: var(--cvet-karta);
        border: 1px solid var(--cvet-ramka);
        border-radius: var(--radius);
        padding: var(--s3);
        margin-bottom: var(--s4);
    }

    .filtry fieldset { border: none; display: flex; flex-direction: column; gap: va

```



```

.filtriy legend { font-weight: 800; margin-bottom: var(--s1); }

.filtriy label { font-size: 14px; color: var(--cvet-tihii); display: block; margin-bottom: var(--s1); }

.filtriy input, .filtriy select {
    width: 100%;
    padding: 10px 12px;
    font-size: 16px;
    font-family: inherit;
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
    background: var(--cvet-karta);
}

/* ===== Каталог ===== */
.katalog {
    display: grid;
    grid-template-columns: 1fr;
    gap: var(--s3);
}

.tovar {
    display: flex;
    flex-direction: column;
    background: var(--cvet-karta);
    border: 1px solid var(--cvet-ramka);
    border-radius: var(--radius);
    padding: var(--s3);
    box-shadow: var(--ten);
    transition: transform .2s, box-shadow .2s, border-color .2s;
}

.tovar:hover {
    transform: translateY(-2px);
    border-color: var(--cvet-osnovnoy);
    box-shadow: var(--ten-navel);
}

.tovar h3 { font-size: 17px; line-height: 1.35; margin-bottom: var(--s1); }

.artikul { font-size: 13px; color: var(--cvet-tihii); }

```

```

.cena {
    font-size: 24px;
    font-weight: 800;
    color: var(--cvet-osnovnoy);
    margin: var(--s2) 0 4px;
}

.cena span { font-size: 14px; font-weight: 400; color: var(--cvet-tihii); }

.nalichie { font-size: 13px; color: var(--cvet-uspeh); font-weight: 600; }
.nalichie.nety { color: var(--cvet-tihii); font-weight: 400; }

.tovar form { margin-top: auto; padding-top: var(--s3); }
.tovar button { width: 100%; }

/* ===== Пусто ===== */
.pusto { text-align: center; padding: 48px var(--s3); color: var(--cvet-tihii); }
.pusto h3 { color: var(--cvet-text); margin-bottom: var(--s1); }
.pusto p { margin-bottom: var(--s3); }

/* ===== Подвал ===== */
footer {
    margin-top: 48px;
    padding-top: var(--s3);
    border-top: 1px solid var(--cvet-ramka);
    color: var(--cvet-tihii);
    font-size: 14px;
}

footer a { color: var(--cvet-osnovnoy); }

/* ===== Планшет ===== */
@media (min-width: 640px) {
    .katalog { grid-template-columns: repeat(2, 1fr); }
    .filtry fieldset { flex-direction: row; flex-wrap: wrap; align-items: flex-
    .filtry input, .filtry select { width: auto; }
    .tovar button { width: auto; }
}

/* ===== Компьютер ===== */
@media (min-width: 960px) {
    header {
        flex-direction: row;
        justify-content: space-between;
    }

```

```

        align-items: center;
        flex-wrap: wrap;
    }

    .logo { font-size: 30px; }

    .katalog { grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); }
}

```

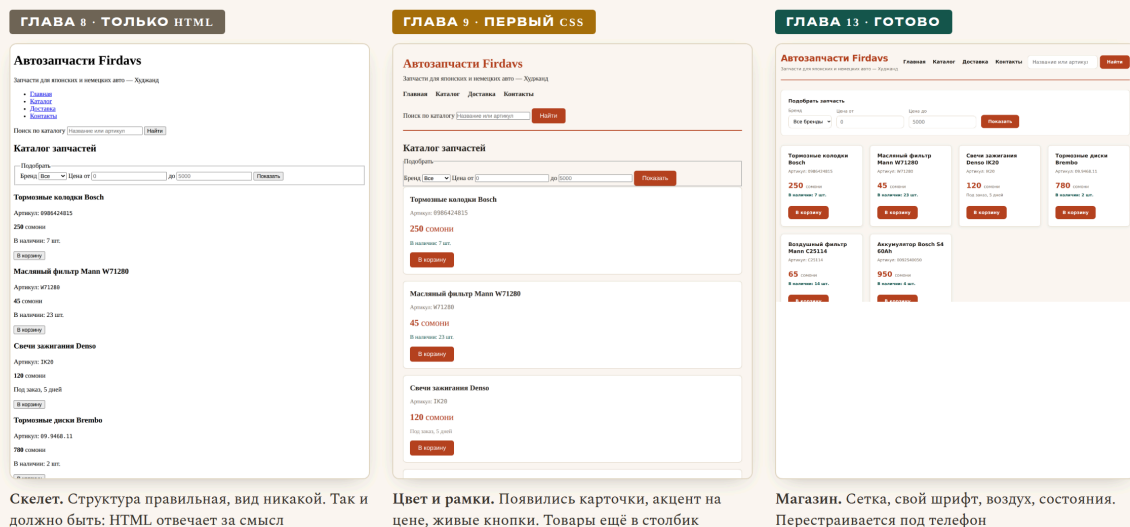
НА ЭКРАНЕ

На экране

Весь путь одной картинкой — от голого HTML до готового магазина:

Один и тот же HTML — три состояния

Разметка почти не менялась. Всё превращение сделали стили



Скелет. Структура правильная, вид никакой. Так и должно быть: HTML отвечает за смысл

Цвет и рамки. Появились карточки, акцент на цене, живые кнопки. Товары ещё в столбик

Магазин. Сетка, свой шрифт, воздух, состояния. Перестраивается под телефон

Разделение обязанностей в действии: чтобы полностью сменить облик сайта, не нужно трогать разметку. Меняются только стили — а на боевом проекте это ещё и три строчки переменных вместо всего файла.

Ещё раз, потому что это важно: **HTML почти не менялся**. Мы добавили пару классов и обёрток. Всё превращение сделал CSS.

ГРАБЛИ

Грабли

Переменные не в `:root`. Объявите их внутри какого-нибудь класса — и они будут видны только там. Всегда `:root`.

Забывать два дефиса. `--cvet: red;` — правильно. `-cvet` или `cvet` — не работает.

Подключить десять начертаний шрифта. Каждое — отдельный файл. Сайт станет заметно медленнее. Хватит трёх.

Шрифт без кириллицы. Проверяйте на Google Fonts фильтром Cyrillic.

Слишком много всего. Пять цветов, четыре шрифта, тени на всём, градиенты, анимации везде. Хороший интерфейс — сдержанный.

Забыть про пустое состояние. Первый же покупатель отфильтрует так, что ничего не найдётся, и увидит белый экран.

задачи

Задачи

Задача 13.1. Переведите свой `style.css` на переменные. Все цвета, отступы и скругления — в `:root`.

Задача 13.2. Поменяйте фирменный цвет магазина на другой, изменив одну строчку. Убедитесь, что перекрасилось всё.

Задача 13.3. Подключите шрифт с Google Fonts. Обязательно проверьте поддержку кириллицы.

Задача 13.4. Сделайте состояние «ничего не найдено» с кнопкой возврата. Чтобы увидеть, временно прокомментируйте товары.

Задача 13.5. Проверьте свои отступы. Все ли кратны 8? Найдите и почините случайные значения.

Задача 13.6. Проверьте контраст текста на webaim.org/resources/contrastchecker. Основной текст должен набрать не меньше 4.5:1.

Задача 13.7. Сделайте три цветовые схемы своего магазина, меняя только переменные. Сохраните как три варианта и выберите лучшую.

Задача 13.8. Покажите сайт трём людям, которые не программируют. Задайте каждому три вопроса: что это за сайт? как найти товар? как купить? Записывайте, где они запинаются.

Это называется **юзабилити-тест**, и он работает лучше любой теории. Три человека находят большинство проблем.

Ответы — в Приложении Г.

В БОЮ В бою

В `assets/css/custom.css` вы найдёте те же приёмы: переменные наверху файла, кратные отступы, состояния для пустого каталога.

Обратите внимание на одну боевую деталь: у карточек товаров задана **минимальная высота**. Причина простая — названия запчастей бывают очень разной длины («Колодки Bosch» и «Комплект сцепления Sachs 3000 951 081 в сборе»), и без минимальной высоты сетка выглядела рвано.

Такие вещи невозможно предусмотреть заранее. Они находятся, когда в каталог заливают настоящие данные. **Верстайте на реальных, а не на придуманных данных** — это сэкономит вам переделку.

ИТОГ Итог

- Переменные в `:root` через `--ima` и `var(--ima)`. Меняете один раз — перекрашивается всё.
- Имена переменных — **по роли**, а не по цвету.
- Шрифт с Google Fonts, обязательно с кириллицей. **Максимум два шрифта**, три начертания.
- Отступы **кратны 8**. Цветов **3-4**. Размеров шрифта **4-5**.
- **Воздуха больше**, чем кажется нужным.
- Контраст текста не ниже 4.5:1.
- Обязательно сделайте состояния: **пусто, загрузка, ошибка**. Каждое должно подсказывать, что делать.
- Верстайте на настоящих данных, а не на «Товар 1, Товар 2».

Часть III закончена. У вас есть красивый адаптивный магазин. Но он неподвижен: кнопки ничего не делают, поиск не ищет.

Со следующей главы начинается JavaScript — и страница оживёт.

JavaScript: переменные, условия, циклы

ЗАЧЕМ ЭТО**Зачем эта глава**

Ваш магазин красивый, но мёртвый. Кнопка «В корзину» ничего не делает. Поиск не ищет. Фильтры не фильтруют.

Начинаем это исправлять. И заодно — вот важное — **вы впервые встретитесь с настоящим программированием.**

HTML и CSS программированием не являются. Это языки описания: «здесь заголовок», «сделай красным». В них нет решений, нет повторений, нет вычислений.

JavaScript — другое дело. Здесь появляются **переменные, условия, циклы**. И вот эта тройка есть в **каждом** языке программирования на свете. Выучив её один раз на JavaScript, вы узнаете её же в PHP (глава 20), а потом в Python, Java — где угодно.

Так что эта глава важнее, чем кажется. Мы учим не JavaScript. Мы учим программирование, а JavaScript просто оказался первым.

РАЗБИРАЕМСЯ**Куда писать код**

Три способа, как и с CSS.

► Отдельный файл — правильный

```
<body>
  ... всё содержимое страницы ...

  <script src="script.js"></script>
</body>
```

⚠ Обратите внимание: `<script>` ставят в самом конце `<body>`, перед `</body>`, а не в `<head>`.

Причина простая: браузер читает страницу сверху вниз. Если скрипт стоит в начале, он выполнится, когда элементов на странице ещё нет — и не найдёт того, с чем должен работать. Классическая ошибка новичка, которую мы обойдём заранее.

► Прямо в странице — для проб

```
<script>
  console.log('Привет');
</script>
```

► В консоли браузера — для опытов

Нажмите F12 → Console, напишите что угодно и нажмите Enter. Код выполнится сразу.

Пользуйтесь этим постоянно, пока учитесь. Не нужно создавать файл, чтобы проверить одну строчку. Консоль — ваш черновик.

РАЗБИРАЕМСЯ

console.log — окно в программу

Первая команда, которую нужно выучить:

```
console.log('Привет!');
console.log(250);
console.log('Цена:', 250, 'сомони');
```

`console.log` выводит значение в консоль браузера (F12 → Console).

Зачем это нужно. Программа выполняется молча. Вы не видите, что происходит внутри: какое значение получилось, зашло ли выполнение в условие, сколько раз повторился цикл.

`console.log` делает невидимое видимым. Это основной инструмент отладки — не только новичка, но и профессионала. Не стесняйтесь ставить его везде.

Правило на будущее: не понимаете, что делает программа — поставьте `console.log` и посмотрите.

► Что это такое

Переменная — именованное место, где лежит значение.

```
let cena = 250;
```

Читается: «создай коробочку с именем `cena` и положи в неё 250».

Дальше можно пользоваться именем вместо значения:

```
console.log(cena);           // 250
console.log(cena * 2);       // 500
```

Зачем это нужно. Три причины:

1. Значение можно посчитать один раз и использовать много.
2. Код становится понятным. `itogo = cena * kolichestvo` читается, а `250 * 3` — нет.
3. Менять легко. Цена в одном месте, а не в двадцати.

► `let` и `const`

```
let kolichestvo = 3;           // значение будет меняться
const NDS = 0.18;             // значение постоянное
```

СЛОВО	ЧТО ОЗНАЧАЕТ	КОГДА
<code>const</code>	Постоянная. Изменить нельзя	По умолчанию — берите её
<code>let</code>	Переменная. Значение будет меняться	Когда действительно нужно менять
<code>var</code>	Устаревший способ	Не используйте. Встретите в старом коде

Правило, которое экономит нервы: всегда начинайте с `const`. Если попытаете изменить — программа честно сообщит об ошибке, и вы задумаетесь, точно ли надо. Меняйте на `let` только тогда, когда действительно нужно.

Почему это важно: чем меньше в программе меняющихся вещей, тем легче понять, что происходит. Это не педантизм, а способ уменьшить количество ошибок.

► Как называть переменные

<code>let cenaTovara = 250;</code>	✓ понятно
<code>let с = 250;</code>	✗ что за с?
<code>let x1 = 250;</code>	✗ и это тоже
<code>let цена = 250;</code>	✗ кириллица — технически можно, практически нельз

Правила:

- Только латиница, цифры, `_` и `$`. Начинать с цифры нельзя.
- Регистр важен: `cena` и `Cena` — разные переменные.
- Принято писать **camelCase**: слова слитно, каждое следующее с большой буквы:
`cenaTovara`, `kolichествоNaSklade`.
- Имя должно объяснять, что внутри.

Хорошее имя переменной — половина понятного кода. Программист читает код в десять раз чаще, чем пишет, и пишет он его в первую очередь для людей.

РАЗБИРАЕМСЯ

Типы данных

В коробочку можно положить разное. От того, что внутри, зависит, что можно делать.

```
const nazvanie = 'Тормозные колодки'; // строка
const cena = 250; // число
const estVnalichii = true; // да/нет
const skidka = null; // «ничего», осознанно
let brend; // «не задано»
```

ТИП	КАК НАЗЫВАЕТСЯ	ПРИМЕРЫ
Строка	string	'текст', "текст", `текст`
Число	number	250, -5, 3.14
Логический	boolean	true, false
Пусто (осознанно)	null	null
Не задано	undefined	переменная без значения

Проверить тип можно так:

```
console.log(typeof cena);      // "number"
console.log(typeof nazvanie);  // "string"
```

► Строки: кавычки и склейка

```
const a = 'Bosch';
const b = "Bosch";
const c = `Bosch`;
```

Все три работают. Одинарные и двойные равноправны — главное, быть последовательным.

А вот **обратные кавычки** (```, клавиша слева от единицы) умеют кое-что особенное:

```
const nazvanie = 'Колодки Bosch';
const cena = 250;

// Старый способ — склейка через плюс
const text1 = 'Товар: ' + nazvanie + ', цена ' + cena + ' сомони';

// Современный способ — шаблонная строка
const text2 = `Товар: ${nazvanie}, цена ${cena} сомони`;
```

Обе строки дадут одно и то же. Но вторая читается в разы лучше — видно готовую фразу, а не мешанину из плюсов и кавычек.

Синтаксис: обратные кавычки вокруг всей строки, `${...}` вокруг подставляемого значения.

Пользуйтесь шаблонными строками всегда. Это одна из тех вещей, которые сразу отличают современный код от кода десятилетней давности.

► Числа: осторожно с дробями

```
console.log(0.1 + 0.2); // 0.30000000000000004
```

Нет, это не ошибка в браузере. Так работают дробные числа **во всех языках программирования** — они хранятся в двоичном виде, и некоторые десятичные дроби в него точно не переводятся.

Практический вывод для магазина: не храните деньги в дробях. Храните в самых мелких единицах — в дирамах (1 сомони = 100 дирамов), — то есть целыми числами. 250 сомони = 25000. Делите на 100 только при показе.

Это правило работает во всех финансовых системах мира. Запомните сейчас, чтобы не переписывать потом.

РАЗБИРАЕМСЯ

Условия: if, else

Вы уже видели `if` в первой главе. Теперь по-настоящему.

```
const ostatok = 7;

if (ostatok > 0) {
  console.log('В наличии');
} else {
  console.log('Нет в наличии');
}
```

Дословно: **если** остаток больше нуля — напиши «в наличии», **иначе** — «нет».

► Операторы сравнения

```
a === b    // строго равно
a !== b    // строго не равно
a > b      // больше
a < b      // меньше
a ≥ b      // больше или равно
a ≤ b      // меньше или равно
```

⚠ Три знака, не два. В JavaScript есть и `=` (два знака), но он ведёт себя странно:

```
console.log(5 = '5');    // true  – сравнил, приведя типы
console.log(5 === '5');  // false – число и строка, разные вещи
```

`=` пытается «догадаться» и приводит типы друг к другу. Это источник загадочных ошибок.

Правило без исключений: всегда `===` и `!==`. Забудьте про `=`.

► Несколько условий

```
const cena = 250;
const ostatok = 7;

if (ostatok > 0 && cena < 300) {
    console.log('Есть и недорого');
}

if (ostatok === 0 || cena > 1000) {
    console.log('Или нет, или дорого');
}

if (!estSkidka) {
    console.log('Скидки нет');
}
```

ЗНАК	НАЗВАНИЕ	СМЫСЛ
&&	И	Оба условия верны
	ИЛИ	Хотя бы одно верно
!	НЕ	Переворачивает: было <code>true</code> — стало <code>false</code>

Читается вслух почти по-русски: «если остаток больше нуля и цена меньше 300».

► Несколько веток

```
if (ostatok === 0) {
  console.log('Нет в наличии');
} else if (ostatok < 3) {
  console.log('Осталось мало');
} else {
  console.log('В наличии');
}
```

Проверяется **сверху вниз**, выполняется **первое подошедшее** — остальные пропускаются.

Порядок важен! Если поставить `ostatok < 3` перед `ostatok === 0`, то ноль попадёт в «осталось мало», потому что ноль тоже меньше трёх.

► Короткая запись

```
const status = ostatok > 0 ? 'В наличии' : 'Под заказ';
```

Называется **тернарный оператор**. Читается: «условие ? если да : если нет».

Удобно для коротких случаев. Для сложной логики берите обычный `if` — вложенные тернарники нечитаемы.

РАЗБИРАЕМСЯ

Циклы: делать много раз

► Зачем

У вас 6 товаров. Или 600. Писать 600 раз «покажи товар» — нереально. Цикл делает это за вас.

► **for** — когда знаете, сколько раз

```
for (let i = 1; i ≤ 5; i++) {  
  console.log(`Товар номер ${i}`);  
}
```

Напечатает пять строк. Разберём заголовок цикла — там три части через `;`:

ЧАСТЬ	ЧТО ДЕЛАЕТ	КОГДА ВЫПОЛНЯЕТСЯ
<code>let i = 1</code>	Создать счётчик и задать начало	Один раз, в начале
<code>i ≤ 5</code>	Условие продолжения	Перед каждым повтором
<code>i++</code>	Увеличить счётчик на 1	После каждого повтора

`i++` — сокращение для `i = i + 1`. Есть и `i--` (уменьшить), и `i += 5` (увеличить на 5).

Имя `i` — от слова *index*. Это одно из немногих коротких имён, которое считается нормальным: все программисты мира понимают, что `i` — счётчик цикла.

► **while** — пока условие верно

```
let ostatok = 5;  
  
while (ostatok > 0) {  
  console.log(`Продали, осталось ${ostatok - 1}`);  
  ostatok = ostatok - 1;  
}
```

Повторяется, пока условие истинно.

⚠ Главная опасность `while` — забыть менять условие внутри:

```
let i = 0;  
while (i < 5) {  
  console.log('Бесконечно!');  
  // забыли i++  
}
```

Это бесконечный цикл. Браузер зависнет намертво. Спасение — закрыть вкладку.

Совершали такую ошибку все. Просто знайте, где искать.

► Прервать и пропустить

```
for (let i = 1; i ≤ 10; i++) {  
  if (i === 5) break;      // выйти из цикла совсем  
  if (i % 2 === 0) continue; // пропустить этот повтор  
  console.log(i);  
}
```

`break` — «хватит, выходим». `continue` — «этот пропускаем, идём к следующему».

`%` — остаток от деления. `i % 2 === 0` означает «i делится на 2 без остатка», то есть чётное.

Приём, которым проверяют чётность во всех языках.

```
// Данные о товаре
const nazvanie = 'Тормозные колодки Bosch';
const cenaZaShtuku = 250;
const ostatokNaSklade = 7;
const skidkaProcent = 10;

// Сколько хочет покупатель
const hochetKupit = 3;

console.log(`Товар: ${nazvanie}`);
console.log(`Цена: ${cenaZaShtuku} сомони`);

// Проверяем наличие
if (ostatokNaSklade === 0) {
    console.log('Нет в наличии, можно заказать');
} else if (hochetKupit > ostatokNaSklade) {
    console.log(`Столько нет. Доступно только ${ostatokNaSklade} шт.`);
} else {
    // Считаем стоимость
    let itogo = cenaZaShtuku * hochetKupit;
    console.log(`${hochetKupit} шт. = ${itogo} сомони`);

    // Скидка от трёх штук
    if (hochetKupit ≥ 3) {
        const skidka = itogo * skidkaProcent / 100;
        itogo = itogo - skidka;
        console.log(`Скидка ${skidkaProcent}%: минус ${skidka} сомони`);
    }

    console.log(`К оплате: ${itogo} сомони`);
}

// Покажем весь остаток по одной штуке
console.log('--- Остаток на складе ---');
for (let i = 1; i ≤ ostatokNaSklade; i++) {
    console.log(`Комплект №${i}`);
}
```


Откройте F12 → Console, вставьте код и нажмите Enter:

F12 → Console: вывод нашей программы

Elements	Console	Sources	Network
> Товар: Тормозные колодки Bosch			
> Цена: 250 сомони			
> 3 шт. = 750 сомони			
> Скидка 10%: минус 75 сомони			
> К оплате: 675 сомони			
> --- Остаток на складе ---			
> Комплект №1			
> Комплект №2			
> Комплект №3			
> Комплект №4			
> Комплект №5			
> Комплект №6			
> Комплект №7			

Программа проверила наличие, посчитала сумму, применила скидку от трёх штук и перечислила остаток.

Программа посчитала стоимость, применила скидку и перечислила остаток. Это первая настоящая программа в книге: она принимает решения и повторяет действия.

ЗАПИСЬ	КАК ЧИТАЕТСЯ	ЧТО ДЕЛАЕТ
<code>const</code>	«конст»	Постоянная, менять нельзя
<code>let</code>	«лет»	Переменная, можно менять
<code>=</code>	присвоить	Положить справа в левое
<code>===</code>	строго равно	Спросить, одинаковы ли
<code>!==</code>	не равно	Спросить, различны ли
<code>&&</code>	и	Оба условия
<code> </code>	или	Хотя бы одно
<code>!</code>	не	Перевернуть
<code>\${...}</code>	подстановка	Вставить значение в шаблонную строку
<code>`</code>	обратная кавычка	Границы шаблонной строки
<code>i++</code>	инкремент	Увеличить на 1
<code>%</code>	остаток	Остаток от деления
<code>console.log()</code>		Показать значение в консоли
<code>typeof</code>		Узнать тип значения
<code>break</code>		Выйти из цикла
<code>continue</code>		Пропустить один повтор
<code>//</code>	комментарий	Заметка для человека

ГРАБЛИ

Грабли

Путать `=` и `===`. `=` кладёт, `===` спрашивает. Самая частая ошибка на свете. Если условие ведёт себя странно — проверьте это первым делом.

Использовать `==`. Всегда `===`.

Бесконечный `while`. Забыли менять условие — браузер завис.

`<script>` в `<head>`. Код выполнится раньше, чем появятся элементы, и ничего не найдёт. Ставьте перед `</body>`.

Деньги в дробях. `0.1 + 0.2` $\neq$ `0.3`. Считайте в дирамах, целыми числами.

Кириллица в именах переменных. Формально разрешена, практически — источник проблем.

Забыть, что регистр важен. `цена` и `Цена` — две разные переменные, и браузер не подскажет.

ЗАДАЧИ **Задачи**

Все задачи решаются прямо в консоли F12. Не переписывайте — **набирайте**.

Задача 14.1. Создайте переменные для своего товара: название, цена, остаток, бренд. Выведите их одной шаблонной строкой.

Задача 14.2. Что выведет каждая строка? Ответьте, не запуская, потом проверьте:

```
console.log(5 + 3);
console.log('5' + 3);
console.log(5 === '5');
console.log(5 = '5');
console.log(10 % 3);
console.log(typeof '250');
console.log(!true);
```

Вторая строка — важный сюрприз. Разберитесь, почему так.

Задача 14.3. Напишите проверку: если товаров на складе больше 10 — «много», от 1 до 10 — «есть», ноль — «нет». Проверьте на трёх значениях.

Задача 14.4. Посчитайте стоимость заказа со скидкой: от 3 штук — 5%, от 10 штук — 10%, от 50 — 15%. Внимание к порядку условий!

Задача 14.5. Циклом выведите числа от 1 до 20, но только чётные. Двумя способами: через `%` и через `i += 2`.

Задача 14.6. Циклом посчитайте сумму чисел от 1 до 100. Проверка: должно получиться 5050.

Задача 14.7. Напишите «таблицу умножения» для числа 7, от 1 до 10, в виде `7 × 3 = 21`.

Задача 14.8. Найдите ошибку и объясните, в чём она:

```
let cena = 250;
if (cena = 300) {
  console.log('Цена 300');
}
```

Задача 14.9. Переведите на JavaScript условие: «если товар в наличии и цена меньше 500, или если у покупателя есть промокод — показать кнопку купить».

Задача 14.10. Что произойдёт? Не запускайте, просто подумайте:

```
let i = 10;
while (i > 0) {
  console.log(i);
}
```

Ответы — в Приложении Г.

Загляните в `assets/js/` этого репозитория. Найдёте те же самые вещи: `const`, `if`, циклы, шаблонные строки.

Особенно посмотрите на код счётчика корзины — там ровно та логика, которую вы только что изучили: проверить остаток, посчитать сумму, показать результат.

И одна деталь из жизни: в боевом коде вы **не найдёте** `console.log` — их убирают перед выкладкой на сайт. Пока учитесь, ставьте сколько угодно. Просто помните, что в готовом сайте их быть не должно: они замедляют работу и показывают посторонним внутреннюю кухню.

ИТОГ

Итог

- Переменные, условия и циклы есть **в каждом языке**. Учим не JavaScript — учим программирование.
- `<script>` ставится **перед** `</body>`.
- `console.log()` — главный инструмент отладки. Не понимаете — выведите.
- **`const` по умолчанию**, `let` — только когда нужно менять. `var` не используйте.
- Имена — латиницей, camelCase, по смыслу.
- Шаблонные строки: ``Цена ${цена} сомони``. Пользуйтесь всегда.
- Деньги — **целыми числами** в мелких единицах. Дроби врут.
- `=` кладёт, `===` спрашивает. **Только** `===`, никогда `=`.
- `&&` — и, `||` — или, `!` — не.
- `for` — когда знаете количество, `while` — пока условие верно.
- Забыть менять условие в `while` = зависший браузер.

Дальше — функции и массивы: научимся хранить список товаров и не повторять код.

Функции, массивы, объекты

ЗАЧЕМ ЭТО

Зачем эта глава

В прошлой главе у нас был один товар, разложенный по четырём переменным. А в магазине их шестьсот.

Заводить `nazvanie1`, `cena1`, `nazvanie2`, `cena2` ... — очевидная нелепость. Нужен способ хранить список и способ описывать один товар целиком.

Плюс третья вещь: мы уже дважды писали похожий код для расчёта скидки. Нужен способ написать один раз и вызывать по имени.

Три инструмента этой главы:

- функции — свой код по имени;
- массивы — списки;
- объекты — сложные данные одной штукой.

Это последняя «теоретическая» глава JavaScript. Со следующей начнём менять настоящую страницу.

РАЗБИРАЕМСЯ

Функции

► Что это и зачем

Функция — это кусок кода с именем. Написали один раз — вызываете сколько угодно.

```
function pozdorovatsya() {
    console.log('Добро пожаловать в наш магазин!');
}

pozdorovatsya();
pozdorovatsya();
```

ЧАСТЬ

ЧТО ОЗНАЧАЕТ

<code>function</code>	«Дальше идёт функция»
<code>pozdorovatsya</code>	Имя, по которому будем звать
<code>()</code>	Место для входных данных
<code>{ }</code>	Тело: что делать
<code>pozdorovatsya();</code>	Вызов — вот сейчас выполни

⚠ Важно: объявление функции её **не выполняет**. Функция ждёт, пока её позовут. Забыть скобки при вызове — частая ошибка: `pozdorovatsya` без `()` не запускает её, а лишь упоминает.

► Параметры — данные на вход

```
function pokazatTovar(nazvanie, cena) {
    console.log(`${nazvanie} — ${cena} сомони`);
}

pokazatTovar('Колодки Bosch', 250);
pokazatTovar('Фильтр Mann', 45);
```

Одна функция, разные данные, разный результат.

- **Параметры** — имена в объявлении (`nazvanie`, `cena`);
- **Аргументы** — реальные значения при вызове (`'Колодки Bosch'`, `250`).

Названия разные, но путать их не страшно — многие используют вперемешку.

► `return` — результат наружу

Функция может не только печатать, но и **возвращать значение**:

```
function schitatItogo(cena, kolichestvo) {  
    return cena * kolichestvo;  
}
```

```
const itogo = schitatItogo(250, 3);  
console.log(itogo); // 750  
console.log(schitatItogo(45, 2)); // 90
```

`return` делает две вещи сразу:

1. отдаёт значение тому, кто вызвал;
2. немедленно завершает функцию — код после него не выполнится.

Второе свойство удобно для проверок:

```
function schitatSoSkidkoy(cena, kolichestvo) {  
    if (kolichestvo ≤ 0) {  
        return 0; // дальше не идём  
    }  
  
    let itogo = cena * kolichestvo;  
  
    if (kolichestvo ≥ 3) {  
        itogo = itogo * 0.9; // скидка 10%  
    }  
  
    return itogo;  
}  
  
console.log(schitatSoSkidkoy(250, 3)); // 675  
console.log(schitatSoSkidkoy(250, 0)); // 0
```

Такой приём называется **ранний выход**: сначала отсекаем плохие случаи, потом спокойно пишем основную логику. Код получается плоским, без глубоко вложенных `if`.

► Значения по умолчанию

```
function schitatDostavku(summa, gorod = 'Худжанд') {  
    if (gorod === 'Худжанд') return 0;  
    return 50;  
}  
  
console.log(schitatDostavku(500));           // 0 – город не указали  
console.log(schitatDostavku(500, 'Душанбе')); // 50
```

Если аргумент не передали, возьмётся значение по умолчанию.

► Стрелочные функции

Современная короткая запись — та самая, что мелькала в первой главе:

```
// Обычная  
function udvoit(x) {  
    return x * 2;  
}  
  
// Стрелочная  
const udvoit = (x) => {  
    return x * 2;  
};  
  
// Совсем короткая: одно выражение — return не нужен  
const udvoit = x => x * 2;
```

Все три делают одно и то же.

Стрелочные функции особенно удобны, когда функцию передают внутрь другой функции — а это в JavaScript происходит постоянно (увидим ниже и в главе 17).

► Зачем вообще функции: три причины

1. Не повторяться. Логика скидки написана один раз. Меняются правила — правите в одном месте.

2. Имя объясняет смысл. `schitatSoSkidkoy(250, 3)` понятно без комментариев, а `250 * 3 * 0.9` — нет.

3. Можно проверять по отдельности. Функцию легко вызвать с разными данными и убедиться, что она считает верно.

Правило хорошего тона: одна функция — одно дело. Если в названии просится «и» (`proveritIsohranit`), это две функции.

РАЗБИРАЕМСЯ

Массивы — списки

► Что это

Массив — упорядоченный список значений.

```
const brendy = ['Bosch', 'Mann', 'Denso', 'Brembo'];  
const ceny = [250, 45, 120, 780];
```

Квадратные скобки, значения через запятую.

► Обращение по номеру

```
console.log(brendy[0]); // 'Bosch'  
console.log(brendy[1]); // 'Mann'  
console.log(brendy[3]); // 'Brembo'  
console.log(brendy[9]); // undefined — такого нет
```

⚠ Нумерация с нуля. Первый элемент — `[0]`, не `[1]`.

Это сбивает с толку всех новичков. Так устроено почти во всех языках программирования, и придётся привыкнуть. Полезный образ: номер — это не «который по счёту», а «на сколько шагов отступить от начала». От начала до первого элемента шагов ноль.

► Длина и последний элемент

```
console.log(brendy.length); // 4  
console.log(brendy[brendy.length - 1]); // 'Brembo' — последний
```

Раз нумерация с нуля, последний элемент всегда `length - 1`. Запомните эту формулу, она понадобится много раз.

► Изменение массива

```
const korzina = [];  
  
korzina.push('Колодки');      // добавить в конец  
korzina.push('Фильтр');  
console.log(korzina);        // ['Колодки', 'Фильтр']  
  
korzina.pop();               // убрать последний  
console.log(korzina);        // ['Колодки']  
  
korzina.unshift('Свечи');    // добавить в начало  
korzina.shift();             // убрать первый  
  
console.log(korzina.includes('Колодки')); // true — есть ли такой  
console.log(korzina.indexOf('Колодки'));  // 0 — на каком месте
```

⚠ Заметьте: массив объявлен через `const`, но мы его меняем — и это работает.

`const` запрещает заменить саму коробку, но не запрещает менять её содержимое. Нельзя `korzina = ['другое']`, можно `korzina.push('другое')`.

Тонкость, которая поначалу удивляет, но логика в ней есть: `const` про переменную, а не про данные.

► Перебор массива

Способ первый — обычный цикл `for`:

```
for (let i = 0; i < brendy.length; i++) {  
  console.log(brendy[i]);  
}
```

Способ второй, удобнее — `for...of`:

```
for (const brend of brendy) {  
  console.log(brend);  
}
```

Читается почти по-английски: «для каждого бренда из списка брендов». Не нужно возиться со счётчиком и не ошибёшься с границами.

Берите `for...of`, когда номер элемента не нужен.

РАЗБИРАЕМСЯ

Объекты — один товар целиком

► Проблема

Хранить товар в отдельных переменных неудобно:

```
const nazvanie = 'Колодки Bosch';
const cena = 250;
const ostatok = 7;
```

А если товаров шесть? Восемнадцать переменных, и никак не видно, что связано с чем.

► Решение

```
const tovar = {
  nazvanie: 'Тормозные колодки Bosch',
  artikl: '0986424815',
  cena: 250,
  ostatok: 7,
  brend: 'Bosch',
  vNalichii: true
};
```

Объект — набор пар «имя: значение», собранных в фигурные скобки.

ЧАСТЬ	НАЗВАНИЕ
<code>nazvanie</code>	Ключ (или свойство)
<code>'Тормозные колодки Bosch'</code>	Значение
<code>:</code>	Разделитель
<code>,</code>	Между парами

► Чтение и запись

```
console.log(tovar.nazvanie);    // через точку — основной способ
console.log(tovar['cena']);      // через скобки — когда имя в переменной

tovar.cena = 230;               // изменить
tovar.foto = 'kolodki.jpg';     // добавить новое свойство

console.log(tovar.vesa);        // undefined — такого свойства нет
```

Обращение к несуществующему свойству — не ошибка, а `undefined`. Это спасает от падений, но иногда прячет опечатки: `товар.cena` (с русской «а») молча вернёт `undefined`.

► Массив объектов — вот оно, главное

Соединяем два инструмента и получаем каталог:

```
const tovary = [
  { id: 1, nazvanie: 'Тормозные колодки Bosch', cena: 250, ostatok: 7, brend: 'Bosch' },
  { id: 2, nazvanie: 'Масляный фильтр Mann',   cena: 45,  ostatok: 23, brend: 'Mann' },
  { id: 3, nazvanie: 'Свечи зажигания Denso',   cena: 120, ostatok: 0,  brend: 'Denso' },
  { id: 4, nazvanie: 'Тормозные диски Brembo',  cena: 780, ostatok: 2,  brend: 'Brembo' },
  { id: 5, nazvanie: 'Воздушный фильтр Mann',   cena: 65,  ostatok: 14, brend: 'Mann' },
  { id: 6, nazvanie: 'Аккумулятор Bosch S4',    cena: 950, ostatok: 4,  brend: 'Bosch' },
];

for (const t of tovary) {
  console.log(`${t.nazvanie} — ${t.cena} сомони`);
}
```

Вот в таком виде данные приходят из базы данных. Ровно в таком. Когда в главе 34 мы начнём доставать товары из MySQL, PHP отдаст нам именно массив объектов.

Так что вы сейчас учите не «структуру данных JavaScript», а способ, которым устроены данные почти во всех веб-приложениях.

Для массивов есть готовые методы. Они короче цикла и понятнее.

► `filter` — отобрать подходящие

```
const vNaLichii = tovary.filter(t => t.ostatok > 0);
const deshevye = tovary.filter(t => t.cena < 200);
const boschi = tovary.filter(t => t.brend === 'Bosch');
```

`filter` проходит по всем элементам и оставляет те, для которых условие истинно. Возвращает новый массив, исходный не трогает.

Это фильтры каталога, ровно те, что вы сверстали в главе 8.

► `map` — превратить каждый

```
const nazvaniya = tovary.map(t => t.nazvanie);
// ['Тормозные колодки Bosch', 'Масляный фильтр Mann', ...]

const soSkidkoy = tovary.map(t => ({
  ...t,
  cena: Math.round(t.cena * 0.9)
}));
```

`map` берёт каждый элемент и превращает во что-то другое. На выходе массив той же длины.

`...t` — `spread`, «возьми всё из `t`». Здесь: скопируй товар целиком, но цену замени. Пригодится часто.

`Math.round()` — округление. Есть ещё `Math.floor()` (вниз), `Math.ceil()` (вверх), `Math.max()`, `Math.min()`.

► `find` — найти один

```
const tovar = tovary.find(t => t.id === 3);
console.log(tovar.nazvanie); // 'Свечи зажигания Denso'
```

`find` возвращает первый подошедший элемент (не массив!) или `undefined`.

Это поиск товара по номеру — то, что делает страница товара.

► **reduce** — свести к одному значению

```
const summa = tovary.reduce((itog, t) => itog + t.cena, 0);  
console.log(summa); // 2210
```

Чуть сложнее остальных: **reduce** идёт по массиву и накапливает результат. **itog** — накопленное, **t** — текущий элемент, **0** — с чего начать.

Это сумма корзины.

► **Цепочки**

Методы можно соединять — и вот тут становится красиво:

```
const itogo = tovary  
  .filter(t => t.ostatok > 0) // только те, что есть  
  .filter(t => t.cena < 500) // и недорогие  
  .map(t => t.cena) // взять цены  
  .reduce((a, b) => a + b, 0); // сложить  
  
console.log(itogo);
```

Читается сверху вниз как список действий. Тот же результат циклом занял бы строк десять и читался бы хуже.

```
const tovary = [
  { id: 1, nazvanie: 'Тормозные колодки Bosch', cena: 250, ostatok: 7, brend: 'Bosch' },
  { id: 2, nazvanie: 'Масляный фильтр Mann', cena: 45, ostatok: 23, brend: 'Mann' },
  { id: 3, nazvanie: 'Свечи зажигания Denso', cena: 120, ostatok: 0, brend: 'Denso' },
  { id: 4, nazvanie: 'Тормозные диски Brembo', cena: 780, ostatok: 2, brend: 'Brembo' },
  { id: 5, nazvanie: 'Воздушный фильтр Mann', cena: 65, ostatok: 14, brend: 'Mann' },
  { id: 6, nazvanie: 'Аккумулятор Bosch S4', cena: 950, ostatok: 4, brend: 'Bosch' }
];

// Показать товар одной строкой
function opisat(t) {
  const status = t.ostatok > 0 ? `в наличии ${t.ostatok} шт.` : 'под заказ';
  return `${t.nazvanie} — ${t.cena} сомони (${status})`;
}

console.log('=== ВСЬ КАТАЛОГ ===');
for (const t of tovary) {
  console.log(opisat(t));
}

console.log('=== ТОЛЬКО В НАЛИЧИИ ===');
tovary.filter(t => t.ostatok > 0).forEach(t => console.log(opisat(t)));

console.log('=== ДЕШЕВЛЕ 200 СОМОНИ ===');
tovary.filter(t => t.cena < 200).forEach(t => console.log(opisat(t)));

// Считаем итоги
const vsegoPozicii = tovary.length;
const naSklade = tovary.reduce((s, t) => s + t.ostatok, 0);
const stoimostSkлада = tovary.reduce((s, t) => s + t.cena * t.ostatok, 0);
const samyiDorogoy = tovary.reduce((max, t) => t.cena > max.cena ? t : max);

console.log('=== ИТОГИ ===');
console.log(`Позиций в каталоге: ${vsegoPozicii}`);
console.log(`Всего штук на складе: ${naSklade}`);
console.log(`Склад стоит: ${stoimostSkлада} сомони`);
console.log(`Самый дорогой: ${samyiDorogoy.nazvanie}`);

// Корзина
```



```
const korzina = [];  
function dobavitVKorzinu(id, kolichество) {  
  const tovar = tovary.find(t => t.id === id);  
  if (!tovar) return 'Товар не найден';  
  if (tovar.ostatok < kolichество) return `Не хватает: есть только ${tovar.os  
  korzina.push({ tovar, kolichество });  
  return `Добавлено: ${tovar.nazvanie} × ${kolichество}`;  
}  
  
console.log('=== КОРЗИНА ===');  
console.log(dobavitVKorzinu(1, 2));  
console.log(dobavitVKorzinu(3, 1));  
console.log(dobavitVKorzinu(99, 1));  
  
const summaKorziny = korzina.reduce((s, p) => s + p.tovar.cena * p.kolichество,  
console.log(`Итого в корзине: ${summaKorziny} сомони`);
```

НА ЭКРАНЕ

На экране

Каталог, фильтры и корзина — вывод программы

```
Elements Console Sources Network
> === ВЕСЬ КАТАЛОГ ===
> Тормозные колодки Bosch — 250 сомони (в наличии 7 шт.)
> Масляный фильтр Mann — 45 сомони (в наличии 23 шт.)
> Свечи зажигания Denso — 120 сомони (под заказ)
> Тормозные диски Brembo — 780 сомони (в наличии 2 шт.)
> Воздушный фильтр Mann — 65 сомони (в наличии 14 шт.)
> Аккумулятор Bosch S4 — 950 сомони (в наличии 4 шт.)
> === ТОЛЬКО В НАЛИЧИИ ===
> Тормозные колодки Bosch — 250 сомони (в наличии 7 шт.)
> Масляный фильтр Mann — 45 сомони (в наличии 23 шт.)
> Тормозные диски Brembo — 780 сомони (в наличии 2 шт.)
> Воздушный фильтр Mann — 65 сомони (в наличии 14 шт.)
> Аккумулятор Bosch S4 — 950 сомони (в наличии 4 шт.)
> === ДЕШЕВЛЕ 200 СОМОНИ ===
> Масляный фильтр Mann — 45 сомони (в наличии 23 шт.)
> Свечи зажигания Denso — 120 сомони (под заказ)
> Воздушный фильтр Mann — 65 сомони (в наличии 14 шт.)
> === ИТОГИ ===
> Позиций в каталоге: 6
> Всего штук на складе: 50
> Склад стоит: 9055 сомони
> Самый дорогой: Аккумулятор Bosch S4
> === КОРЗИНА ===
> Добавлено: Тормозные колодки Bosch × 2
> Не хватает: есть только 0
> Товар не найден
> Итого в корзине: 500 сомони
```

Товар №99 не найден, свечей нет в наличии — функция вернула осмысленный ответ вместо падения.

Заметьте, что произошло в строке про товар 99 и про свечи: функция **проверила** и вернула осмысленный ответ вместо того, чтобы упасть. Это и есть аккуратный код.

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>function имя() { }</code>	Объявить функцию
<code>имя()</code>	Вызвать её
<code>return</code>	Вернуть значение и выйти из функции
<code>(x) ⇒ x * 2</code>	Стрелочная функция
<code>[1, 2, 3]</code>	Массив
<code>массив[0]</code>	Первый элемент. Нумерация с нуля
<code>.length</code>	Сколько элементов
<code>.push()</code> / <code>.pop()</code>	Добавить / убрать с конца
<code>for (const x of массив)</code>	Перебрать все элементы
<code>{ kluch: znachenie }</code>	Объект
<code>obekt.kluch</code>	Прочитать свойство
<code>.filter(...)</code>	Отобрать подходящие → новый массив
<code>.map(...)</code>	Превратить каждый → новый массив
<code>.find(...)</code>	Найти первый подходящий → один элемент
<code>.reduce(...)</code>	Свести к одному значению
<code>.forEach(...)</code>	Сделать что-то с каждым, ничего не возвращая
<code>... obekt</code>	Spread: «взять всё отсюда»
<code>Math.round()</code>	Округлить

ГРАБЛИ

Грабли

Забыть скобки при вызове. `pozdorovatsya` — упоминание, `pozdorovatsya()` — вызов.

Считать с единицы. Первый элемент — `[0]`. Последний — `[length - 1]`.

Ждать, что `filter` изменит массив. Не изменит. Он возвращает новый: `const novyi = staryi.filter(...)`.

Путать `find` и `filter`. `find` даёт один элемент, `filter` — массив. Обратиться к `.nazvanie` у результата `filter` — получить `undefined`.

Забывать `return`. Функция без `return` возвращает `undefined`. Если результат «не приходит» — проверьте это.

Опечатка в имени свойства. `tovar.cena` с русской «а» вернёт `undefined` без всякой ошибки. Ищется мучительно.

Считать, что `const` защищает содержимое массива. Не защищает. Он лишь запрещает заменить сам массив.

ЗАДАЧИ

Задачи

Задача 15.1. Напишите функцию `sSkidkoy(cena, procent)`, которая возвращает цену со скидкой. Проверьте на трёх значениях.

Задача 15.2. Что вернёт каждая строка?

```
const a = [10, 20, 30, 40];
console.log(a[0]);
console.log(a[4]);
console.log(a.length);
console.log(a[a.length - 1]);
```

Задача 15.3. Возьмите массив товаров из главы и выведите:

1. только товары бренда Mann;
2. только те, что дороже 100 сомони;
3. только названия, без остальных данных;
4. общее количество штук на складе.

Задача 15.4. Напишите функцию `naitiPoArtikulu(artikul)`, которая ищет товар и возвращает его или строку «не найден».

Задача 15.5. Напишите функцию `dostavka(summa)`: до 300 сомони — доставка 50, от 300 до 1000 — 25, свыше 1000 — бесплатно.

Задача 15.6. Отсортируйте товары по цене. Подсказка:

```
const po_cene = [...tovary].sort((a, b) => a.cena - b.cena);
```

Зачем здесь `[...tovary]`? Попробуйте без него и посмотрите, что стало с исходным массивом. Вывод важный.

Задача 15.7. Сгруппируйте товары по брендам: сколько позиций у каждого бренда.

Задача 15.8. Найдите ошибку:

```
const tovar = tovary.filter(t => t.id === 3);  
console.log(tovar.nazvanie);
```

Задача 15.9. Напишите функцию `korzinaItogo(korzina)`, которая принимает массив покупок и возвращает объект `{ pozicii, shtuk, summa }`.

Задача 15.10. Перепишите цикл в цепочку методов:

```
let summa = 0;  
for (const t of tovary) {  
  if (t.ostatok > 0 && t.cena < 500) {  
    summa = summa + t.cena;  
  }  
}
```

Ответы — в Приложении Г.

Массив объектов — то, как выглядят данные **на всём пути** внутри магазина.

Смотрите цепочку: в базе лежат строки таблицы → PHP забирает их и получает массив массивов → отдаёт в браузер как JSON → JavaScript получает массив объектов и рисует карточки.

Форма одна и та же на каждом этапе. Поэтому то, что вы выучили в этой главе, пригодится и в PHP (глава 22), и в SQL (глава 27), и в работе с чужими API (глава 49).

Заглянуть можно в `api/` — там PHP отдаёт данные именно в таком виде.

ИТОГ

Итог

- **Функция** — код с именем. Объявили один раз, вызываете сколько нужно.
- `return` возвращает значение **и завершает** функцию. Отсекайте плохие случаи в начале.
- Одна функция — одно дело.
- **Массив** `[...]` — список. Нумерация **с нуля**, последний — `length - 1`.
- `for...of` — лучший способ перебрать, когда номер не нужен.
- **Объект** `{...}` — набор «ключ: значение». Один товар целиком.
- **Массив объектов** — то, как приходят данные из любой базы.
- `filter` отбирает, `map` превращает, `find` находит один, `reduce` сводит к числу.
- Эти методы **не меняют** исходный массив, а возвращают новый.
- `const` защищает переменную, а не содержимое массива.

Дальше — самое интересное: научимся менять настоящую страницу из кода.

DOM: меняем страницу из кода

ЗАЧЕМ ЭТО

Зачем эта глава

Две главы мы писали в консоль. Полезно для учёбы, бесполезно для покупателя — он консоль не открывает.

Пора соединить JavaScript со страницей. В этой главе научимся:

- **находить** элементы на странице;
- **менять** их текст, стили и классы;
- **создавать** новые элементы из данных.

Последнее — самое важное. Помните массив товаров из прошлой главы? Сейчас мы превратим его в настоящие карточки на странице. Это ровно то, чем занимается любой современный сайт.

РАЗБИРАЕМСЯ

Что такое DOM

Когда браузер получает HTML, он не хранит его как текст. Он строит **дерево объектов** — по одному объекту на каждый тег.

Это дерево и называется **DOM** — *Document Object Model*, «объектная модель документа».

```
document
├── html
│   ├── head
│   │   ├── meta
│   │   └── title
│   └── body
│       ├── header
│       │   └── h1
│       └── main
│           ├── article
│           │   ├── h3
│           │   └── p
│           └── article
```

Ключевая мысль: **JavaScript** меняет не **HTML-файл**, а это **дерево**. Файл на диске остаётся прежним. Меняется то, что браузер держит в памяти — и сразу же перерисовывает на экране.

Отсюда важное следствие: **все изменения пропадают при обновлении страницы**. Браузер перечитает файл и построит дерево заново.

Чтобы изменения сохранялись, их нужно записать на сервер — этим займёмся в части про PHP.

`document` — точка входа, «вся страница». С него начинается любая работа с DOM.

РАЗБИРАЕМСЯ Находим элементы

► Два метода, которых достаточно

```
const zagolovok = document.querySelector('h1');
const vseTovary = document.querySelectorAll('.tovar');
```

МЕТОД

ЧТО ВОЗВРАЩАЕТ

`querySelector`

Первый подошедший элемент (или `null`)

`querySelectorAll`

Все подошедшие, списком

Отличная новость: **внутри** пишутся обычные CSS-селекторы — те самые, что вы учили в главе 9.

```
document.querySelector('#poisk');           // по id
document.querySelector('.cena');           // по классу
document.querySelector('.tovar h3');       // вложенность
document.querySelector('input[type="search"]'); // по атрибуту
document.querySelectorAll('.tovar');       // все карточки
```

Ничего нового учить не надо. Знаете CSS — умеете искать в DOM.

► Если не нашлось

```
const кнопка = document.querySelector('.кнопка-kotoroy-net');
console.log(кнопка); // null
```

`querySelector` вернёт `null`, а обращение к свойству `null` уронит программу:

```
кнопка.textContent = 'Купить';
// TypeError: Cannot read properties of null
```

Это самая частая ошибка новичка в DOM. Причины почти всегда две:

1. Опечатка в селекторе (забыли точку перед классом);
2. Скрипт выполнен **раньше**, чем появился элемент — тот самый случай, ради которого `<script>` ставят перед `</body>`.

Защититься просто:

```
const кнопка = document.querySelector('.кнопка');
if (кнопка) {
  кнопка.textContent = 'Купить';
}
```

► Перебрать найденные

```
const tovary = document.querySelectorAll('.tovar');

console.log(tovary.length);    // сколько нашлось

tovary.forEach(t => {
  console.log(t.textContent);
});
```

`querySelectorAll` возвращает не совсем массив, а похожую на него коллекцию. `forEach` и `for...of` работают, а вот `map` и `filter` — нет. Если нужны они, превратите в настоящий массив: `[...tovary]`.

РАЗБИРАЕМСЯ

Меняем содержимое

► `textContent` — текст

```
const zagolovok = document.querySelector('h1');

console.log(zagolovok.textContent);    // прочитать
zagolovok.textContent = 'Новое название'; // записать
```

► `innerHTML` — с разметкой

```
const blok = document.querySelector('.cena');
blok.innerHTML = '<strong>250</strong> сомони';
```

Разница принципиальная:

	TEXTCONTENT	INNERHTML
Теги внутри	Покажет как текст	Выполнит как разметку
Скорость	Быстрее	Медленнее
Безопасность	Безопасно	Опасно для чужих данных

► Почему `innerHTML` опасен

Представьте, что вы показываете отзыв покупателя:

```
otzyv.innerHTML = tekstOtPolzovatelya;
```

А покупатель написал в отзыве:

```
<script>ukrastCookie()</script>
```

Браузер честно выполнит этот код — на странице, у всех остальных посетителей. Это называется **XSS-атака**, и она входит в тройку самых распространённых способов взлома сайтов.

Правило: для чужих данных всегда `textContent`. `innerHTML` — только для разметки, которую вы написали сами.

Подробно разберём в главе 36. Но привычку вырабатывайте сейчас.

РАЗБИРАЕМСЯ

Меняем стили

► Через `classList` — правильный способ

```
const karta = document.querySelector('.tovar');

karta.classList.add('vydelena');           // добавить класс
karta.classList.remove('vydelena');        // убрать
karta.classList.toggle('vydelena');        // есть – убрать, нет – добавить
console.log(karta.classList.contains('vydelena')); // проверить
```

А сам вид описан в CSS:

```
.tovar.vydelena {
  border-color: #b5411f;
  box-shadow: 0 4px 16px rgba(181, 65, 31, 0.2);
}
```

Почему это правильно: оформление остаётся в CSS, JavaScript только переключает состояние. Разделение обязанностей из первой главы соблюдено.

► Через `style` — когда значение вычисляется

```
element.style.display = 'none';
element.style.width = progress + '%';
```

Годится, когда значение известно только во время работы — например, ширина полосы загрузки.

⚠ Обратите внимание на две особенности:

```
element.style.backgroundColor = 'red';    // не background-color!
element.style.width = '300px';           // единицы обязательны!
```

В JavaScript свойства пишутся **camelCase** вместо дефисов, а числовые значения требуют единиц измерения. `width = 300` без `px` просто не сработает.

Правило выбора: есть заранее известный набор состояний — `classList`. Значение вычисляется — `style`.

РАЗБИРАЕМСЯ

Работа с атрибутами

```
const kartinka = document.querySelector('img');

kartinka.src = 'novoe-foto.jpg';
kartinka.alt = 'Новое описание';

const ssylka = document.querySelector('a');
ssylka.href = '/catalog';

const pole = document.querySelector('input');
console.log(pole.value);           // что ввёл человек
pole.value = 'Bosch';             // подставить значение
```

⚠ Для полей ввода нужен `value`, а не `textContent`. Частая ошибка.

► Свои данные в атрибутах

```
<button data-tovar-id="42" data-cena="250">В корзину</button>
```

```
const кнопка = document.querySelector('button');
console.log(кнопка.dataset.tovarId); // "42"
console.log(кнопка.dataset.cena);    // "250"
```

Атрибуты, начинающиеся с `data-`, — официальный способ хранить свои данные прямо в разметке. Читаются через `dataset`, причём `data-tovar-id` превращается в `dataset.tovarId` (дефисы уходят, camelCase появляется).

Незаменимо, когда нужно понять, какой именно товар кликнули.

⚠ Всё, что приходит из `dataset`, — строка. `"250" + 1` даст `"2501"`, а не `251`.
Превращайте в число: `Number(кнопка.dataset.cena)`.

РАЗБИРАЕМСЯ

Создаём элементы

Вот главное умение этой главы.

► Способ 1: собрать по частям

```
const карта = document.createElement('article');
карта.className = 'товар';

const заголовок = document.createElement('h3');
заголовок.textContent = 'Тормозные колодки Bosch';

карта.append(заголовок);
document.querySelector('.katalog').append(карта);
```

МЕТОД	ЧТО ДЕЛАЕТ
<code>createElement('div')</code>	Создать элемент (пока нигде не показан)
<code>append(что)</code>	Вложить внутрь, в конец
<code>prepend(что)</code>	Вложить в начало
<code>remove()</code>	Удалить элемент

Способ надёжный и безопасный, но многословный.

► Способ 2: через `innerHTML`

```
const katalog = document.querySelector('.katalog');

katalog.innerHTML = `
  <article class="tovar">
    <h3>Тормозные колодки Bosch</h3>
    <p class="цена">250 сомони</p>
  </article>
`;
```

Короче и нагляднее. Годится, когда данные свои, а не пришли от пользователя.

► Способ 3: рисуем весь каталог из массива

А теперь соединим всё, что знаем:

```

const tovary = [
  { id: 1, nazvanie: 'Тормозные колодки Bosch', cena: 250, ostatok: 7 },
  { id: 2, nazvanie: 'Масляный фильтр Mann',      cena: 45, ostatok: 23 },
  { id: 3, nazvanie: 'Свечи зажигания Denso',      cena: 120, ostatok: 0 }
];

function narisovatKatalog(spisok) {
  const katalog = document.querySelector('.katalog');

  if (spisok.length === 0) {
    katalog.innerHTML = '<p class="pusto">Ничего не нашлось</p>';
    return;
  }

  katalog.innerHTML = spisok.map(t => `
    <article class="tovar">
      <h3>${t.nazvanie}</h3>
      <p class="cena">${t.cena} <span>сомони</span></p>
      <p class="nalichie ${t.ostatok > 0 ? '' : 'nety'}">
        ${t.ostatok > 0 ? `В наличии: ${t.ostatok} шт.` : 'Под заказ'}
      </p>
      <button data-id="${t.id}">В корзину</button>
    </article>
  `).join('');
}

narisovatKatalog(tovary);

```

Разберём, что здесь происходит:

1. `map` превращает каждый товар в кусок HTML;
2. `join('')` склеивает все куски в одну строку;
3. `innerHTML` вставляет всё разом.

Это и есть современный подход к сайтам. Разметка не пишется руками — она строится из данных. Три товара или три тысячи — код тот же.

Именно так работают React, Vue и все остальные модные библиотеки. Внутри у них, конечно, сложнее и умнее, но идея ровно эта.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Каталог из данных</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <header>
    <div class="logo">Автозапчасти Firdavs</div>
  </header>

  <main>
    <p class="schetchik"></p>
    <div class="katalog"></div>
  </main>
</div>

<script src="script.js"></script>
</body>
</html>
```

Обратите внимание: каталог в HTML пустой. Ни одной карточки. Всё нарисует JavaScript.


```

const tovary = [
  { id: 1, nazvanie: 'Тормозные колодки Bosch', cena: 250, ostatok: 7, brend: 'Bosch' },
  { id: 2, nazvanie: 'Масляный фильтр Mann', cena: 45, ostatok: 23, brend: 'Mann' },
  { id: 3, nazvanie: 'Свечи зажигания Denso', cena: 120, ostatok: 0, brend: 'Denso' },
  { id: 4, nazvanie: 'Тормозные диски Brembo', cena: 780, ostatok: 2, brend: 'Brembo' },
  { id: 5, nazvanie: 'Воздушный фильтр Mann', cena: 65, ostatok: 14, brend: 'Mann' },
  { id: 6, nazvanie: 'Аккумулятор Bosch S4', cena: 950, ostatok: 4, brend: 'Bosch' }
];

function kartochka(t) {
  const estOstatok = t.ostatok > 0;
  return `
    <article class="tovar">
      <h3>${t.nazvanie}</h3>
      <p class="artikul">${t.brend}</p>
      <p class="cena">${t.cena} <span>сомони</span></p>
      <p class="nalichie ${estOstatok ? '' : 'nety'}">${
        estOstatok ? `В наличии: ${t.ostatok} шт.` : 'Под заказ, 5 дней'
      }</p>
      <button data-id="${t.id}" ${estOstatok ? '' : 'disabled'}>В корзину</button>
    </article>
  `;
}

function narisovat(spisok) {
  const katalog = document.querySelector('.katalog');
  const schetchik = document.querySelector('.schetchik');

  schetchik.textContent = `Найдено товаров: ${spisok.length}`;

  if (spisok.length === 0) {
    katalog.innerHTML = '<p class="pusto">Ничего не нашлось</p>';
    return;
  }

  katalog.innerHTML = spisok.map(kartochka).join('');
}

narisovat(tovary);

```

Обратите внимание на `disabled` у кнопки: если товара нет, кнопка автоматически

становится неактивной. Логика вида задаётся **данными**, а не руками.

НА ЭКРАНЕ

На экране

Автозапчасти Firdavs

Найдено товаров: 6

Тормозные колодки Bosch

Bosch

250 сомони

В наличии: 7 шт.

В корзину

Масляный фильтр Mann

Mann

45 сомони

В наличии: 23 шт.

В корзину

Свечи зажигания Denso

Denso

120 сомони

Под заказ, 5 дней

В корзину

Тормозные диски Brembo

Brembo

780 сомони

В наличии: 2 шт.

В корзину

Воздушный фильтр Mann

Mann

65 сомони

В наличии: 14 шт.

В корзину

Аккумулятор Bosch S4

Bosch

950 сомони

В наличии: 4 шт.

В корзину

Откройте F12 → Elements. Вы увидите **все шесть карточек** в дереве — хотя в исходном файле их нет.

Теперь нажмите правую кнопку → «Посмотреть код страницы». Там пусто.

Разница между этими двумя окнами и есть DOM. «Код страницы» — то, что прислал сервер. «Elements» — то, что сейчас в памяти браузера.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>document</code>	Вся страница, точка входа
<code>querySelector('.класс')</code>	Найти первый по CSS-селектору
<code>querySelectorAll('.класс')</code>	Найти все
<code>.textContent</code>	Текст элемента. Безопасно
<code>.innerHTML</code>	Содержимое с разметкой. Опасно для чужих данных
<code>.value</code>	Значение поля ввода
<code>.classList.add/remove/toggle</code>	Управление классами
<code>.style.backgroundColor</code>	Стиль напрямую. camelCase, единицы обязательны
<code>.dataset.tovarId</code>	Прочитать <code>data-tovar-id</code> . Всегда строка
<code>createElement('div')</code>	Создать элемент
<code>.append(что)</code>	Вложить в конец
<code>.remove()</code>	Удалить
<code>.map(...).join('')</code>	Собрать HTML из массива
<code>null</code>	Ничего не нашлось

ГРАБЛИ

Грабли

`Cannot read properties of null`. Элемент не найден. Проверьте селектор (точка перед классом!) и положение `<script>` — он должен быть перед `</body>`.

Забывать точку или решётку. `querySelector('tovar')` ищет **тег** `<tovar>`. Нужно `.tovar`.

`innerHTML` с чужими данными. Дыра в безопасности. Для отзывов, имён, комментариев — только `textContent`.

`textContent` вместо `value` для полей. У `<input>` значение читается через `value`.

Забывать единицы в `style`. `el.style.width = 300` не работает. Нужно `'300px'`.

`dataset` возвращает строку. `Number(...)` перед арифметикой.

Ждать, что изменения сохранятся. Обновили страницу — всё вернулось. DOM живёт только в памяти.

Перерисовывать весь список в цикле по одному. `katalog.innerHTML += ...` внутри цикла заставляет браузер перестраивать дерево на каждой итерации. Собирайте строку целиком, вставляйте один раз.

ЗАДАЧИ

Задачи

Задача 16.1. Найдите на своей странице заголовок и поменяйте его текст из консоли.

Задача 16.2. Что вернёт каждая строка на странице из примера?

```
document.querySelector('.tovar');
document.querySelectorAll('.tovar').length;
document.querySelector('.tovar h3').textContent;
document.querySelector('.netu-takogo');
```

Задача 16.3. Постройте каталог из массива по примеру главы. Добавьте в массив седьмой товар и убедитесь, что он появился сам.

Задача 16.4. Напишите функцию, которая рисует только товары в наличии. Подсказка: `narisovat(tovary.filter(...))`.

Задача 16.5. Сделайте так, чтобы товары дороже 500 сомони получали класс `dorogoy`, а в CSS у него была золотая рамка.

Задача 16.6. Найдите ошибку:

```
const cena = document.querySelector('.cena');
const novaya = cena.textContent + 100;
console.log(novaya);
```

Задача 16.7. Выведите на страницу итоги: сколько позиций, сколько штук на складе, общая стоимость склада. Используйте `reduce` из главы 15.

Задача 16.8. Через `data-` атрибуты передайте в каждую кнопку `id` и цену товара. Выведите их в консоль для всех кнопок сразу.

Задача 16.9. Откройте любой крупный сайт, F12 → Elements. Найдите блок с товаром и удалите его клавишей Delete. Он исчез. Обновите страницу — вернулся. Объясните своими словами, почему.

Ответы — в Приложении Г.

В БОЮ В бою

В `assets/js/` этого репозитория найдите код, который рисует карточки товаров в результатах поиска. Увидите ту же связку `map` + `join` + `innerHTML`.

Одна боевая деталь, которую стоит заметить: перед вставкой все данные от поставщика прогоняются через экранирование — превращают `<` в `<`, чтобы чужой текст никогда не выполнялся как разметка.

Это защита от XSS, о которой мы говорили. На боевом сайте она обязательна: названия запчастей приходят из чужого API, и никто не гарантирует, что там не окажется чего-то опасного.

ИТОГ

- DOM — дерево объектов, которое браузер строит из HTML и держит в памяти.
- JavaScript меняет **дерево**, а не файл. Обновление страницы всё вернёт.
- `querySelector` — первый, `querySelectorAll` — все. **Внутри обычные CSS-селекторы.**
- Не нашлось — `null`. Обращение к `null` роняет программу.
- `textContent` — текст и **безопасность**. `innerHTML` — разметка и **риск XSS**.
- Для чужих данных только `textContent`.
- `classList` для состояний, `style` для вычисленных значений.
- `data-` атрибуты + `dataset` — хранение своих данных в разметке. Всегда строка.
- `map` + `join` + `innerHTML` — построить разметку из данных. Главный приём главы.
- Собирайте строку целиком и вставляйте один раз.

Дальше — события: научимся реагировать на клики и ввод.

События: реакция на действия

ЗАЧЕМ ЭТО

Зачем эта глава

Каталог рисуется из данных — уже хорошо. Но он всё ещё не реагирует на человека. Нажали кнопку — ничего. Набрали в поиске — ничего.

Событие — это «что-то произошло»: щелчок, ввод буквы, прокрутка, отправка формы. Научившись их ловить, вы сможете сделать интерфейс по-настоящему живым.

Эта глава — последний кусок теории JavaScript. Дальше только практика.

РАЗБИРАЕМСЯ

Как поймать событие

► Основной способ

```
const кнопка = document.querySelector('button');

кнопка.addEventListener('click', function () {
  console.log('Нажали!');
});
```

Читается: «кнопка, слушай событие `click`, и когда оно случится — выполни вот это».

ЧАСТЬ

ЧТО ОЗНАЧАЕТ

<code>addEventListener</code>	«Добавить слушателя события»
<code>'click'</code>	Какое событие ждём
<code>function () {...}</code>	Что делать, когда случится

Со стрелочной функцией короче:

```
кнопка.addEventListener('click', () => {  
  console.log('Нажали!');  
});
```

► Почему не `onClick`

Есть и такой способ:

```
кнопка.onClick = () => console.log('Нажали');
```

Работает, но у него ограничение: **обработчик может быть только один**. Назначите второй — первый пропадёт.

`addEventListener` позволяет повесить сколько угодно обработчиков на одно событие. Пользуйтесь им.

И совсем плохой способ, который встречается в старом коде:

```
<button onClick="добавит()">В корзину</button>
```

Логика в разметке — то же самое, что инлайн-стили из главы 9. Смешивает обязанности и мешает поддерживать.

СОБЫТИЕ	КОГДА ПРОИСХОДИТ	ГДЕ ПРИМЕНЯЕТСЯ
<code>click</code>	Щелчок мышью или тап пальцем	Кнопки, ссылки
<code>input</code>	Каждое изменение поля ввода	Живой поиск
<code>change</code>	Поле изменилось и потеряло фокус	Выпадающие списки, галочки
<code>submit</code>	Отправка формы	Проверка перед отправкой
<code>keydown</code>	Нажали клавишу	Горячие клавиши, Enter
<code>focus</code> / <code>blur</code>	Поле получило / потеряло фокус	Подсказки
<code>mouseenter</code> / <code>mouseleave</code>	Курсор вошёл / вышел	Всплывающие подсказки
<code>scroll</code>	Прокрутка страницы	Прилипающая шапка
<code>DOMContentLoaded</code>	Страница построена	Запуск скрипта

► `input` против `change` — важная разница

```
const pole = document.querySelector('input');

pole.addEventListener('input', () => {
  console.log('Изменилось:', pole.value); // на каждую букву
});

pole.addEventListener('change', () => {
  console.log('Закончили:', pole.value); // когда ушли из поля
});
```

Для живого поиска нужен `input` — он срабатывает на каждую букву. Для выпадающего списка удобнее `change`.

Обработчик получает на вход объект с подробностями:

```
кнопка.addEventListener('click', (event) => {  
  console.log(event.target);    // на чём именно кликнули  
  console.log(event.type);      // 'click'  
});
```

СВОЙСТВО

ЧТО ДАЁТ

<code>event.target</code>	Элемент, на котором произошло событие
<code>event.type</code>	Название события
<code>event.key</code>	Какая клавиша (для клавиатурных)
<code>event.preventDefault()</code>	Отменить обычное поведение браузера

Имя `event` можно сокращать до `e` — так делают почти все.

► `preventDefault` — ОТМЕНИТЬ ПОВЕДЕНИЕ ПО УМОЛЧАНИЮ

У некоторых элементов есть встроенное поведение: ссылка переходит, форма перезагружает страницу.

```
const форма = document.querySelector('form');  
  
форма.addEventListener('submit', (e) => {  
  e.preventDefault();    // страница НЕ перезагрузится  
  console.log('Обрабатываем сами');  
});
```

Зачем. Форма по умолчанию отправляется на сервер и перезагружает страницу. Иногда это правильно (глава 24). Но если мы хотим обработать данные прямо здесь, без перезагрузки, — нужен `preventDefault`.

► Проблема

Повесим обработчик на все кнопки каталога:

```
document.querySelectorAll('.tovar button').forEach(кнопка => {
  кнопка.addEventListener('click', () => {
    console.log('В корзину');
  });
});
```

Работает. Но есть две беды.

Беда первая. Кнопки, созданные **после** этого кода, обработчик не получат. А в прошлой главе мы рисуем каталог из данных — и перерисовываем его при каждом фильтре. Все обработчики теряются.

Беда вторая. Шестьсот товаров — шестьсот слушателей в памяти.

► Решение

Вешаем один обработчик на родителя и смотрим, куда именно кликнули:

```
document.querySelector('.katalog').addEventListener('click', (e) => {
  const кнопка = e.target.closest('button');
  if (!кнопка) return; // кликнули мимо кнопки

  const id = Number(кнопка.dataset.id);
  console.log('В корзину товар', id);
});
```

Это называется **делеги́рование событий**.

Разберём:

СТРОКА

ЧТО ДЕЛАЕТ

Слушатель на <code>.katalog</code>	Ловим клики по всему каталогу
<code>e.target</code>	Элемент, на который реально попал клик
<code>.closest('button')</code>	Подняться вверх до ближайшей кнопки
<code>if (!knopka) return</code>	Кликнули не по кнопке — выходим

Зачем `closest`. Внутри кнопки может быть иконка или текст. Клик попадёт в них, а не в саму кнопку. `closest` поднимается по дереву вверх и находит ближайшего подходящего предка (или сам элемент).

Что мы выиграли:

- один слушатель вместо шестисот;
- работает для элементов, которых ещё не существует;
- каталог можно перерисовывать сколько угодно — обработчик не потеряется.

Запомните этот приём. Он используется в любом реальном проекте.

РАЗБИРАЕМСЯ

Запускать скрипт вовремя

Если `<script>` стоит в `<head>`, элементов ещё нет:

```
document.addEventListener('DOMContentLoaded', () => {  
  // здесь вся страница уже построена  
  narisovat(tovary);  
});
```

Событие `DOMContentLoaded` — «дерево готово».

Но если `<script>` стоит перед `</body>>`, как мы и договорились в главе 14, это уже не нужно: к моменту выполнения скрипта страница построена.

Знать полезно: вы обязательно встретите эту обёртку в чужом коде.

Собираем всё: рисование из данных, фильтры, поиск, корзину.

```
const tovary = [
  { id: 1, nazvanie: 'Тормозные колодки Bosch', cena: 250, ostatok: 7, brend: 'Bosch' },
  { id: 2, nazvanie: 'Масляный фильтр Mann', cena: 45, ostatok: 23, brend: 'Mann' },
  { id: 3, nazvanie: 'Свечи зажигания Denso', cena: 120, ostatok: 0, brend: 'Denso' },
  { id: 4, nazvanie: 'Тормозные диски Brembo', cena: 780, ostatok: 2, brend: 'Brembo' },
  { id: 5, nazvanie: 'Воздушный фильтр Mann', cena: 65, ostatok: 14, brend: 'Mann' },
  { id: 6, nazvanie: 'Аккумулятор Bosch S4', cena: 950, ostatok: 4, brend: 'Bosch' }
];

// Корзина живёт здесь
const korzina = [];

// --- Отрисовка ---

function kartochka(t) {
  const est = t.ostatok > 0;
  return `
    <article class="tovar">
      <h3>${t.nazvanie}</h3>
      <p class="artikul">${t.brend}</p>
      <p class="cena">${t.cena} <span>сомони</span></p>
      <p class="nalichie ${est ? '' : 'nety'}">${
        est ? `В наличии: ${t.ostatok} шт.` : 'Под заказ'
      }</p>
      <button data-id="${t.id}" ${est ? '' : 'disabled'}>В корзину</button>
    </article>`;
}

function narisovat(spisok) {
  const katalog = document.querySelector('.katalog');
  document.querySelector('.schetchik').textContent = `Найдено: ${spisok.length}`;

  katalog.innerHTML = spisok.length
    ? spisok.map(kartochka).join('')
    : '<p class="pusto">Ничего не нашлось. Попробуйте другой запрос.</p>';
}
```

```
// --- Фильтрация ---
```

```
function primenitFiltry() {  
    const zapros = document.querySelector('#poisk').value.trim().toLowerCase();  
    const brend = document.querySelector('#brend').value;  
    const tolkoVNalichii = document.querySelector('#v-nalichii').checked;  
  
    let spisok = tovary;  
  
    if (zapros) {  
        spisok = spisok.filter(t => t.nazvanie.toLowerCase().includes(zapros));  
    }  
    if (brend) {  
        spisok = spisok.filter(t => t.brend === brend);  
    }  
    if (tolkoVNalichii) {  
        spisok = spisok.filter(t => t.ostatok > 0);  
    }  
  
    narisovat(spisok);  
}
```

```
// --- Корзина ---
```

```
function obnovitKorzinu() {  
    const shtuk = korzina.reduce((s, p) => s + p.kolichestvo, 0);  
    const summa = korzina.reduce((s, p) => s + p.tovar.cena * p.kolichestvo, 0)  
  
    document.querySelector('.korzina-shtuk').textContent = shtuk;  
    document.querySelector('.korzina-summa').textContent = summa;  
}  
  
function dobavitVKorzinu(id) {  
    const tovar = tovary.find(t => t.id === id);  
    if (!tovar) return;  
  
    const uzheEst = korzina.find(p => p.tovar.id === id);  
    if (uzheEst) {  
        uzheEst.kolichestvo = uzheEst.kolichestvo + 1;  
    } else {  
        korzina.push({ tovar, kolichestvo: 1 });  
    }  
}
```

```

    obnovitKorzinu();
}

// --- События ---

document.querySelector('#poisk').addEventListener('input', primenitFiltry);
document.querySelector('#brend').addEventListener('change', primenitFiltry);
document.querySelector('#v-nalichii').addEventListener('change', primenitFiltry);

// Делегирование: один слушатель на весь каталог
document.querySelector('.katalog').addEventListener('click', (e) => {
    const knopka = e.target.closest('button');
    if (!knopka || knopka.disabled) return;

    dobavitVKorzinu(Number(knopka.dataset.id));

    // Короткая обратная связь — человек должен видеть, что нажатие сработало
    knopka.textContent = 'Добавлено';
    setTimeout(() => { knopka.textContent = 'В корзину'; }, 900);
});

// Форма поиска не должна перезагружать страницу
document.querySelector('.filtry').addEventListener('submit', (e) => {
    e.preventDefault();
    primenitFiltry();
});

// Первый показ
narisovat(tovary);
obnovitKorzinu();

```

► Что здесь важного

setTimeout(функция, 900) — «выполни через 900 миллисекунд». Кнопка на секунду меняет надпись и возвращается. Мелочь, но без неё человек не понимает, сработало ли нажатие.

Обратная связь обязательна. Любое действие должно быть заметно: изменилась надпись, дёрнулся счётчик, мигнула карточка. Интерфейс, который молчит в ответ на нажатие, кажется сломанным.

`.trim()` убирает пробелы по краям, `.toLowerCase()` приводит к нижнему регистру.

Вместе они делают поиск нечувствительным к регистру и случайным пробелам — человек напишет «Bosch» и всё равно найдёт.

`.includes(zapros)` проверяет, содержится ли подстрока. Это и есть поиск по названию.

НА ЭКРАНЕ

На экране

Набрали в поиске «фильтр» — каталог мгновенно сузился, счётчик обновился:

Автозапчасти Firdavs

Живой поиск без перезагрузки страницы

Корзина: 0 шт. · 0 сомони

Поиск по названию

Бренд

фильтр



Все бренды



☐ Только в наличии

Найдено: 2

Масляный фильтр Mann

Mann

45 сомони

В наличии: 23 шт.

В корзину

Воздушный фильтр Mann

Mann

65 сомони

В наличии: 14 шт.

В корзину

Страница не перезагружалась. Ни одного запроса на сервер. Всё происходит в браузере, мгновенно.

Нажали «В корзину» — счётчик вырос, кнопка отчиталась:

Автозапчасти Firdavs

Живой поиск без перезагрузки страницы

Корзина: 3 шт. · 1280 сомони

Поиск по названию

Бренд

например, фильтр

Все бренды ▾

☐ Только в наличии

Найдено: 6

Тормозные колодки
Bosch

Bosch

250 сомони

В наличии: 7 шт.

Добавлено

Масляный фильтр
Mann

Mann

45 сомони

В наличии: 23 шт.

В корзину

Свечи зажигания
Denso

Denso

120 сомони

Под заказ

В корзину

Тормозные диски
Brembo

Brembo

780 сомони

В наличии: 2 шт.

Добавлено

Воздушный фильтр
Mann

Mann

65 сомони

Аккумулятор Bosch
S4

Bosch

950 сомони

Сайт с нуля

241

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>addEventListener('click', fn)</code>	Слушать событие
<code>'click'</code>	Щелчок или тап
<code>'input'</code>	Каждое изменение поля
<code>'change'</code>	Изменилось и потеряло фокус
<code>'submit'</code>	Отправка формы
<code>e.target</code>	Элемент, на котором произошло событие
<code>e.preventDefault()</code>	Отменить обычное поведение
<code>.closest('button')</code>	Найти ближайшего подходящего предка
делегирование	Один слушатель на родителе вместо многих на детях
<code>setTimeout(fn, 900)</code>	Выполнить через 900 мс
<code>.trim()</code>	Убрать пробелы по краям
<code>.toLowerCase()</code>	В нижний регистр
<code>.includes('текст')</code>	Содержит ли подстроку
<code>.checked</code>	Отмечена ли галочка
<code>DOMContentLoaded</code>	Дерево страницы готово

ГРАБЛИ

Грабли

Обработчики теряются после перерисовки. Классика. Лечится делегированием.

Забывать `preventDefault` у формы. Страница перезагрузится, и покажется, что «ничего не работает».

Вешать слушателя на элемент, которого ещё нет. `querySelector` вернёт `null`, и `addEventListener` уронит программу. Проверьте порядок и положение `<script>`.

Вызвать функцию вместо того, чтобы передать её.

```
кнопка.addEventListener('click', primenitFiltry()); // ❌ вызовется сразу
кнопка.addEventListener('click', primenitFiltry); // ✅ передали саму функцию
```

Разница в скобках. Ошибка коварная: код «работает один раз при загрузке», а на клик не реагирует.

Не давать обратной связи. Нажали — ничего не изменилось — нажали ещё пять раз.

Обработчик на `input` без ограничения. Если внутри тяжёлая операция или запрос на сервер, он полетит на каждую букву. Лечится приёмом `debounce` — разберём в главе 35.

ЗАДАЧИ

Задачи

Задача 17.1. Соберите пример из главы. Проверьте: поиск, фильтр по бренду, галочка наличия, кнопки корзины.

Задача 17.2. Добавьте сортировку: выпадающий список «по цене возрастанию / убыванию / по названию». Событие `change`.

Задача 17.3. Сделайте кнопку «Сбросить фильтры»: очищает поиск, ставит бренд «все», снимает галочку и перерисовывает каталог.

Задача 17.4. Добавьте фильтр по цене: два поля «от» и «до», событие `input`.

Задача 17.5. Сделайте так, чтобы по нажатию Enter в поле поиска страница не перезагружалась. Подсказка: событие `keydown` и `e.key === 'Enter'`, либо `submit` формы.

Задача 17.6. Найдите ошибку:

```
document.querySelectorAll('.tovar button').forEach(k => {
  k.addEventListener('click', () => dobavitVKorzinu(k.dataset.id));
});
narisovat(tovary);
```

Две проблемы. Найдите обе.

Задача 17.7. Сделайте раскрывающийся блок «Подробнее» у каждой карточки. Через делегирование и `classList.toggle`.

Задача 17.8. Добавьте счётчику корзины анимацию при изменении: пусть на мгновение увеличивается. Подсказка: добавить класс, снять через `setTimeout`.

Задача 17.9. Сделайте кнопку «Очистить корзину» с подтверждением через `confirm('Точно?')`.

Задача 17.10. Откройте любой интернет-магазин, нажмите «в корзину» и следите за F12 → Network. Ушёл ли запрос на сервер? Что это говорит о том, где хранится корзина?

Ответы — в Приложении Г.

В БОЮ

В бою

В боевом магазине делегирование используется везде — иначе после каждого фильтра пришлось бы заново развешивать обработчики на сотни карточек.

Но есть принципиальная разница с нашим примером, и её важно понять.

У нас корзина живёт в переменной `korzina`. Обновите страницу — она пуста.

В настоящем магазине корзина хранится на сервере. Нажатие «в корзину» отправляет запрос в `api/`, PHP записывает товар в сессию или в базу, и корзина переживает и обновление страницы, и закрытие браузера, и переход с телефона на компьютер.

Почему нельзя оставить корзину в JavaScript: покупатель может подделать что угодно в браузере, включая цены. Помните правило из первой главы? **Деньги считает только сервер.**

Как это сделать — в главе 38.

ИТОГ

- `addEventListener('событие', функция)` — основной способ. Не `onclick`, не атрибуты в разметке.
- `click` — нажатие, `input` — каждая буква, `change` — после потери фокуса, `submit` — отправка формы.
- `e.target` — где произошло, `e.preventDefault()` — отменить обычное поведение.
- **Делегирование:** один слушатель на родителе + `closest`. Работает для будущих элементов и экономит память. **Главный приём главы.**
- Передавайте функцию без скобок: `addEventListener('click', fn)`.
- Каждое действие должно давать **обратную связь**.
- `trim()` + `toLowerCase()` + `includes()` — базовый поиск по тексту.
- Корзина в JavaScript исчезает при обновлении. Настоящая живёт на сервере.

Следующая глава — практическая. Доведём поиск и корзину до готового вида.

Практика: живой поиск и корзина

ЗАЧЕМ ЭТО

Зачем эта глава

Соберём всё, чему научились, в законченную вещь: каталог с поиском, фильтрами, сортировкой и корзиной, которая не пропадает при обновлении страницы.

И разберём четыре приёма, которые отличают учебный код от рабочего:

- `localStorage` — как запомнить данные в браузере;
- `debounce` — как не дёргать поиск на каждую букву;
- подсветка совпадений — и почему здесь легко открыть дыру в безопасности;
- честные состояния — что показывать, когда ничего не нашлось.

РАЗБИРАЕМСЯ

`localStorage` — память браузера

► Проблема

Наша корзина живёт в переменной. Обновили страницу — пусто. Покупатель, который случайно нажал F5, теряет всё.

► Решение

У браузера есть небольшое хранилище, которое переживает и обновление, и закрытие:

```
localStorage.setItem('korzina', '...');    // записать
const dannye = localStorage.getItem('korzina'); // прочитать
localStorage.removeItem('korzina');          // удалить
localStorage.clear();                        // очистить всё
```

⚠ Хранить можно только строки. Массив или объект придётся превратить в текст:

```
// Сохраняем
localStorage.setItem('korzina', JSON.stringify(korzina));

// Читаем
const sohranennoe = localStorage.getItem('korzina');
const korzina = sohranennoe ? JSON.parse(sohranennoe) : [];
```

ФУНКЦИЯ

ЧТО ДЕЛАЕТ

`JSON.stringify(объект)`

Превратить объект или массив **в строку**

`JSON.parse(строка)`

Превратить строку **обратно** в объект

JSON — формат обмена данными, похожий на объект JavaScript. С ним вы будете встречаться постоянно: в нём общаются браузер и сервер (глава 49), в нём приходят данные от чужих API.

► Что важно знать про `localStorage`

СВОЙСТВО

ПОДРОБНОСТЬ

Объём	Около 5 МБ. Для корзины с запасом
Срок жизни	Бессрочно, пока не удалят
Область	Только этот сайт в этом браузере
Другое устройство	Не увидит. Это память браузера, не аккаунта
Доступ из JavaScript	Полный — и в этом главная опасность

⚠ **Никогда не храните в `localStorage`:**

- пароли;
- номера карт;
- токены доступа;
- цены, по которым будете считать заказ.

Любой скрипт на странице может это прочитать и изменить. `localStorage` годится для удобства (что лежало в корзине, какие фильтры выбраны, тёмная тема), но не для того, за что отвечают деньги.

В корзине храните только номера товаров и количество. Цену возьмите с сервера при оформлении. Иначе покупатель откроет консоль, поменяет цену на 1 и оформит заказ.

РАЗБИРАЕМСЯ

Debounce — не дёргать на каждую букву

Наш поиск работает по событию `input` — на каждую букву. Для шести товаров в памяти это нормально.

А если товаров десять тысяч и поиск идёт **на сервере**? Человек набирает «колодки» — уходит семь запросов, шесть из которых уже никому не нужны.

Debounce — приём «подожди, пока перестанут печатать»:

```
function debounce(funkciya, zaderzhka = 300) {
  let tajmer;
  return function (...argumenty) {
    clearTimeout(tajmer);
    tajmer = setTimeout(() => funkciya(...argumenty), zaderzhka);
  };
}

// Применяем
const poiskSZaderzhkoy = debounce(primenitFiltry, 300);
document.querySelector('#poisk').addEventListener('input', poiskSZaderzhkoy);
```

Как работает: каждое нажатие **отменяет** предыдущий таймер и ставит новый. Пока человек печатает, функция не вызывается. Замолчал на 300 мс — сработала один раз.

Разберём непривычное:

ЗАПИСЬ

ЧТО ОЗНАЧАЕТ

<code>clearTimeout(tajmer)</code>	Отменить запланированное
<code>...argumenty</code>	«Сколько бы аргументов ни было — заberi все»
Функция возвращает функцию	Обычное дело в JavaScript. Внутренняя помнит <code>tajmer</code>

Функция, возвращающая функцию, поначалу выглядит странно. Просто примите как рабочий приём — со временем станет привычным.

300 миллисекунд — хорошее значение по умолчанию: человек не замечает задержки, а запросов становится в разы меньше.

РАЗБИРАЕМСЯ

Подсветка совпадений — и ловушка в ней

Хочется подсветить найденное:

```
function podsvetit(tekst, zapros) {  
  if (!zapros) return tekst;  
  const regexp = new RegExp(`(${zapros})`, 'gi');  
  return tekst.replace(regexp, '<mark>$1</mark>');  
}
```

`<mark>` — семантический тег «выделенное», браузер красит его жёлтым.

⚠ А теперь ловушка. Мы вставляем результат через `innerHTML`, а внутри — текст, который набрал пользователь. Если он введёт `<img src=x onerror=alert(1)>`, код выполнится.

На нашей странице это безобидно — вредит только себе. Но если поисковый запрос попадает в ссылку, которой можно поделиться, — это уже настоящая XSS-уязвимость.

Лечится экранированием — превращением опасных символов в безопасные:

```
function ekranirovat(tekst) {
    return String(tekst)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

function podsvetit(tekst, zapros) {
    const bezopasnyiTekst = ekranirovat(tekst);
    if (!zapros) return bezopasnyiTekst;

    const bezopasnyiZapros = ekranirovat(zapros).replace(/[\.\*\+\?\^\$\{\}\(\)\[\]\\\]/g,
    const regexp = new RegExp(`(${bezopasnyiZapros})`, 'gi');
    return bezopasnyiTekst.replace(regexp, '<mark>$1</mark>');
}
```

Теперь `<` превращается в `<` и показывается как текст, а не выполняется.

Второй `replace` с непонятными символами экранирует спецсимволы регулярных выражений — без него запрос вроде `(` уронит поиск. Регулярные выражения — отдельная большая тема; пока достаточно знать, что эта строчка защищает от падения.

Запомните правило. Данные от пользователя опасны всегда. Даже свои собственные — сегодня поле заполняете вы, завтра оно придет из чужого API. Экранируйте.

```
// ===== Данные =====
```

```
const tovary = [  
  { id: 1, nazvanie: 'Тормозные колодки Bosch', artikl: '0986424815', cena:  
  { id: 2, nazvanie: 'Масляный фильтр Mann', artikl: 'W71280', cena:  
  { id: 3, nazvanie: 'Свечи зажигания Denso', artikl: 'IK20', cena:  
  { id: 4, nazvanie: 'Тормозные диски Brembo', artikl: '09.9468.11', cena:  
  { id: 5, nazvanie: 'Воздушный фильтр Mann', artikl: 'C25114', cena:  
  { id: 6, nazvanie: 'Аккумулятор Bosch S4', artikl: '0092S40050', cena:  
  { id: 7, nazvanie: 'Салонный фильтр Mann', artikl: 'CU2545', cena:  
  { id: 8, nazvanie: 'Ремень ГРМ Bosch', artikl: '1987949095', cena:  
];
```

```
// ===== Корзина: только id и количество =====
```

```
// Цену не храним осознанно: её должен считать сервер, иначе покупатель
```

```
// поменяет число в браузере и оформит заказ по своей цене.
```

```
let korzina = zagruziKorzinu();
```

```
function zagruziKorzinu() {  
  try {  
    const dannye = localStorage.getItem('korzina');  
    return dannye ? JSON.parse(dannye) : [];  
  } catch (e) {  
    // Данные могли испортиться — начинаем с пустой, а не падаем  
    return [];  
  }  
}
```

```
function sohraniKorzinu() {  
  localStorage.setItem('korzina', JSON.stringify(korzina));  
}
```

```
// ===== Вспомогательное =====
```

```
function ekranirovat(tekst) {  
  return String(tekst)  
    .replace(/&/g, '&amp;')  
    .replace(/</g, '&lt;');
```

```

        .replace(</>/g, '&gt;');
        .replace(/"/g, '&quot;');
    }

    function podsvetit(tekst, zapros) {
        const bezopasnyi = ekranirovat(tekst);
        if (!zapros) return bezopasnyi;
        const shablon = ekranirovat(zapros).replace(/[\.\*\?^\$\{\}\|\[\]\\\]/g, '\\$&');
        return bezopasnyi.replace(new RegExp(`(${shablon})`, 'gi'), '<mark>$1</mark>');
    }

    function debounce(funkciya, zaderzhka = 300) {
        let tajmer;
        return (...argumenty) => {
            clearTimeout(tajmer);
            tajmer = setTimeout(() => funkciya(...argumenty), zaderzhka);
        };
    }

    // ===== Отрисовка =====

    function kartochka(t, zapros) {
        const est = t.ostatok > 0;
        const vKorzine = korzina.find(p => p.id === t.id);

        return `
        <article class="tovar">
            <h3>${podsvetit(t.nazvanie, zapros)}</h3>
            <p class="artikul">Артикул: ${podsvetit(t.artikul, zapros)}</p>
            <p class="cena">${t.cena} <span>сомони</span></p>
            <p class="nalichie ${est ? '' : 'nety'}">${
                est ? `В наличии: ${t.ostatok} шт.` : 'Под заказ, 5 дней'
            }</p>
            <button data-id="${t.id}" ${est ? '' : 'disabled'}>
                ${vKorzine ? `В корзине: ${vKorzine.kolichestvo}` : 'В корзину'}
            </button>
        </article>`;
    }

    function narisovat(spisok, zapros) {
        const katalog = document.querySelector('.katalog');
        document.querySelector('.schetchik').textContent =

```

```

    spisok.length ? `Найдено товаров: ${spisok.length}` : '';

    if (spisok.length === 0) {
        katalog.innerHTML = `
            <div class="pusto">
                <h3>Ничего не нашлось</h3>
                <p>Попробуйте другой запрос или сбросьте фильтры.</p>
                <button class="sbros">Показать все товары</button>
            </div>`;
        return;
    }

    katalog.innerHTML = spisok.map(t => kartochka(t, zapros)).join('');
}

// ===== Фильтры =====

function primenitFiltry() {
    const zapros = document.querySelector('#poisk').value.trim();
    const nizhnij = zapros.toLowerCase();
    const brend = document.querySelector('#brend').value;
    const tolkoEst = document.querySelector('#v-nalichii').checked;
    const sortirovka = document.querySelector('#sort').value;

    let spisok = [...tovary];

    if (nizhnij) {
        spisok = spisok.filter(t =>
            t.nazvanie.toLowerCase().includes(nizhnij) ||
            t.artikul.toLowerCase().includes(nizhnij)
        );
    }

    if (brend)    spisok = spisok.filter(t => t.brend === brend);
    if (tolkoEst) spisok = spisok.filter(t => t.ostatok > 0);

    if (sortirovka === 'cena-vozh' ) spisok.sort((a, b) => a.cena - b.cena);
    if (sortirovka === 'cena-ubyv')  spisok.sort((a, b) => b.cena - a.cena);
    if (sortirovka === 'nazvanie')   spisok.sort((a, b) => a.nazvanie.localeCompare(b.nazvanie));

    narisovat(spisok, zapros);
}

```

```

// ===== Корзина =====

function obnovitKorzinu() {
    const shtuk = korzina.reduce((s, p) => s + p.kolichestvo, 0);
    const summa = korzina.reduce((s, p) => {
        const t = tovary.find(x => x.id === p.id);
        return s + (t ? t.cena * p.kolichestvo : 0);
    }, 0);

    document.querySelector('.korzina-shtuk').textContent = shtuk;
    document.querySelector('.korzina-summa').textContent = summa;
    document.querySelector('.korzina-ochistit').hidden = shtuk === 0;
}

function dobavit(id) {
    const tovar = tovary.find(t => t.id === id);
    if (!tovar || tovar.ostatok === 0) return;

    const est = korzina.find(p => p.id === id);
    if (est) {
        if (est.kolichestvo >= tovar.ostatok) return; // больше, чем на складе
        est.kolichestvo++;
    } else {
        korzina.push({ id, kolichestvo: 1 });
    }

    sohranitKorzinu();
    obnovitKorzinu();
}

function ochistit() {
    korzina = [];
    sohranitKorzinu();
    obnovitKorzinu();
    primenitFiltry();
}

// ===== События =====

document.querySelector('#poisk').addEventListener('input', debounce(primenitFiltry));
document.querySelector('#brend').addEventListener('change', primenitFiltry);
document.querySelector('#v-nalichii').addEventListener('change', primenitFiltry);

```

```

document.querySelector('#sort').addEventListener('change', primenitFiltry);

document.querySelector('.filtry').addEventListener('submit', (e) => {
    e.preventDefault();
    primenitFiltry();
});

// Одно делегирование на весь каталог
document.querySelector('.katalog').addEventListener('click', (e) => {
    const sbros = e.target.closest('.sbros');
    if (sbros) {
        document.querySelector('#poisk').value = '';
        document.querySelector('#brend').value = '';
        document.querySelector('#v-nalichii').checked = false;
        primenitFiltry();
        return;
    }

    const knopka = e.target.closest('button[data-id]');
    if (!knopka || knopka.disabled) return;

    dobavit(Number(knopka.dataset.id));
    primenitFiltry(); // перерисуем, чтобы кнопка показала количество
});

document.querySelector('.korzina-ochistit').addEventListener('click', () => {
    if (confirm('Очистить корзину?')) ochistit();
});

// ===== Старт =====

primenitFiltry();
obnovitKorzinu();

```

НА ЭКРАНЕ

На экране

Набрали «фильтр» — совпадения подсвечены, счётчик обновился:

Добавили товары — кнопки показывают количество, корзина считает сумму:

А теперь главное: обновите страницу клавишей F5. Корзина осталась. Это `localStorage`.

ГРАБЛИ

Грабли

`JSON.parse` на испорченных данных падает. Пользователь мог руками поменять `localStorage`, или у вас поменялся формат. Оборачивайте в `try/catch`, как в примере.

Хранить цены в `localStorage`. Покупатель поменяет и оформит по своей цене. Только id и количество.

Подсветка через `innerHTML` без экранирования. Прямая дыра XSS.

Debounce на обработчике фильтров. Выпадающий список и галочка должны срабатывать сразу — задержка там только раздражает. Debounce нужен только для текстового ввода.

Перерисовывать каталог ради счётчика корзины. Дорого. Правильнее менять только нужный элемент. В нашем примере перерисовка допустима — товаров восемь. На тысяче так делать нельзя.

Забыть, что `localStorage` привязан к браузеру. Корзина не переедет на телефон покупателя. Настоящие магазины хранят её на сервере — сделаем в главе 38.

ЗАДАЧИ

Задачи

Задача 18.1. Соберите каталог из главы целиком. Проверьте: поиск, фильтры, сортировку, корзину, сохранение после F5.

Задача 18.2. Добавьте кнопку «-» рядом с «В корзине: N», чтобы можно было убавлять количество.

Задача 18.3. Сделайте выпадающую панель корзины: список товаров с названиями, количеством и суммой по каждому.

Задача 18.4. Добавьте фильтр по цене «от» и «до». Не забудьте debounce для текстовых полей.

Задача 18.5. Сохраняйте в `localStorage` ещё и выбранные фильтры. Вернулись на сайт — фильтры на месте.

Задача 18.6. Сделайте так, чтобы поиск игнорировал регистр и лишние пробелы внутри запроса: «тормозные колодки» должно находить.

Задача 18.7. Проверьте безопасность: введите в поиск `<b>тест</b>`. Текст должен показаться как есть, а не стать жирным. Если стал жирным — экранирование не работает.

Задача 18.8. Замерьте разницу от `debounce`. Поставьте `console.log` в `применилFilttry`, наберите «колодки» и посчитайте вызовы с задержкой и без.

Задача 18.9. Добавьте в карточки кнопку «Сравнить» — до трёх товаров, список тоже в `localStorage`.

Задача 18.10. Откройте F12 → Application → Local Storage. Найдите свою корзину. Поменяйте количество руками и обновите страницу.

Получилось? Тогда объясните своими словами, почему цены нельзя доверять браузеру.

Ответы — в Приложении Г.

В БОЮ В бою

Ваш каталог работает целиком в браузере — потому что все восемь товаров лежат в файле со скриптом.

В настоящем магазине так нельзя, и вот почему:

НАШ КАТАЛОГ

БОЕВОЙ МАГАЗИН

Где товары	В коде страницы	В базе, десятки тысяч
Кто фильтрует	Браузер	Сервер, запросом к базе
Что приходит	Всё сразу	Только текущая страница, 20 штук
Где корзина	<code>localStorage</code>	На сервере, в сессии или базе
Кто считает цену	Браузер	Только сервер

Загрузить десять тысяч товаров в браузер невозможно: страница будет открываться минуту, а на мобильном интернете — вечность.

Но приёмы остаются те же. Делегирование, `debounce`, экранирование, состояния «пусто» — всё это в боевом коде есть. Меняется только источник данных: вместо массива в файле — запрос на сервер.

Этим и займёмся со следующей главы.

ИТОГ

Итог

- `localStorage` хранит строки и переживает перезагрузку. `JSON.stringify` и `JSON.parse` для объектов.
- В корзине — только `id` и количество. Цены считает сервер.
- Никаких паролей, токенов и денег в `localStorage`.
- `Debounce` — задержка для текстового ввода. 300 мс. Для списков и галочек не нужен.
- Экранируйте всё, что приходит от пользователя, перед вставкой в `innerHTML`.
- Оборачивайте `JSON.parse` в `try/catch`.
- Состояние «ничего не нашлось» должно предлагать выход.
- Каталог целиком в браузере годится для десятков товаров, не для тысяч.

Часть IV закончена. Вы умеете делать интерфейс живым.

Со следующей главы начинается бэкенд — та самая подсобка из первой главы. Мы наконец дойдём до сервера, где считаются настоящие деньги.

Что делает PHP и первый скрипт

ЗАЧЕМ ЭТО

Зачем эта глава

Мы переходим из торгового зала в подсобку.

До сих пор весь код выполнялся у **покупателя** — в его браузере. Теперь мы уходим на **сервер**, и это меняет очень многое:

- покупатель больше не видит наш код;
- покупатель не может его изменить;
- зато мы получаем доступ к базе данных, файлам и чужим сервисам.

Именно здесь считаются деньги, проверяются пароли и решается, кого пускать в админку.

В этой главе установим PHP, запустим первый скрипт и — главное — **на практике** почувствуем **разницу** между тем, что выполняется в браузере, и тем, что на сервере.

РАЗБИРАЕМСЯ

Ставим PHP

► Что нужно

Для работы понадобятся три вещи, но ставятся они одним махом:

ЧТО	ЗАЧЕМ
PHP	Выполняет наш серверный код
MySQL	База данных. Понадобится с главы 26
Веб-сервер	Принимает запросы из браузера

► Как поставить

СИСТЕМА	ЧТО СТАВИТЬ	ОТКУДА
Windows	OpenServer или XAMPP	<code>ospanel.io</code> / <code>apachefriends.org</code>
macOS	XAMPP или через Homebrew	<code>apachefriends.org</code>
Linux	Через пакетный менеджер	<code>sudo apt install php mysql-server</code>

Ставится как обычная программа. После установки запускаете панель управления и нажимаете «Старт».

► Проверяем

Откройте терминал и наберите:

```
php -v
```

Должно ответить что-то вроде:

```
PHP 8.3.14 (cli) (built: Nov 21 2024)
```

Если пишет «команда не найдена» — PHP установился, но система не знает, где его искать. На Windows это лечится добавлением пути к PHP в переменную `PATH`; в панели OpenServer есть готовая кнопка «Добавить в PATH».

Версия PHP 8 или выше. Седьмая ещё работает, но многое из книги в ней недоступно.

РАЗБИРАЕМСЯ

Первый скрипт

► Создаём файл

В папке проекта создайте `index.php`. Обратите внимание на расширение: не `.html`, а `.php`.

```
<?php
echo 'Привет! Это работает на сервере.';
```

► Запускаем

У PHP есть встроенный сервер — его достаточно для учёбы. В терминале, находясь в папке проекта:

```
php -S localhost:8000
```

Ответит примерно так:

```
PHP 8.3.14 Development Server (http://localhost:8000) started
```

Теперь откройте в браузере `http://localhost:8000`.

⚠ **Терминал не закрывайте.** Пока команда работает, работает и сервер. Закроете — сайт перестанет открываться. Остановить сервер: **Ctrl+C**.

► Что важно в адресе

Смотрите на адресную строку. Раньше было:

```
file:///Users/firdavs/projects/moi-magazin/index.html
```

Теперь:

```
http://localhost:8000
```

Это принципиальная разница:

Кто открывает файл	Браузер, напрямую с диска	Сервер обрабатывает и отдаёт результат
PHP выполняется	Нет. Покажет исходный код	Да
Это «настоящий сайт»	Нет	Да, только пока локальный

PHP-файл, открытый двойным щелчком, не работает. Обязательно через сервер. Это первое, обо что спотыкаются все новички.

РАЗБИРАЕМСЯ

Как PHP уживается с HTML

► Теги `<?php` и `?>`

```
<h1>Каталог</h1>

<?php
    $tovar = 'Тормозные колодки';
    echo $tovar;
?>

<p>Обычный HTML снова.</p>
```

Всё между `<?php` и `?>` — код PHP. Всё снаружи — обычный HTML, который отдаётся как есть.

⚠ Если в файле только PHP-код, закрывающий `?>` не ставят. Это не каприз: случайный пробел или перенос строки после `?>` попадёт в ответ и может сломать заголовки или загрузку файлов. Ошибку эту ищут долго и мучительно.

► Короткая запись для вывода

```
<h1><?php echo $nazvanie; ?></h1>
<h1><?= $nazvanie ?></h1>
```

`<?=` — то же самое, что `<?php echo`. В шаблонах пишут именно так: короче и читается лучше.

► Смешивать можно

```
<?php
$tovary = ['Колодки', 'Фильтр', 'Свечи'];
?>

<ul>
<?php foreach ($tovary as $tovar): ?>
    <li><?= $tovar ?></li>
<?php endforeach; ?>
</ul>
```

Получится список из трёх пунктов. Разберём эту конструкцию подробно в главе 22 — пока просто посмотрите, что HTML и PHP спокойно переплетаются.

Обратите внимание на непривычную запись `foreach (...): ... endforeach;` вместо фигурных скобок. В шаблонах она удобнее: сразу видно, что чем закрывается, когда между ними двадцать строк разметки.

РАЗБИРАЕМСЯ

Главное отличие от JavaScript

Вот тот момент, ради которого стоило начинать с JavaScript. Сравните.

JavaScript выполняется **после** того, как страница пришла в браузер. Покупатель может открыть F12 и увидеть весь код.

PHP выполняется **до** того, как страница отправится. Покупатель получает только **результат**.

► Убедитесь сами

Создайте `test.php`:

```
<?php
$sekretnayaNacenka = 1.35;
$cenaPostavshika = 185;
$cenaDlyaPokupatelya = $cenaPostavshika * $sekretnayaNacenka;
?>

<h1>Тормозные колодки</h1>
<p>Цена: <?= round($cenaDlyaPokupatelya) ?> сомони</p>
```

Откройте через сервер, потом нажмите правую кнопку → «Посмотреть код страницы».

Вы увидите:

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
```

И всё. Ни наценки, ни цены поставщика, ни самой формулы. Покупатель видит только итог.

Если бы тот же расчёт был на JavaScript, покупатель узнал бы и закупочную цену, и размер наценки — и, возможно, пошёл бы к поставщику напрямую.

Вот зачем нужен сервер. Не «чтобы было сложнее», а чтобы прятать то, что не должно быть видно.

РАЗБИРАЕМСЯ

Правила синтаксиса

Несколько вещей, которые отличаются от JavaScript. Перечислю сразу, чтобы не спотыкаться.

► Точка с запятой обязательна

```
echo 'Привет';
$cena = 250;
```

В JavaScript её часто можно опустить. В PHP — нет. Забыли — синтаксическая ошибка и белый экран.

► Переменные с долларом

```
$cena = 250;  
$nazvanie = 'Колодки';
```

Всегда `$`. Забыть его — самая частая опечатка при переходе с JavaScript.

► Склейка строк — точкой, а не плюсом

```
$text = 'Цена: ' . $cena . ' сомони';
```

⚠ В JavaScript было `+`. В PHP `+` — это только сложение чисел. Для склейки строк — точка.

Перепутаете — получите неожиданный результат или предупреждение.

► Подстановка в двойных кавычках

```
$nazvanie = 'Колодки';  
  
echo "Товар: $nazvanie";      // Товар: Колодки  
echo 'Товар: $nazvanie';     // Товар: $nazvanie
```

Двойные кавычки подставляют переменные, одинарные — нет.

Это не мелочь, а важное отличие от JavaScript, где кавычки равноправны.

Для сложных случаев переменную берут в фигурные скобки:

```
echo "Цена: {$tovar['cena']} сомони";
```

Практический совет: используйте одинарные кавычки по умолчанию и двойные — когда нужна подстановка. Так сразу видно, где происходит вставка значений.

► Комментарии

```
// однострочный
# тоже однострочный, но так почти не пишут
/* многострочный
   комментарий */
```

РАЗБИРАЕМСЯ

Как читать ошибки PHP

Ошибки — не беда, а подсказка. Но их нужно видеть.

► Включите показ ошибок

Для учёбы добавьте в начало файла:

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);
```

⚠ На боевом сайте так делать нельзя. Текст ошибки может выдать пути на диске, имена таблиц и другие внутренности. Там ошибки пишут в лог, а посетителю показывают вежливое сообщение (глава 56).

► Как читать ошибку

```
Parse error: syntax error, unexpected token ";" in /var/www/index.php on line 1
```

Разбирается механически:

ЧАСТЬ	ЧТО ГОВОРИТ
Parse error	Тип: синтаксическая ошибка
unexpected token ";"	Что не так: точка с запятой не на месте
/var/www/index.php	В каком файле
on line 12	В какой строке

⚠ **Важная тонкость:** PHP часто указывает на строку, где заметил проблему, а не где она возникла. Забытая точка с запятой в строке 11 покажется ошибкой в строке 12.

Правило: увидели ошибку в строке N — проверьте и строку N-1.

► Типы ошибок

ТИП	ЧТО ОЗНАЧАЕТ	ПРИМЕР
Parse error	Синтаксис. Файл вообще не запустился	Забыли <code>;</code> или скобку
Fatal error	Выполнение остановилось	Вызвали несуществующую функцию
Warning	Что-то не так, но работаем дальше	Файл не найден
Notice / Deprecated	Мелкие замечания	Обращение к несуществующему ключу

Белый экран без текста — почти всегда Parse error при выключенном показе ошибок. Включите — и увидите причину.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

// Данные о магазине
$nazvanieMagazina = 'Автозапчасти Firdavs';
$gorod = 'Худжанд';

// Данные о товаре
$tovar = 'Тормозные колодки Bosch';
$artikul = '0986424815';
$cenaPostavshika = 185;
$nacenka = 1.35;
$ostatok = 7;

// Считаем цену — покупатель этого не увидит
$cena = round($cenaPostavshika * $nacenka);

// Определяем статус
if ($ostatok > 5) {
    $status = 'В наличии';
} elseif ($ostatok > 0) {
    $status = 'Осталось мало';
} else {
    $status = 'Под заказ';
}

// Текущая дата — её знает только сервер
$data = date('d.m.Y H:i');
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title><?= $tovar ?> — <?= $nazvanieMagazina ?></title>
</head>
<body>
    <h1><?= $nazvanieMagazina ?></h1>
    <p><?= $gorod ?></p>
```

```
<h2><?= $tovar ?></h2>
<p>Артикул: <?= $artikul ?></p>
<p>Цена: <strong><?= $cena ?></strong> сомони</p>
<p>Наличие: <?= $status ?> (<?= $ostatok ?> шт.)</p>

<hr>
<p>Страница собрана сервером <?= $data ?></p>
</body>
</html>
```

Обратите внимание на дату. Она берётся с сервера — браузер её не вычислял. Обновите страницу: время изменится. Значит, страница **собирается заново при каждом запросе**.

В этом ключевое отличие от статичного HTML-файла, который просто лежит на диске.

НА ЭКРАНЕ

На экране

Цена 250 сомони. А в исходном коде страницы, который получил браузер, нет ни числа 185, ни коэффициента 1.35 — только результат.

ЗАПИСЬ

КАК ЧИТАЕТСЯ

ЧТО ДЕЛАЕТ

<code><?php</code>		Начало кода PHP
<code>?></code>		Конец. В чистом PHP-файле не ставится
<code><?= \$x ?></code>		Короткая запись <code><?php echo \$x; ?></code>
<code>echo</code>	«эхо»	Вывести на страницу
<code>\$цена</code>		Переменная. Доллар обязателен
<code>.</code>	точка	Склейка строк (в JS был <code>+</code>)
<code>;</code>		Конец команды. Обязательна
<code>'текст'</code>		Строка без подстановки переменных
<code>"текст \$x"</code>		Строка с подстановкой
<code>round()</code>		Округлить
<code>date('d.m.Y')</code>		Текущая дата сервера
<code>elseif</code>		«Иначе если». В PHP пишется слитно
<code>php -S localhost:8000</code>		Запустить встроенный сервер
<code>localhost</code>		Ваш собственный компьютер

ГРАБЛИ

Грабли

Открыть `.php` двойным щелчком. Браузер покажет исходник. Только через сервер.

Забывать `$ . цена = 250` — PHP решит, что это константа, и выдаст ошибку.

Склеить строки плюсом. `'Цена: ' + $цена` — в PHP это попытка сложения.

Забывать точку с запятой. Parse error, обычно с указанием на следующую строку.

Оставить `?>` в конце файла. Пробел после него попадёт в вывод. В файлах с одним PHP закрывающий тег не ставят.

Белый экран. Включите `display_errors` — увидите причину.

Закройте терминал с запущенным сервером. Сервер остановится, сайт перестанет открываться.

Русские буквы превратились в кракозябры. Проверьте `<meta charset="UTF-8">` и что сам файл сохранён в UTF-8.

ЗАДАЧИ

Задачи

Задача 19.1. Поставьте PHP, проверьте `php -v`, запустите встроенный сервер и откройте страницу в браузере.

Задача 19.2. Сделайте страницу товара из примера. Замените товар на свой.

Задача 19.3. Что выведет каждая строка?

```
<?php
$a = 5;
$b = '5';
echo $a + $b;
echo 'Цена: ' . $a;
echo "Цена: $a";
echo 'Цена: $a';
```

Задача 19.4. Найдите три ошибки:

```
<?php
сена = 250
echo "Цена: " + сена;
```

Задача 19.5. Сделайте страницу, которая показывает разное приветствие в зависимости от времени суток: до 12 — «Доброе утро», до 18 — «Добрый день», позже — «Добрый вечер». Подсказка: `date('H')` даёт текущий час.

Задача 19.6. Посчитайте на сервере цену со скидкой: если товаров больше трёх, скидка 10%. Выведите обе цены — исходную и итоговую.

Задача 19.7. Откройте «Посмотреть код страницы» на своей PHP-странице. Найдите там формулу расчёта цены. Нашли? Объясните, почему нет.

Задача 19.8. Специально сломайте файл: уберите точку с запятой. Что покажет браузер? Теперь включите `display_errors` и посмотрите снова.

Задача 19.9. Сделайте так, чтобы страница показывала, сколько дней осталось до Нового года. Подсказка: `strtotime('2027-01-01')` и `time()`.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `index.php` в корне этого репозитория. Первые строки будут вам знакомы: подключение общих файлов, проверки, подготовка данных — и только потом HTML.

Это стандартная структура PHP-страницы: **сначала логика, потом разметка**.

Ещё обратите внимание: в боевом коде нет `ini_set('display_errors', 1)`. Настройки показа ошибок вынесены в конфигурацию сервера — на рабочем сайте они выключены, а ошибки пишутся в лог.

ИТОГ

- PHP выполняется **на сервере**, до отправки страницы. Покупатель видит только результат.
- Поэтому цены, наценки и доступы считает PHP, а не JavaScript.
- Расширение файла — `.php`, открывать только **через сервер**: `php -S localhost:8000`.
- `<?php ... ?>` — код, `<?= $x ?>` — короткий вывод.
- `$` перед переменными обязателен, `;` в конце обязательна.
- Склейка строк — **точка**, не плюс.
- Двойные кавычки подставляют переменные, одинарные — нет.
- В чистом PHP-файле **не ставьте** `?>` в конце.
- Ошибка в строке N — проверьте и строку N-1.
- Белый экран = Parse error при выключенном показе ошибок.

Дальше — переменные и типы PHP подробно.

Переменные, типы и вывод

ЗАЧЕМ ЭТО

Зачем эта глава

Хорошая новость: вы уже знаете эту тему. Переменные, строки, числа — всё это было в главе 14, когда мы учили JavaScript.

Плохая новость: в PHP всё это записывается немного иначе, и на мелких отличиях спотыкаются даже опытные люди, переключающиеся между языками.

Поэтому глава построена как **сравнение**: что одинаково, что различается и где именно вас подстерегает ошибка.

РАЗБИРАЕМСЯ

Переменные

► Объявление

```
$cena = 250;
$nazvanie = 'Тормозные колодки';
$vNalichii = true;
```

В JAVASCRIPT

```
const cena = 250;
```

```
let cena = 250;
```

В PHP

```
$cena = 250;
```

```
$cena = 250;
```

Никаких **let** и **const**. Просто имя со знаком доллара.

⚠ Доллар обязателен всегда — и при создании, и при использовании:

```
$cena = 250;
echo $cena;           // 
echo cena;           //  PHP решит, что это константа, и выдаст ошибку
```

► Константы — если значение не должно меняться

```
define('NDS', 0.18);
const KURS_RUBLYA = 1.15;

echo NDS;           // без доллара!
```

Константы пишут ЗАГЛАВНЫМИ — это соглашение всех языков. Меняться они не могут.

► Имена

Правила те же, что в JavaScript: латиница, цифры, `_`, начинать не с цифры, регистр важен.

А вот стиль другой:

```
$cena_tovara = 250;           // так принято в PHP: snake_case
$cenaTovara = 250;           // тоже встречается, но реже
```

В PHP принят `snake_case` — слова через подчёркивание. В JavaScript был `camelCase`.

Строгого запрета нет, но **придерживайтесь одного стиля в проекте**. Разной — признак неаккуратности.

РАЗБИРАЕМСЯ

Типы данных

```
$cena = 250;           // int — целое
$ves = 1.5;           // float — дробное
$name = 'Колодки';     // string — строка
$estVnalichii = true;  // bool — да/нет
$skidka = null;        // null — ничего
$tovary = ['a', 'b'];  // array — массив (глава 22)
```

Проверить тип:

```
var_dump($cena);           // int(250)
var_dump($nazvanie);       // string(7) "Колодки"
var_dump($estVnalichii);   // bool(true)
```

`var_dump()` — главный инструмент отладки в PHP. Аналог `console.log`, но показывает ещё и тип, и длину. Пользуйтесь им постоянно.

Для массивов удобнее:

```
print_r($tovary);
```

Красивее всего — в `<pre>`, чтобы сохранились отступы:

```
echo '<pre>';
print_r($tovary);
echo '</pre>';
```

РАЗБИРАЕМСЯ

Строки: три отличия от JavaScript

► Отличие 1: склейка точкой

```
$text = 'Цена: ' . $cena . ' сомони';
```

⚠ В JavaScript был `+`. В PHP плюс — только сложение чисел.

```
echo 'Цена: ' + 250;    // ❌ ошибка или неожиданный результат
echo 'Цена: ' . 250;    // ✅
```

Есть и сокращение для дописывания:

```
$text = 'Товар: ';
$text .= 'Колодки';      // теперь 'Товар: Колодки'
```

► Отличие 2: кавычки не равноправны

```
$nazvanie = 'Колодки';

echo "Товар: $nazvanie";      // Товар: Колодки
echo 'Товар: $nazvanie';      // Товар: $nazvanie
```

КАВЫЧКИ	ПОДСТАВЛЯЮТ ПЕРЕМЕННЫЕ	РАБОТАЮТ БЫСТРЕЕ
Одинарные <code>' ... '</code>	Нет	Да
Двойные <code>" ... "</code>	Да	Чуть медленнее

Для сложных выражений — фигурные скобки:

```
echo "Цена: {$tovar['цена']} сомони";
echo "Итого: {$цена} сомони";
```

Совет: одинарные по умолчанию, двойные — когда нужна подстановка. Так по кавычкам сразу видно, где происходит вставка.

► Отличие 3: нет шаблонных строк с обратными кавычками

Того, к чему вы привыкли в JavaScript, здесь нет:

```
`Цена ${цена} сомони`      // так в JavaScript
```

```
"Цена $цена сомони"        // так в PHP
```

⚠ Обратные кавычки в PHP означают совсем другое — выполнение команды операционной системы. Никогда их не используйте: это прямая дорога к взлому сервера.

► Полезные функции для строк

```
strlen('Колодки');           // длина в байтах
mb_strlen('Колодки');        // длина в символах — для русского нужна э
mb_strtolower('BOSCH');      // bosch
mb_strtoupper('bosch');      // BOSCH
trim(' текст ');            // убрать пробелы по краям
str_replace('Bosch', 'BOSCH', $s); // заменить
mb_substr($s, 0, 10);        // первые 10 символов
str_contains($s, 'фильтр');  // содержит ли (PHP 8+)
explode(',', 'a,b,c');       // разбить в массив
implode(' ', $massiv);       // склеить массив в строку
number_format(1234.5, 2, '.', ' '); // 1 234.50
```

⚠ Обратите внимание на приставку **mb_** — от *multibyte*, «многобайтовый».

Русская буква занимает два байта, английская — один. Обычная `strlen('Привет')` вернёт 12, а не 6. `substr` может разрезать букву пополам и выдать мусор.

Правило: для строк с русским текстом всегда берите функции с **mb_**. Это одна из тех вещей, о которых не пишут в учебниках, а потом полдня ищут причину странного поведения.

РАЗБИРАЕМСЯ

Числа

```
$a = 10;
$b = 3;

echo $a + $b;    // 13
echo $a - $b;    // 7
echo $a * $b;    // 30
echo $a / $b;    // 3.3333333333333
echo $a % $b;    // 1 — остаток
echo $a ** 2;    // 100 — возведение в степень
echo intval($a, $b); // 3 — целочисленное деление
```

Полезные функции:

```
round(3.7);           // 4
round(3.14159, 2);    // 3.14
floor(3.7);           // 3 — вниз
ceil(3.2);            // 4 — вверх
abs(-5);              // 5
max(3, 7, 2);         // 7
min(3, 7, 2);         // 2
rand(1, 100);         // случайное
number_format(1234567); // 1,234,567
```

► Деньги — снова про целые числа

Помните правило из главы 14? В PHP оно тоже действует:

```
var_dump(0.1 + 0.2 === 0.3); // false
```

Храните деньги в дирамах, целыми числами. 250 сомони = 25000 дирамов. Делите на 100 только при показе.

РАЗБИРАЕМСЯ

Приведение типов — главная ловушка PHP

Вот тема, на которой ловятся все.

PHP очень свободно обращается с типами. Иногда это удобно, иногда опасно.

```
echo '5' + 3;           // 8 — строка стала числом
echo '5 сомони' + 3;    // 8 — с предупреждением
echo 5 . 3;             // 53 — числа стали строкой
```

► Сравнение: `==` против `===`

```
var_dump(5 == '5');      // true - сравнил, приведя типы
var_dump(5 === '5');    // false - разные типы

var_dump(0 == 'abc');    // false в PHP 8 (в PHP 7 было true!)
var_dump('' == null);    // true
var_dump('0' == false);  // true
```

⚠ Эти сравнения — источник **реальных уязвимостей**. Классический пример: проверка токена через `==`, где токен, начинающийся с `0e`, случайно совпадал с другим.

Правило без исключений: всегда `===` и `!==`.

► Явное превращение

```
$stroka = '250';

$chislo = (int) $stroka;      // 250
$drobnoe = (float) '3.14';    // 3.14
$text = (string) 250;         // '250'
$logika = (bool) 1;           // true

// Аккуратнее и понятнее:
$chislo = intval($stroka);
$drobnoe = floatval($stroka);
```

Данные из формы всегда приходят строками. `$_POST['kolichestvo']` — это строка `'3'`, а не число. Перед арифметикой превращайте явно.

► Что считается «пустым»

```
empty('');      // true
empty('0');     // true ← внимание!
empty(0);       // true
empty([]);      // true
empty(null);    // true
empty('текст'); // false
```


⚠ `empty('0')` возвращает `true`. Это ловушка: если человек ввёл в поле цифру ноль, `empty()` решит, что поле не заполнено.

Для проверки «поле заполнено» надёжнее:

```
if (isset($_POST['kol']) && $_POST['kol'] !== '') { ... }
```

ФУНКЦИЯ

ЧТО ПРОВЕРЯЕТ

<code>isset(\$x)</code>	Переменная существует и не <code>null</code>
<code>empty(\$x)</code>	Существует и «пустая» (см. ловушку выше)
<code>is_numeric(\$x)</code>	Является ли числом (в том числе строкой-числом)
<code>\$x === null</code>	Строго ли <code>null</code>

РАЗБИРАЕМСЯ

Вывод

```
echo 'Привет';  
echo 'Цена: ', $цена, ' сомони';    // можно через запятую  
  
print 'Привет';                    // почти то же самое  
  
printf('Цена: %d сомони', $цена);  // с форматированием  
echo number_format(1234.5, 2);     // 1,234.50
```

`echo` — основной способ. `printf` пригодится для форматирования чисел.

► Экранирование при выводе — обязательная привычка

Помните XSS из главы 16? В PHP та же опасность:

```
$имя = $_POST['имя'];                // а вдруг там <script>...  
echo $имя;                          // ❌ опасно  
echo htmlspecialchars($имя);        // ✅ безопасно
```

`htmlspecialchars()` превращает `<` в `<`, `>` в `>`, кавычки — в их коды. Опасный код показывается как текст и не выполняется.

Правило: любое значение, пришедшее извне, выводится через `htmlspecialchars()`. Извне — это форма, база данных, чужой API, адресная строка. То есть почти всё.

Многие пишут короткую обёртку:

```
function e(string $text): string {  
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');  
}  
  
echo e($imya);
```

Подробно — в главе 36, но привычку заводите сейчас.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

function e(string $text): string {
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
}

// Данные (потом придут из базы)
$tovar = 'Тормозные колодки Bosch';
$artikul = '0986424815';
$brend = 'Bosch';

// Деньги — в дирамах, целыми
$cena_postavshika_diram = 18500;
$nacenka_procent = 35;
$dostavka_diram = 1200;

$ostatok = 7;
$kolichestvo = 3;

// Считаем
$cena_diram = (int) round($cena_postavshika_diram * (1 + $nacenka_procent / 100);
$cena_diram += $dostavka_diram;
$itogo_diram = $cena_diram * $kolichestvo;

// Скидка от трёх штук
$skidka_diram = 0;
if ($kolichestvo ≥ 3) {
    $skidka_diram = (int) round($itogo_diram * 0.10);
    $itogo_diram -= $skidka_diram;
}

// Для показа — переводим в сомони
function somoni(int $diram): string {
    return number_format($diram / 100, 2, '.', ' ');
}

$status = $ostatok > 5 ? 'В наличии' : ($ostatok > 0 ? 'Осталось мало' : 'Под э
```

```

?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title><?= e($tovar) ?></title>
</head>
<body>
    <h1><?= e($tovar) ?></h1>
    <p>Бренд: <?= e($brend) ?> · Артикул: <?= e($artikul) ?></p>
    <p>Наличие: <?= $status ?> (<?= $ostatok ?> шт.)</p>

    <h2>Расчёт</h2>
    <p>Цена за штуку: <?= somoni($cena_diram) ?> сомони</p>
    <p>Количество: <?= $kolichество ?> шт.</p>
    <?php if ($skidka_diram > 0): ?>
        <p>Скидка за объём: -<?= somoni($skidka_diram) ?> сомони</p>
    <?php endif; ?>
    <p><strong>К оплате: <?= somoni($itogo_diram) ?> сомони</strong></p>
</body>
</html>

```

Обратите внимание на три вещи:

Деньги в дирамах. Все расчёты целыми числами, перевод в сомони только при выводе.

function somoni(int \$diram): string — здесь указаны **типы**: на входе целое, на выходе строка. PHP проверит и сообщит, если передадут не то. Необязательно, но очень полезно — ошибки находятся сразу.

Всё выводится через e(). Даже там, где данные пока свои. Привычка.

РАЗБОР ПО СЛОВАМ

Разбор по словам

PHP	JAVASCRIPT	ЧТО ДЕЛАЕТ
<code>\$cena = 250;</code>	<code>let cena = 250;</code>	Переменная
<code>.</code>	<code>+</code>	Склейка строк
<code>.=</code>	<code>+=</code>	Дописать к строке
<code>"текст \$x"</code>	<code>`текст \${x}`</code>	Подстановка
<code>echo</code>	<code>console.log</code> (примерно)	Вывести
<code>var_dump()</code>	<code>console.log</code>	Показать значение и тип
<code>print_r()</code>	—	Показать массив читаемо
<code>===</code>	<code>===</code>	Строгое сравнение. Всегда так
<code>(int) \$x</code>	<code>Number(x)</code>	В число
<code>mb_strlen()</code>	<code>.length</code>	Длина с учётом русских букв
<code>htmlspecialchars()</code>	—	Экранировать перед выводом
<code>isset()</code>	<code>≡ undefined</code>	Существует ли
<code>empty()</code>	—	Пустая ли. Осторожно с <code>'0'</code>
<code>number_format()</code>	<code>toLocaleString</code>	Красиво вывести число

ГРАБЛИ

Грабли

Забывать `$`. Самая частая опечатка при переходе с JavaScript.

Склеить строки плюсом. В PHP это сложение.

`strlen` на русском тексте. Вернёт байты, а не буквы. Нужен `mb_strlen`.

Использовать `==`. Источник настоящих уязвимостей. Только `===`.

`empty('0')` считает ноль пустым. Для форм используйте `isset` плюс сравнение с `''`.

Не экранировать вывод. Дыра XSS. `htmlspecialchars` для всего внешнего.

Ждать, что данные из формы придут числами. Всегда строки. Превращайте явно.

Деньги в дробях. Дираны, целые числа.

задачи

Задачи

Задача 20.1. Что выведет каждая строка? Ответьте, потом проверьте:

```
<?php
$a = 5;
$b = '5';
var_dump($a == $b);
var_dump($a === $b);
echo $a + $b;
echo $a . $b;
echo "Значение: $a";
echo 'Значение: $a';
```

Задача 20.2. Найдите четыре ошибки:

```
<?php
цена = 250;
$nazvanie = 'Колодки'
echo "Товар: " + $nazvanie
echo 'Цена: $цена';
```

Задача 20.3. Напишите функцию `somoni($diram)`, которая переводит дираны в сомони с двумя знаками после точки и пробелом между тысячами.

Задача 20.4. Посчитайте: цена поставщика 4300 рублей, курс 1.15 сомони за рубль, наценка 40%, доставка 80 сомони. Какая итоговая цена? Считайте в дирамах.

Задача 20.5. Проверьте разницу:

```
<?php
$s = 'Колодки';
echo strlen($s);
echo mb_strlen($s);
```

Почему числа разные? Объясните своими словами.

Задача 20.6. Что вернёт `empty()` для каждого значения: `''`, `'0'`, `0`, `'текст'`, `[]`, `null`, `false`, `'false'`? Проверьте все восемь.

Задача 20.7. Напишите функцию `sklonenie($n, $odin, $dva, $mnogo)`, которая возвращает правильное слово: 1 товар, 2 товара, 5 товаров.

Подсказка: смотрите на остаток от деления на 10 и на 100.

Задача 20.8. Проверьте экранирование. Присвойте переменной `'<script>alert(1)</script>'` и выведите двумя способами: через `echo` и через `htmlspecialchars`. Посмотрите исходный код страницы. В чём разница?

Задача 20.9. Сделайте страницу «прайс-лист» из пяти товаров, где цена считается из закупочной с наценкой. Все деньги — целыми числами.

Ответы — в Приложении Г.

Откройте `includes/` в этом репозитории и поищите вызовы `htmlspecialchars`. Найдёте их всюду, где что-то выводится.

Ещё найдите работу с ценами — увидите то же правило: расчёты в целых числах, форматирование только при показе.

И одна деталь, важная именно для нашего проекта: цены приходят от поставщика в рублях и пересчитываются в сомони по курсу с наценкой. Все промежуточные вычисления идут в мелких единицах, иначе на тысяче товаров накопится расхождение в несколько сомони — и бухгалтерия не сойдётся.

ИТОГ

Итог

- `$переменная` — доллар обязателен. Никаких `let` и `const`.
- Склейка строк — **точка**, не плюс.
- Двойные кавычки подставляют переменные, одинарные — нет.
- `var_dump()` показывает значение и тип. Главный инструмент отладки.
- Для русского текста — функции с приставкой `mb_`.
- Только `===`. `=` в PHP опаснее, чем в JavaScript.
- Данные из формы — **всегда строки**. Превращайте явно.
- `empty('')` возвращает `true`. Помните об этом при проверке форм.
- Экранируйте вывод через `htmlspecialchars()`.
- Деньги — целыми числами в мелких единицах.
- В PHP принят `snake_case`.

Дальше — условия и циклы: та же логика, что в JavaScript, но с важными шаблонными приёмами.

Условия и циклы

ЗАЧЕМ ЭТО

Зачем эта глава

Логика в PHP устроена так же, как в JavaScript: `if`, `else`, `for`, `while`. Вы это уже знаете.

Но есть два важных дополнения, ради которых стоит прочитать главу:

1. Особая запись для шаблонов — когда PHP переплетается с HTML.
2. `switch` и `match` — удобные конструкции для множественного выбора, которые в шаблонах магазина встречаются постоянно.

Плюс разберём, как не превратить страницу в кашу из тегов и кода.

РАЗБИРАЕМСЯ

Условия

► Основное

```
if ($ostatok > 0) {
    echo 'В наличии';
} elseif ($ostatok === 0 && $pod_zakaz) {
    echo 'Под заказ';
} else {
    echo 'Нет';
}
```

Единственное отличие от JavaScript: `elseif` пишется слитно. `else if` раздельно тоже работает, но принято слитно.

► Операторы

```
$a === $b      // строго равно
$a !== $b      // строго не равно
$a > $b        // больше
$a ≤ $b        // меньше или равно
$a && $b        // и
$a || $b        // или
!$a            // не
$a ⇔ $b        // «космический корабль»: -1, 0 или 1
```

⇔ пригодится для сортировки в главе 22.

► Короткие записи

```
// Тернарный — как в JavaScript
$status = $ostatok > 0 ? 'Есть' : 'Нет';

// Если пусто — взять запасное (PHP 7+)
$brend = $_GET['brend'] ?? 'Все';

// То же с присваиванием
$_SESSION['korzina'] ??= [];
```

?? — оператор «null coalescing», и он невероятно полезен.

```
$stranica = $_GET['page'] ?? 1;
```

Читается: «взять `$_GET['page']`, а если его нет или он `null` — взять 1».

Без него пришлось бы писать:

```
$stranica = isset($_GET['page']) ? $_GET['page'] : 1;
```

⚠ Не путайте ?? с ?: :

ОПЕРАТОР

СРАБАТЫВАЕТ, КОГДА СЛЕВА

??

не существует или `null`

?:

«пусто» — `0`, `'`, `[]`, `false` тоже

Для параметров из адресной строки берите `??` — иначе `page=0` неожиданно превратится в единицу.

РАЗБИРАЕМСЯ

Множественный выбор

► switch

```
switch ($status) {  
    case 'new':  
        $podpis = 'Новый заказ';  
        break;  
    case 'paid':  
        $podpis = 'Оплачен';  
        break;  
    case 'shipped':  
    case 'delivering':  
        $podpis = 'В пути';  
        break;  
    default:  
        $podpis = 'Неизвестный статус';  
}
```

⚠ **break** обязателен. Без него выполнение «протечёт» в следующий `case`. Иногда это нужно нарочно (как выше для `shipped` и `delivering`), но чаще это забытый `break` и загадочная ошибка.

► `match` — современная замена (PHP 8)

```
$podpis = match ($status) {  
    'new' ⇒ 'Новый заказ',  
    'paid' ⇒ 'Оплачен',  
    'shipped', 'delivering' ⇒ 'В пути',  
    'done' ⇒ 'Доставлен',  
    default ⇒ 'Неизвестный статус',  
};
```

Чем `match` лучше:

	SWITCH	MATCH
<code>break</code>	Нужен	Не нужен
Сравнение	<code>=</code> (нестрогое!)	<code>===</code> строгое
Возвращает значение	Нет	Да
Забыли вариант	Молча пропустит	Ошибка

Берите `match`, если у вас PHP 8. Он короче и безопаснее. Статусы заказов, роли пользователей, типы доставки — всё это ложится на `match` идеально.

РАЗБИРАЕМСЯ Циклы

► `for` и `while` — как в JavaScript

```
for ($i = 1; $i ≤ 5; $i++) {  
    echo "Страница $i ";  
}  
  
$ostatok = 3;  
while ($ostatok > 0) {  
    echo "Продали, осталось " . (--$ostatok) . "\n";  
}
```

► **foreach** — главный цикл PHP

```
$brendy = ['Bosch', 'Mann', 'Denso'];

foreach ($brendy as $brend) {
    echo $brend;
}
```

С ключами:

```
$ceny = ['kolodki' => 250, 'filtr' => 45];

foreach ($ceny as $kod => $cena) {
    echo "$kod: $cena сомони\n";
}
```

foreach — аналог **for...of** из JavaScript, но умеет ещё и ключи. В работе с массивами используется в 90% случаев.

Подробно — в следующей главе, там же и про массивы.

► **break** и **continue**

```
foreach ($tovary as $t) {
    if ($t['ostatok'] === 0) continue; // пропустить
    if ($t['cena'] > 1000) break;       // выйти совсем
    echo $t['nazvanie'];
}
```

РАЗБИРАЕМСЯ

Шаблонный синтаксис — важная тема

Вот то, ради чего написана эта глава.

► Проблема

Когда PHP смешивается с HTML, обычные фигурные скобки читаются плохо:

```
<?php if ($ostatok > 0) { ?>
    <p class="est">В наличии</p>
    <button>Купить</button>
<?php } else { ?>
    <p class="net">Под заказ</p>
<?php } ?>
```

Между `{` и `}` двадцать строк разметки, и уже непонятно, что чему принадлежит. Одинокая `}` в начале строки среди HTML выглядит как мусор.

► Решение: альтернативный синтаксис

```
<?php if ($ostatok > 0): ?>
    <p class="est">В наличии</p>
    <button>Купить</button>
<?php else: ?>
    <p class="net">Под заказ</p>
<?php endif; ?>
```

Вместо `{` пишется `:`, вместо `}` — осмысленное слово: `endif`, `endforeach`, `endfor`, `endwhile`, `endswitch`.

Сразу видно, что именно закрывается — особенно когда конструкций несколько подряд.

► Полный набор

```
<?php if ($x): ?>      ... <?php endif; ?>
<?php foreach ($a as $b): ?> ... <?php endforeach; ?>
<?php for ($i=0; $i<5; $i++): ?> ... <?php endfor; ?>
<?php while ($x): ?>   ... <?php endwhile; ?>
```

Правило, принятое почти везде:

- В логике (расчёты, обработка данных) — обычные фигурные скобки.
- В шаблоне (где вперемешку с HTML) — двоеточие и `endif`.

Ещё одно правило, важнее синтаксиса.

► **Плохо: логика вперемишку с разметкой**

```
<div class="katalog">
<?php
foreach ($tovary as $t) {
    $cena = $t['zakup'] * 1.35;
    if ($t['ostatok'] > 0) {
        $cena = $cena * 0.95;
    }
    echo '<article><h3>' . $t['nazvanie'] . '</h3>';
    echo '<p>' . round($cena) . ' сомони</p></article>';
}
?>
</div>
```

Читать это тяжело: расчёты, условия и разметка перемешаны, HTML спрятан внутри строк.

► Хорошо: сначала данные, потом вывод

```
<?php
// 1. ЛОГИКА — считаем всё заранее
foreach ($tovary as &$t) {
    $t['cena'] = (int) round($t['zakup'] * 1.35);
    if ($t['ostatok'] > 0) {
        $t['cena'] = (int) round($t['cena'] * 0.95);
    }
}
unset($t); // обязательно после цикла со ссылкой
?>

<!-- 2. ВЫВОД — только показываем готовое -->
<div class="katalog">
    <?php foreach ($tovary as $t): ?>
        <article class="tovar">
            <h3><?= e($t['nazvanie']) ?></h3>
            <p class="cena"><?= $t['cena'] ?> сомони</p>
        </article>
    <?php endforeach; ?>
</div>
```

Разница огромная:

- Сверху — вся логика, видна целиком;
- Снизу — чистая разметка, читается как HTML;
- Верстальщик может править низ, не боясь сломать расчёты.

Это правило распространяется на весь бэкэнд. Оно называется «отделение логики от представления», и на нём построены все шаблонизаторы и фреймворки.

⚠ Про `&$t` в примере: амперсанд означает «работать с оригиналом, а не с копией», иначе изменения потеряются. А `unset($t)` после цикла обязателен — без него переменная остаётся ссылкой на последний элемент, и следующий цикл его затрёт. Это классическая ловушка PHP; в главе 22 разберём подробнее и покажем, как обойтись без ссылок вовсе.


```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

function e(string $t): string {
    return htmlspecialchars($t, ENT_QUOTES, 'UTF-8');
}

// --- Данные ---
$tovary = [
    ['nazvanie' => 'Тормозные колодки Bosch', 'zakup' => 18500, 'ostatok' => 7,
    ['nazvanie' => 'Масляный фильтр Mann', 'zakup' => 3300, 'ostatok' => 23,
    ['nazvanie' => 'Свечи зажигания Denso', 'zakup' => 8900, 'ostatok' => 0,
    ['nazvanie' => 'Тормозные диски Brembo', 'zakup' => 57800, 'ostatok' => 2,
    ['nazvanie' => 'Воздушный фильтр Mann', 'zakup' => 4800, 'ostatok' => 14,
    ['nazvanie' => 'Аккумулятор Bosch S4', 'zakup' => 70400, 'ostatok' => 4,
];

// --- Логика: считаем цены и статусы ---
$nacenka = 1.35;
$gotovye = [];

foreach ($tovary as $t) {
    $cena_diram = (int) round($t['zakup'] * $nacenka);

    // Акционным — минус 15%
    if ($t['status'] === 'akciya') {
        $cena_diram = (int) round($cena_diram * 0.85);
    }

    // Подпись наличия
    if ($t['ostatok'] === 0) {
        $nalichie = 'Под заказ, 5 дней';
        $klass_nalichiya = 'nety';
    } elseif ($t['ostatok'] ≤ 3) {
        $nalichie = "Осталось {$t['ostatok']} шт.";
        $klass_nalichiya = 'malo';
    } else {
        $nalichie = "В наличии: {$t['ostatok']} шт.";
    }
}
```

```

        $klass_nalichiya = '';
    }

    // Метка — match вместо трёх if
    $metka = match ($t['status']) {
        'hit'      ⇒ 'Хит продаж',
        'akciya'   ⇒ 'Акция -15%',
        default    ⇒ '',
    };

    $gotovye[] = [
        'nazvanie' ⇒ $t['nazvanie'],
        'cena'     ⇒ number_format($cena_diram / 100, 2, '.', ' '),
        'nalichie' ⇒ $nalichie,
        'klass'    ⇒ $klass_nalichiya,
        'metka'    ⇒ $metka,
        'mozhno_kupit' ⇒ $t['ostatok'] > 0,
    ];
}

$vsego_shtuk = 0;
foreach ($tovary as $t) {
    $vsego_shtuk += $t['ostatok'];
}
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Каталог</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <header>
        <div class="logo">Автозапчасти Firdavs</div>
    </header>

    <main>
        <p class="schetchik">
            Позиций: <?= count($gotovye) ?> · всего на складе: <?= $vsego_shtuk
        </p>

```

```

<div class="katalog">
  <?php foreach ($gotovye as $t): ?>
    <article class="tovar">
      <?php if ($t['metka'] !== ''): ?>
        <span class="metka"><?= e($t['metka']) ?></span>
      <?php endif; ?>

      <h3><?= e($t['nazvanie']) ?></h3>
      <p class="cena"><?= $t['cena'] ?> <span>сомони</span></p>
      <p class="nalichie <?= $t['klass'] ?>"><?= e($t['nalichie'])

      <?php if ($t['mozhno_kupit']): ?>
        <button>В корзину</button>
      <?php else: ?>
        <button disabled>Нет в наличии</button>
      <?php endif; ?>
    </article>
  <?php endforeach; ?>
</div>
</main>
</div>
</body>
</html>

```

Обратите внимание на структуру: сверху сплошная логика, снизу чистая разметка. Ни одного `echo` с HTML внутри.

НА ЭКРАНЕ

На экране

Метки «Хит продаж» и «Акция», разные подписи наличия, неактивная кнопка у отсутствующего товара — всё решено на сервере.

Откройте исходный код страницы: там только готовый HTML. Ни закупочных цен, ни коэффициента наценки, ни логики скидок.

ЗАПИСЬ

ЧТО ДЕЛАЕТ

<code>elseif</code>	«Иначе если». Слитно
<code>??</code>	Взять запасное, если не существует или <code>null</code>
<code>?:</code>	Взять запасное, если «пусто» (включая 0 и „“)
<code>↔</code>	Сравнение для сортировки: -1, 0, 1
<code>switch</code> / <code>case</code> / <code>break</code>	Множественный выбор. <code>break</code> обязателен
<code>match</code>	Современная замена <code>switch</code> . Строгое сравнение, возвращает значение
<code>foreach (\$a as \$v)</code>	Перебрать значения
<code>foreach (\$a as \$k => \$v)</code>	Перебрать ключи и значения
<code><?php if (): ?> ... <?php endif; ?></code>	Шаблонный синтаксис
<code>count(\$massiv)</code>	Сколько элементов
<code>&\$t</code>	Ссылка: работать с оригиналом
<code>unset(\$t)</code>	Удалить переменную. Обязательно после цикла со ссылкой

ГРАБЛИ

Грабли

Забывать `break` в `switch`. Выполнение протечёт в следующий вариант. Берите `match`, там этой проблемы нет.

`switch` сравнивает нестрогое. `switch (0)` совпадёт с `case 'abc'` в старых версиях. `match` строгий.

Путать `??` и `?:`. `$page = $_GET['page'] ? : 1` превратит `page=0` в 1.

Смешивать логику и разметку. Сначала посчитайте всё, потом выводите.

Забыть `unset` после `foreach` со ссылкой. Одна из самых коварных ошибок PHP: последний элемент массива непредсказуемо меняется.

Писать HTML внутри `echo`. Кавычки внутри кавычек, экранирование, нечитаемо. Выходите из PHP и пишите разметку как разметку.

ЗАДАЧИ

Задачи

Задача 21.1. Перепишите на `match`:

```
if ($rol === 'admin') $podpis = 'Администратор';
elseif ($rol === 'manager') $podpis = 'Менеджер';
elseif ($rol === 'seller') $podpis = 'Продавец';
else $podpis = 'Покупатель';
```

Задача 21.2. Что выведет? Объясните почему:

```
<?php
$page = 0;
echo $page ?? 5;
echo "\n";
echo $page ?: 5;
```

Задача 21.3. Напишите функцию, которая по статусу заказа возвращает и подпись, и цвет: `new` — «Новый», серый; `paid` — «Оплачен», синий; `shipped` — «Отправлен», жёлтый; `done` — «Доставлен», зелёный; `cancelled` — «Отменён», красный.

Задача 21.4. Перепишите в шаблонном синтаксисе:

```
<?php if ($user) { ?>
    <p>Привет, <?= $user ?></p>
<?php } else { ?>
    <a href="/login">Войти</a>
<?php } ?>
```

Задача 21.5. Сделайте пагинацию: выведите ссылки на страницы с 1 по 10, причём текущая страница — жирным без ссылки.

Задача 21.6. Циклом выведите таблицу умножения от 1 до 9 настоящей HTML-таблицей.

Задача 21.7. Найдите ошибку:

```
<?php
switch ($status) {
    case 'new':
        echo 'Новый';
    case 'paid':
        echo 'Оплачен';
}
```

Задача 21.8. Сделайте каталог из примера, добавьте седьмой товар со статусом `aksiya` и убедитесь, что метка и скидка применились сами.

Задача 21.9. Добавьте вывод: сколько товаров в наличии, сколько под заказ, на какую сумму весь склад.

Ответы — в Приложении Г.

В БОЮ В бою

Откройте `pages/` или `admin/` в этом репозитории и посмотрите на любой шаблон. Увидите ровно эту структуру: наверху логика, ниже разметка с `<?php foreach: ?>` и `<?php endforeach; ?>`.

Найдите обработку статусов заказа — там как раз `match` или `switch` с человеческими подписями. Статусы в базе хранятся короткими кодами (`new`, `paid`, `shipped`), а покупателю показываются по-русски.

Так делают всегда: в базе — код, на экране — перевод. Иначе при смене формулировки пришлось бы обновлять все записи в базе.

ИТОГ

- `if`, `elseif`, `else` — как в JavaScript, но `elseif` слитно.
- `??` — «если не существует или null». Для параметров из адреса — только он.
- `match` лучше `switch`: строгое сравнение, не нужен `break`, возвращает значение.
- `foreach` — главный цикл PHP. Умеет ключи: `as $k => $v`.
- В шаблонах пишите `if (): ... endif;` вместо фигурных скобок.
- Сначала логика, потом разметка. Не смешивайте.
- Не пишите HTML внутри `echo`.
- После `foreach` со ссылкой `&$t` обязателен `unset($t)`.

Дальше — массивы: как хранить каталог и что с ним можно делать.

Массивы

ЗАЧЕМ ЭТО**Зачем эта глава**

Массив — самая используемая вещь в PHP. Каталог товаров, строки из базы данных, данные формы, настройки, результат запроса к чужому API — всё это массивы.

В PHP массивы устроены иначе, чем в JavaScript, и это отличие принципиальное: в PHP массив и объект — одно и то же. Того разделения на `[...]` и `{...}`, к которому вы привыкли, здесь нет.

Разберёмся с этим, а заодно с двумя десятками функций, которые делают за вас почти всю работу.

РАЗБИРАЕМСЯ**Два вида массивов****► Список — как в JavaScript**

```
$brendy = ['Bosch', 'Mann', 'Denso'];  
  
echo $brendy[0];           // Bosch  
echo count($brendy);       // 3
```

Ключи здесь — числа, начиная с нуля. Знакомо.

► Ассоциативный — вместо объектов

```
$tovar = [
    'nazvanie' => 'Тормозные колодки Bosch',
    'artikul' => '0986424815',
    'cena' => 25000,
    'ostatok' => 7,
];

echo $tovar['nazvanie'];
echo $tovar['cena'];
```

Ключи — строки. Это и есть замена объекта из JavaScript.

JAVASCRIPT

```
const t = { cena: 250 }
```

```
t.cena
```

```
t['cena']
```

PHP

```
$t = ['cena' => 250];
```

```
$t['cena']
```

```
$t['cena']
```

⚠ Точка не работает. В PHP точка — это склейка строк. Только квадратные скобки и кавычки: `$tovar['cena']`.

Забыть кавычки — распространённая ошибка:

```
echo $tovar[cena]; // ❌ PHP ищет константу cena
echo $tovar['cena']; // ✅
```

► Массив массивов — каталог

```
$tovary = [
    ['id' => 1, 'nazvanie' => 'Колодки Bosch', 'cena' => 25000, 'brend' => 'Bosch'],
    ['id' => 2, 'nazvanie' => 'Фильтр Mann', 'cena' => 4500, 'brend' => 'Mann'],
    ['id' => 3, 'nazvanie' => 'Свечи Denso', 'cena' => 12000, 'brend' => 'Denso'],
];

echo $tovary[0]['nazvanie']; // Колодки Bosch
echo $tovary[2]['cena']; // 12000
```

Запомните эту структуру. Ровно в таком виде вам придут данные из MySQL в главе 32. Каждая строка таблицы — ассоциативный массив, все строки вместе — список.

РАЗБИРАЕМСЯ

Основные операции

```
$korzina = [];  
  
// Добавить  
$korzina[] = 'Колодки';           // в конец  
$korzina[] = 'Фильтр';  
array_push($korzina, 'Свечи');    // то же самое  
  
// По ключу  
$tovar['цена'] = 26000;           // изменить или добавить  
  
// Удалить  
unset($korzina[1]);               // удалить элемент  
array_pop($korzina);              // убрать последний  
array_shift($korzina);            // убрать первый  
  
// Проверить  
count($korzina);                  // сколько элементов  
empty($korzina);                  // пустой ли  
in_array('Колодки', $korzina);    // есть ли значение  
array_key_exists('цена', $tovar); // есть ли ключ  
isset($tovar['цена']);            // есть ли ключ и не null
```

⚠ Ловушка с `unset`. После удаления элемента из середины ключи **не** перенумеровываются:

```
$a = ['x', 'y', 'z'];  
unset($a[1]);  
print_r($a);    // [0 => 'x', 2 => 'z'] — ключа 1 больше нет!
```

Если нужна сплошная нумерация, перестройте массив:

```
$a = array_values($a);    // [0 ⇒ 'x', 1 ⇒ 'z']
```

Это важно, когда массив уходит в JSON: массив с «дырками» превратится в объект, и JavaScript получит не то, что ждал.

РАЗБИРАЕМСЯ

Перебор

```
// Только значения
foreach ($tovary as $t) {
    echo $t['nazvanie'];
}

// Ключ и значение
foreach ($tovar as $kluch ⇒ $znachenie) {
    echo "$kluch: $znachenie\n";
}

// С номером по порядку
foreach ($tovary as $i ⇒ $t) {
    echo ($i + 1) . ' . ' . $t['nazvanie'];
}
```

► Про ссылки & — и почему лучше без них

```
foreach ($tovary as &$t) {
    $t['cena'] = $t['cena'] * 1.1;
}
unset($t);    // ОБЯЗАТЕЛЬНО
```

Без `&` вы меняете **копию**, и изменения теряются. Со ссылкой — оригинал.

Но `&` — источник знаменитой ловушки PHP. Если забыть `unset($t)`, переменная остаётся ссылкой на последний элемент, и следующий `foreach` его затрёт. Ошибка находится тяжело: массив «сам собой» портится.

Способ надёжнее — собирать новый массив:

```

$s_cenami = [];
foreach ($tovary as $t) {
    $t['cena'] = (int) round($t['cena'] * 1.1);
    $s_cenami[] = $t;
}

```

Или через `array_map` (см. ниже). Ссылки нужны редко — обходитесь без них, пока не появится настоящая причина.

РАЗБИРАЕМСЯ

Функции, которые делают работу за вас

В PHP их несколько сотен, но реально нужны примерно двадцать.

► Фильтрация и преобразование

```

// Отобрать подходящие
$v_nalichii = array_filter($tovary, fn($t) => $t['ostatok'] > 0);

// Превратить каждый
$nazvaniya = array_map(fn($t) => $t['nazvanie'], $tovary);

// Свести к одному значению
$summa = array_reduce($tovary, fn($itog, $t) => $itog + $t['cena'], 0);

```

Знакомо? Это те же `filter`, `map`, `reduce` из главы 15 — только записываются иначе.

JAVASCRIPT

PHP

`arr.filter(fn)`

`array_filter($arr, fn)`

`arr.map(fn)`

`array_map(fn, $arr)` ⚠️ порядок другой!

`arr.reduce(fn, 0)`

`array_reduce($arr, fn, 0)`

⚠️ У `array_map` функция идёт первой, а у `array_filter` — второй. Досадная непоследовательность языка, о которой нужно просто помнить.

`fn($t) => ...` — короткая стрелочная функция (PHP 7.4+). Полная запись:

```
array_filter($tovary, function ($t) {
    return $t['ostatok'] > 0;
});
```

⚠ **array_filter** сохраняет ключи. После фильтрации в массиве могут остаться дырки: `[0 ⇒ ..., 3 ⇒ ..., 5 ⇒ ...]`. Если это мешает — `array_values()`.

► Работа с ключами и значениями

```
array_keys($tovar);           // все ключи
array_values($tovar);         // все значения
array_column($tovary, 'nazvanie'); // колонка из массива массивов
array_column($tovary, 'nazvanie', 'id'); // колонка с ключом по id
array_combine($kluchi, $znacheniya); // собрать массив из двух
```

array_column — незаменимая вещь для работы с данными из базы:

```
// Все названия одной строкой
$nazvaniya = array_column($tovary, 'nazvanie');

// Быстрый поиск по id: превращаем список в справочник
$po_id = array_column($tovary, null, 'id');
echo $po_id[3]['nazvanie']; // мгновенно, без перебора
```

► Сортировка

```
sort($massiv);           // по значениям, ключи теряются
rsort($massiv);          // по убыванию
asort($massiv);          // по значениям, ключи сохраняются
ksort($massiv);          // по ключам

// По полю — самое нужное
usort($tovary, fn($a, $b) => $a['cena'] <=> $b['cena']); // дешёвые пе
usort($tovary, fn($a, $b) => $b['cena'] <=> $a['cena']); // дорогие пе
usort($tovary, fn($a, $b) => strcmp($a['nazvanie'], $b['nazvanie'])); // по наз
```

Вот и пригодился оператор `<=>` из прошлой главы: он возвращает `-1`, `0` или `1` — ровно то, что нужно сортировке.

⚠ **usort** меняет исходный массив, а не возвращает новый. Нужна копия — сделайте её заранее.

► Объединение и части

```
array_merge($a, $b);           // склеить два массива
array_slice($tovary, 0, 20);    // первые 20 — пагинация!
array_unique($brendy);         // убрать повторы
array_search('Bosch', $brendy); // найти ключ по значению
in_array('Bosch', $brendy);     // есть ли такое значение
array_sum($sceny);             // сумма
array_reverse($tovary);        // перевернуть
```

array_slice — это постраничный вывод:

```
$na_stranice = 20;
$stranica = 2;
$kusok = array_slice($tovary, ($stranica - 1) * $na_stranice, $na_stranice);
```

РАЗБИРАЕМСЯ

Массивы и формы

Забегая вперёд в главу 24: данные из формы приходят массивом.

```
$_GET['brend']           // ?brend=bosch
$_POST['name']           // из формы методом POST
$_SESSION['korzina']     // корзина в сессии
```

Это обычные ассоциативные массивы. Всё, что вы выучили, к ним применимо.

Одна тонкость: форма умеет присылать сразу массив значений:

```
<input type="checkbox" name="brendy[]" value="bosch">
<input type="checkbox" name="brendy[]" value="mann">
```

```
$vybrannye = $_POST['brendy'] ?? []; // массив отмеченных
```

Квадратные скобки в `name` — вот что превращает поля в массив.

```
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

function e(string $t): string {
    return htmlspecialchars($t, ENT_QUOTES, 'UTF-8');
}

function somoni(int $dram): string {
    return number_format($dram / 100, 2, '.', ' ');
}

// --- Каталог: так же придут данные из базы ---
$tovary = [
    ['id'⇒1, 'nazvanie'⇒'Тормозные колодки Bosch', 'artikul'⇒'0986424815', 'cer
    ['id'⇒2, 'nazvanie'⇒'Масляный фильтр Mann', 'artikul'⇒'W71280', 'cer
    ['id'⇒3, 'nazvanie'⇒'Свечи зажигания Denso', 'artikul'⇒'IK20', 'cer
    ['id'⇒4, 'nazvanie'⇒'Тормозные диски Brembo', 'artikul'⇒'09.9468.11', 'cer
    ['id'⇒5, 'nazvanie'⇒'Воздушный фильтр Mann', 'artikul'⇒'C25114', 'cer
    ['id'⇒6, 'nazvanie'⇒'Аккумулятор Bosch S4', 'artikul'⇒'0092S40050', 'cer
    ['id'⇒7, 'nazvanie'⇒'Салонный фильтр Mann', 'artikul'⇒'CU2545', 'cer
];

// --- Фильтры (пока задаём прямо здесь, в главе 24 возьмём из формы) ---
$filtr_brend = 'Mann';
$tolko_v_nalichii = true;
$sortirovka = 'cena_vozr';

// --- Обработка ---
$spisok = $tovary;

if ($filtr_brend !== '') {
    $spisok = array_filter($spisok, fn($t) ⇒ $t['brend'] === $filtr_brend);
}

if ($tolko_v_nalichii) {
    $spisok = array_filter($spisok, fn($t) ⇒ $t['ostatok'] > 0);
}

// после array_filter ключи с дырками — выравниваем
$spisok = array_values($spisok);
```



```

match ($sortirovka) {
    'cena_vozr' ⇒ usort($spisok, fn($a, $b) ⇒ $a['cena'] ⇔ $b['cena']),
    'cena_ubyv' ⇒ usort($spisok, fn($a, $b) ⇒ $b['cena'] ⇔ $a['cena']),
    'nazvanie' ⇒ usort($spisok, fn($a, $b) ⇒ strcmp($a['nazvanie'], $b['nazv
    default      ⇒ null,
};

// --- Сводка по всему каталогу ---
$vse_brendy = array_values(array_unique(array_column($tovary, 'brend')));
sort($vse_brendy);

$vsego_shtuk = array_sum(array_column($tovary, 'ostatok'));
$stoimost_sklada = array_reduce(
    $tovary,
    fn($itog, $t) ⇒ $itog + $t['cena'] * $t['ostatok'],
    0
);
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Каталог</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <header>
        <div class="logo">Автозапчасти Firdavs</div>
    </header>

    <main>
        <p class="schetchik">
            Бренды: <?= e(implode(', ', $vse_brendy)) ?> .
            всего на складе <?= $vsego_shtuk ?> шт. на
            <?= somoni($stoimost_sklada) ?> сомони
        </p>

        <p class="schetchik">
            Фильтр: <strong><?= e($filtr_brend ?: 'все бренды') ?></strong>,
            <?= $tolko_v_nalichii ? 'только в наличии' : 'все' ?> .

```

```

        найдено <?= count($spisok) ?>
    </p>

    <div class="katalog">
        <?php foreach ($spisok as $t): ?>
            <article class="tovar">
                <h3><?= e($t['nazvanie']) ?></h3>
                <p class="artikul">Артикул: <?= e($t['artikul']) ?></p>
                <p class="cena"><?= somoni($t['cena']) ?> <span>сомони</span>
                <p class="nalichie">В наличии: <?= $t['ostatok'] ?> шт.</p>
                <button>В корзину</button>
            </article>
        <?php endforeach; ?>
    </div>
</main>
</div>
</body>
</html>

```

НА ЭКРАНЕ

На экране

Три фильтра Mapn, отсортированы по возрастанию цены. Сводка сверху посчитана функциями `array_column`, `array_sum` и `array_reduce` — без единого цикла.

ФУНКЦИЯ	ЧТО ДЕЛАЕТ
<code>['a', 'b']</code>	Список
<code>['kluch' ⇒ 'znachenie']</code>	Ассоциативный массив (замена объекта)
<code>\$t['cena']</code>	Обращение по ключу. Кавычки обязательны
<code>count()</code>	Сколько элементов
<code>\$a[] = \$x</code>	Добавить в конец
<code>unset(\$a[1])</code>	Удалить. Ключи не перенумеруются
<code>array_values()</code>	Перестроить ключи подряд
<code>array_keys()</code>	Все ключи
<code>array_column(\$a, 'pole')</code>	Вытащить колонку
<code>array_column(\$a, null, 'id')</code>	Превратить список в справочник по id
<code>array_filter(\$a, fn)</code>	Отобрать. Сохраняет ключи
<code>array_map(fn, \$a)</code>	Превратить каждый. Функция первым аргументом
<code>array_reduce(\$a, fn, 0)</code>	Свести к одному значению
<code>array_sum()</code>	Сумма
<code>array_unique()</code>	Убрать повторы
<code>array_slice(\$a, \$ot, \$skolko)</code>	Кусок — пагинация
<code>array_merge(\$a, \$b)</code>	Склеить
<code>in_array(\$v, \$a)</code>	Есть ли значение
<code>usort(\$a, fn)</code>	Сортировка по своему правилу. Меняет оригинал
<code>implode(',', ' ', \$a)</code>	Массив → строка
<code>explode(',', ' ', \$s)</code>	Строка → массив
<code>fn(\$x) ⇒ ...</code>	Короткая функция

ГРАБЛИ**Грабли**

Обращаться через точку. `$товар.цена` — это склейка строк. Нужны скобки.

Забыть кавычки в ключе. `$товар[цена]` — PHP ищет константу.

Ждать, что `unset` перенумерует ключи. Не перенумерует. `array_values()`.

Забыть `array_values` после `array_filter`. Массив с дырками при выводе в JSON превратится в объект, и JavaScript сломается.

Перепутать порядок аргументов. `array_map(fn, $a)`, но `array_filter($a, fn)`.

Забыть `unset($t)` после `foreach` со ссылкой. Массив портится.

Считать, что `usort` вернёт новый массив. Он меняет исходный и возвращает `true`.

Обращаться к несуществующему ключу. В PHP 8 это Warning и `null`. Проверяйте через `?? : $t['цена'] ?? 0`.

ЗАДАЧИ**Задачи**

Задача 22.1. Создайте массив из семи товаров. Выведите: все названия, общую стоимость склада, список уникальных брендов.

Задача 22.2. Что выведет?

```
<?php
$a = ['x', 'y', 'z'];
unset($a[1]);
print_r($a);
echo count($a);
print_r(array_values($a));
```

Задача 22.3. Отсортируйте товары по остатку — сначала те, которых больше всего.

Задача 22.4. Через `array_column` получите справочник «артикул → название» и найдите товар по артикулу без перебора.

Задача 22.5. Сделайте пагинацию: разбейте каталог по 3 товара на страницу и выведите вторую страницу.

Задача 22.6. Сгруппируйте товары по брендам: получите массив, где ключ — бренд, а значение — список его товаров.

Задача 22.7. Найдите ошибки:

```
<?php
$tovar = ['cena' => 250];
echo $tovar.cena;
echo $tovar[cena];
$spisok = array_map($tovary, fn($t) => $t['cena']);
```

Задача 22.8. Посчитайте средний чек: общая стоимость склада, делённая на количество позиций. Округлите до целых сомони.

Задача 22.9. Напишите функцию `naiti_po_id($tovary, $id)`, которая возвращает товар или `null`. Двумя способами: циклом и через `array_filter`.

Задача 22.10. Возьмите массив товаров и подготовьте из него данные для выпадающего списка брендов с количеством: «Mann (3)», «Bosch (2)», «Denso (1)».

Ответы — в Приложении Г.

В БОЮ В бою

Откройте любой файл в `includes/` этого репозитория, где обрабатываются товары. Вы увидите массивы ровно такого вида — потому что именно так их отдаёт база данных.

Особенно посмотрите, как формируется прайс: `array_column` для выборки цен, `array_filter` для отсева недоступных, `usort` для сортировки по цене.

Одна практическая деталь: в боевом коде перед `array_filter` часто идёт проверка `if (empty($tovary)) return [];`. Это защита от «пустого» случая — если поставщик не ответил и массив пуст, дальше идти незачем.

Пустые данные — нормальная ситуация, а не исключительная. Проверяйте их всегда.

ИТОГ **Итог**

- В PHP массив заменяет и список, и объект. Разделения нет.
- Обращение — только `$t['kluch']`, точка не работает, кавычки обязательны.
- Массив ассоциативных массивов — то, как приходят данные из базы.
- `array_filter` сохраняет ключи → нужен `array_values()`.
- `array_map(fn, $a)`, но `array_filter($a, fn)` — порядок разный.
- `array_column` превращает список в справочник — мгновенный поиск по id.
- `usort` + `↔` — сортировка по любому полю. **Меняет оригинал.**
- `array_slice` — страничный вывод.
- Ссылки `&$t` лучше избегать; если использовали — обязателен `unset`.
- Проверяйте существование ключа через `??`.

Дальше — функции и подключение файлов: как перестать копировать код.

Функции, `include` и `require`

ЗАЧЕМ ЭТО**Зачем эта глава**

У вас уже несколько страниц, и на каждой одно и то же: шапка, меню, подвал, функция `e()`, функция `somoni()`.

Пока страниц три, копировать терпимо. Когда их станет тридцать, а меню нужно будет поменять — вы поймёте, почему главный принцип программирования звучит так:

***Не повторяйся.** Каждый кусок знания должен быть записан ровно в одном месте.*

В этой главе научимся выносить общее: **функции** для повторяющейся логики и **`include`** для повторяющейся разметки. К концу главы у вашего проекта появится нормальная структура папок — та самая, что в боевом магазине.

РАЗБИРАЕМСЯ**Функции****► Основы — как в JavaScript**

```
function pokazatCenu($cena) {  
    return number_format($cena / 100, 2, '.', ' ') . ' сомони';  
}  
  
echo pokazatCenu(25000);    // 250.00 сомони
```

Всё знакомо: `function`, параметры, `return`.

► Типы — то, чего не было в JavaScript

```
function somoni(int $diram): string {  
    return number_format($diram / 100, 2, '.', ' ');  
}
```

ЧАСТЬ

ЧТО ОЗНАЧАЕТ

`int $diram`

На вход ожидается **целое число**

`: string`

На выходе будет **строка**

Если передать не то, PHP сообщит об ошибке **сразу**, а не через десять строк в непонятном месте.

Указывайте типы всегда. Это самая дешёвая защита от ошибок, какая бывает: пишете на пять символов больше, а ловите целый класс проблем.

Доступные типы: `int`, `float`, `string`, `bool`, `array`, `?string` (строка или null), `void` (ничего не возвращает).

```
function nayti(array $tovary, int $id): ?array {  
    foreach ($tovary as $t) {  
        if ($t['id'] === $id) return $t;  
    }  
    return null;  
}
```

`?array` означает «массив или ничего». Честное описание: функция может не найти товар.

► Значения по умолчанию

```
function dostavka(int $summa, string $gorod = 'Худжанд'): int {  
    if ($gorod === 'Худжанд') return 0;  
    return $summa > 50000 ? 0 : 5000;  
}  
  
echo dostavka(30000);           // 0  
echo dostavka(30000, 'Душанбе'); // 5000
```


⚠ Параметры со значением по умолчанию идут **последними**. Иначе их нельзя пропустить.

► Именованные аргументы (PHP 8)

```
function sozdat_tovar(  
    string $nazvanie,  
    int $cena,  
    int $ostatok = 0,  
    bool $aktivnyi = true,  
    string $brend = ''  
): array {  
    return compact('nazvanie', 'cena', 'ostatok', 'aktivnyi', 'brend');  
}  
  
// Можно указывать только нужное, по именам  
$t = sozdat_tovar(nazvanie: 'Колодки', cena: 25000, brend: 'Bosch');
```

Читается гораздо лучше, чем `sozdat_tovar('Колодки', 25000, 0, true, 'Bosch')` — где попробуй угадай, что означает `true`.

► Область видимости — важное отличие от JavaScript

```
$nacenka = 1.35;  
  
function poschitat($cena) {  
    return $cena * $nacenka;    // ❌ не работает!  
}
```

⚠ Функция в PHP не видит внешние переменные. Совсем. В JavaScript видела, в PHP — нет.

Правильно — передать параметром:

```
function poschitat(int $cena, float $nacenka): int {  
    return (int) round($cena * $nacenka);  
}
```

Есть слово `global`, которое даёт доступ к внешней переменной. **Не пользуйтесь им.** Функция, зависящая от глобального состояния, ведёт себя непредсказуемо: работает или нет в зависимости от того, что происходило раньше.

Хорошая функция зависит только от своих параметров. Такую легко проверить, легко переиспользовать и легко понять.

► Одна функция — одно дело

```
// ❌ Делает три вещи
function obrabotat_zakaz($dannye) {
    // проверяет
    // сохраняет в базу
    // отправляет письмо
}

// ✅ Три функции
function proverit_zakaz(array $d): array { ... }
function sohranit_zakaz(array $d): int { ... }
function otpravit_pismo(int $id): bool { ... }
```

Признак, что пора разделять: в названии функции просится «и».

РАЗБИРАЕМСЯ

include и require — подключение файлов

► Проблема

Шапка сайта на десяти страницах. Меняем телефон — правим десять файлов. Один забыли — на сайте два разных телефона.

► Решение

Выносим шапку в отдельный файл `includes/header.php` и подключаем:

```
<?php require 'includes/header.php'; ?>

<h1>Каталог</h1>

<?php require 'includes/footer.php'; ?>
```

Теперь шапка в одном месте.

► Четыре формы подключения

ЧТО ПИСАТЬ	ЕСЛИ ФАЙЛА НЕТ
<code>include</code>	Предупреждение, работа продолжается
<code>require</code>	Фатальная ошибка , всё останавливается
<code>include_once</code>	То же, но не подключит повторно
<code>require_once</code>	То же, но не подключит повторно

Правило:

- Файл **необходим** для работы (настройки, подключение к базе, функции) — `require_once`.
- Файл **желателен** (баннер, необязательный блок) — `include`.

`_once` защищает от двойного подключения. Если файл с функциями подключить дважды, PHP выдаст «функция уже объявлена» и упадёт. С `require_once` этого не случится.

На практике `require_once` покрывает почти все случаи.

► Пути — где ошибаются все

```
require 'includes/header.php';           // ⚠ относительно чего?
require __DIR__ . '/includes/header.php'; // ✅ надёжно
```

Проблема относительных путей: они считаются от файла, который **запущен**, а не от того, где написана строчка. Подключили файл из подпапки — и путь внутри него сломался.

`__DIR__` — это папка, где лежит **текущий** файл. С ним путь всегда верный, откуда бы файл ни подключили.

Ещё удобнее — задать корень проекта один раз:

```
// includes/config.php
define('KOREN', __DIR__ . '/../');

// в любом файле
require_once KOREN . '/includes/functions.php';
```

Вот как выглядит нормально организованный проект:

```
moi-magazin/
├─ includes/
│   ├── config.php      настройки
│   ├── functions.php   общие функции
│   ├── header.php      шапка + меню
│   └─ footer.php       подвал
├─ assets/
│   ├── css/style.css
│   └─ js/script.js
├─ pages/
│   ├── catalog.php
│   ├── tovar.php
│   └─ contacts.php
└─ index.php
```

Знакомо? Так устроен и autodoc.tj, код которого лежит рядом с этой книгой.

► `includes/config.php` — настройки

```
<?php
// Всё, что может понадобится поменять, — здесь

define('SAIT_NAZVANIE', 'Автозапчасти Firdavs');
define('SAIT_GOROD', 'Худжанд');
define('SAIT_TELEFON', '+992 92 777 77 77');
define('SAIT_POCHTA', 'info@autodoc.tj');

define('NACENKA', 1.35);
define('DOSTAVKA_DIRAM', 1200);
define('BESPLATNO_OT_DIRAM', 100000);

define('KOREN', __DIR__ . '/../');
```

⚠ Пароли и ключи API в такой файл не пишут, если проект лежит в git. Их выносят в отдельный файл, который в репозиторий не попадает. Разберём в главе 32.

```

<?php

/** Экранирование для безопасного вывода. Короткое имя — потому что используется
function e(?string $text): string {
    return htmlspecialchars((string) $text, ENT_QUOTES, 'UTF-8');
}

/** Дирамы в читаемые сомони. Деньги внутри системы всегда целые. */
function somoni(int $diram): string {
    return number_format($diram / 100, 2, '.', ' ');
}

/** Цена для покупателя из закупочной. */
function cena_prodazhi(int $zakup_diram): int {
    return (int) round($zakup_diram * NACENKA);
}

/** Доставка: бесплатно от определённой суммы. */
function cena_dostavki(int $summa_diram): int {
    return $summa_diram ≥ BESPLATNO_OT_DIRAM ? 0 : DOSTAVKA_DIRAM;
}

/** Правильное окончание: 1 товар, 2 товара, 5 товаров. */
function sklonenie(int $n, string $odin, string $dva, string $mnogo): string {
    $n = abs($n) % 100;
    if ($n ≥ 11 && $n ≤ 19) return $mnogo;
    $n = $n % 10;
    if ($n ≡ 1) return $odin;
    if ($n ≥ 2 && $n ≤ 4) return $dva;
    return $mnogo;
}

/** Найти товар по id. Возвращает null, если не нашёлся. */
function nayti_tovar(array $tovary, int $id): ?array {
    foreach ($tovary as $t) {
        if ($t['id'] ≡ $id) return $t;
    }
    return null;
}

```

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><?= e($zagolovok ?? SAIT_NAZVANIE) ?></title>
  <link rel="stylesheet" href="/assets/css/style.css">
</head>
<body>
<div class="container">
  <header>
    <div>
      <div class="logo"><?= e(SAIT_NAZVANIE) ?></div>
      <div class="slogan"><?= e(SAIT_GOROD) ?></div>
    </div>
    <nav>
      <ul>
        <li><a href="/">Главная</a></li>
        <li><a href="/pages/catalog.php">Каталог</a></li>
        <li><a href="/pages/contacts.php">Контакты</a></li>
      </ul>
    </nav>
  </header>
  <main>
```

Обратите внимание на `$zagolovok ?? SAIT_NAZVANIE` : страница может задать свой заголовок, а если не задала — возьмётся название сайта.

Приём простой, но именно так шаблон становится **гибким**: общий код один, а отличия задаются переменными.

► includes/footer.php — подвал

```
</main>

<footer>
    <p><?= e(SAIT_GOROD) ?> .
        <a href="tel:<?= preg_replace('/\D/', '', SAIT_TELEFON) ?>"><?= e(SA
        <a href="mailto:<?= e(SAIT_POCHTA) ?>"><?= e(SAIT_POCHTA) ?></a></p>
    <p>&copy; <?= date('Y') ?> <?= e(SAIT_NAZVANIE) ?></p>
</footer>
</div>
</body>
</html>
```

`date('Y')` подставит текущий год — не придётся править копирайт каждый январь.

► Страница становится короткой

```
<?php
require_once __DIR__ . '/includes/config.php';
require_once __DIR__ . '/includes/functions.php';

$zagolovok = 'Каталог — ' . SAIT_NAZVANIE;
$tovary = [ /* ... */ ];

require __DIR__ . '/includes/header.php';
?>

<h1>Каталог запчастей</h1>

<div class="katalog">
    <?php foreach ($tovary as $t): ?>
        <article class="tovar">
            <h3><?= e($t['nazvanie']) ?></h3>
            <p class="cena"><?= somoni(cena_prodazhi($t['zakup
```